

Notas de Aulas de Compiladores

Universidade Federal de Ouro Preto

Rodrigo Ribeiro

Sumário

I	Introdução	13
1	Introdução à construção de compiladores	14
1.1	Introdução	14
1.2	O desafio da compilação	14
1.3	Exemplo de passos feitos por um compilador	15
1.4	Estrutura de um compilador	16
1.5	Análise léxica	17
1.6	Análise sintática	17
1.7	Análise semântica	17
1.8	Geração de código	18
1.9	Otimização de código	18
1.10	Visão geral de um compilador	19
1.11	Conclusão	19
II	Análise léxica	20
2	Introdução à análise léxica	21
2.1	Introdução	21
2.2	A linguagem Exp	21
2.3	Analísadores léxicos ad-hoc	21
2.4	Analísadores léxicos ad-hoc: vantagens e desvantagens	23
2.5	Conclusão	24
3	Análise léxica baseada em expressões regulares	25
3.1	Introdução	25
3.2	Autômatos finitos determinísticos	25
3.3	Autômatos finitos não determinísticos	26
3.4	Expressões regulares	28
3.5	Gerando AFNs a partir de ERs.	30
3.6	Critério do maior prefixo	33
3.7	Derivadas de expressões regulares	34
3.8	Implementação em Haskell	36
3.9	Conclusão	37
4	Geradores de analisadores léxicos	38

<i>SUMÁRIO</i>	3
4.1 Introdução	38
4.2 Construindo um analisador léxico utilizando o Alex	38
4.2.1 Definindo os tokens	38
4.2.2 Sintaxe de expressões regulares no Alex	38
4.2.3 Estrutura de especificações Alex	39
4.3 Conclusão	40
III Análise sintática	41
5 Introdução à análise sintática	42
5.1 Introdução	42
5.2 Gramáticas livres de contexto	42
5.2.1 Representando precedência e associatividade	44
5.2.2 Ambiguidade do else vazio	44
5.2.3 Remoção de Recursão à Esquerda	45
5.2.4 Fatoração à Esquerda	46
5.2.5 Árvores de sintaxe abstrata	46
5.2.6 Outros problemas na análise sintática	47
5.3 Conclusão	48
6 Análise descendente recursiva	49
6.1 Introdução	49
6.2 Combinadores de parsing	49
6.2.1 O tipo Parser	49
6.2.2 Instâncias de classes de tipos	50
6.2.3 Combinadores primitivos	50
6.2.4 Combinadores de alto nível	51
6.2.5 Aplicações para tipos concretos	52
6.2.6 Combinadores para encadeamento de operadores	52
6.3 O parser de expressões	53
6.3.1 Tokens e análise léxica	53
6.3.2 A árvore de sintaxe abstrata	54
6.3.3 O combinador gen para gramáticas com operadores	54
6.3.4 O analisador sintático	54
6.4 O parser de expressões com Megaparsec	55
6.4.1 O tipo Parser no Megaparsec	56
6.4.2 A análise léxica com Megaparsec	56
6.4.3 O analisador sintático com makeExprParser	57
6.4.4 A função principal de parsing	57
6.5 Conclusão	58
7 Parsing expression grammars	59
7.1 Introdução	59

7.2	Sintaxe e semântica de PEGs	59
7.2.1	Predicados	60
7.2.2	Repetição	60
7.2.3	Terminação em PEGs	60
7.3	Implementação em Haskell	61
7.3.1	O tipo Result	61
7.3.2	O tipo PExp	61
7.3.3	Instâncias e combinadores	61
7.3.4	O combinador try	62
7.3.5	A escolha ordenada	62
7.3.6	O predicado not	63
7.3.7	Combinadores léxicos	63
7.4	Exemplo: parser de expressões	63
7.5	PEGs e GLCs: principais diferenças	64
7.6	Conclusão	64
8	Análise sintática LL(1)	66
8.1	Introdução	66
8.2	Exemplo Informal	66
8.3	Construção da tabela LL(1)	67
8.3.1	Anulável	67
8.3.2	Conjunto FIRST	67
8.3.3	Conjunto FOLLOW	68
8.3.4	Algoritmo para construção da tabela de parsing	68
8.4	Algoritmo de Análise Sintática LL(1)	68
8.5	Exemplo de Construção de Tabela	69
8.5.1	Gramática LL(1): Expressões Aritméticas	69
8.5.2	Gramática não-LL(1): Conflito FIRST/FIRST	70
8.5.3	Gramática não-LL(1): Conflito FIRST/FOLLOW	70
8.6	A Implementação em Haskell	70
8.6.1	Representação da Gramática	71
8.6.2	Conjuntos FIRST (Parsing.First)	71
8.6.3	Conjuntos FOLLOW (Parsing.Follow)	71
8.6.4	Construção da Tabela (Parsing.LL.BuildTable)	72
8.6.5	O Parser (Parsing.LL.Parser)	73
8.7	Conclusão	74
9	Introdução à análise ascendente	75
9.1	Introdução	75
9.2	Algoritmo de Earley	75
9.2.1	Exemplo	76
9.3	Analísadores LR(0) e SLR	78
9.4	Implementação	81

9.4.1	Algoritmo de Earley	81
9.4.2	LR(0) e SLR	82
9.5	Conclusão	85
10	Análise sintática LR(1) e LALR	86
10.1	Introdução	86
10.2	O Algoritmo LR(1)	87
10.2.1	Itens LR(1)	87
10.2.2	Fechamento LR(1)	87
10.2.3	Função goto LR(1)	87
10.2.4	Construção da Tabela LR(1)	87
10.3	Exemplo: Tabela LR(1) para Expressões Aritméticas	88
10.3.1	Construção dos estados	88
10.3.2	Tabela LR(1)	89
10.4	O Algoritmo LALR(1)	90
10.4.1	Motivação	90
10.4.2	Fusão de Estados	90
10.4.3	Construção da Tabela LALR(1)	91
10.5	Exemplo: Tabela LALR(1) para Expressões Aritméticas	91
10.5.1	Identificação de estados com o mesmo núcleo	91
10.5.2	Estados LALR resultantes	91
10.5.3	Tabela LALR(1)	92
10.6	Implementação	92
10.6.1	LR(1)	92
10.6.2	LALR(1)	94
10.7	Comparando algoritmos de análise sintática	95
10.8	Conclusão	96
11	Geradores de analisadores sintáticos	97
11.1	Introdução	97
11.2	Estrutura de uma especificação Happy	97
11.2.1	Cabeçalho	97
11.2.2	Diretivas	98
11.2.3	Declaração de tokens	98
11.2.4	Declarações de precedência	99
11.2.5	Regras gramaticais	99
11.2.6	Rodapé	99
11.3	Gerando o parser e inspecionando a tabela	100
11.3.1	A opção -i	100
11.4	Conflitos shift-reduce	100
11.5	Conflitos reduce-reduce	101
11.6	Conclusão	102

IV Semântica formal e interpretadores	103
12 Introdução à semântica operacional	104
12.1 Introdução	104
12.2 A Linguagem de Expressões	104
12.3 Semântica Small-Step	105
12.3.1 Valores	105
12.3.2 Regras de Redução	105
12.3.3 Sequências de Redução	106
12.3.4 Exemplo: Small-Step	106
12.4 Semântica Big-Step	106
12.4.1 Regras de Avaliação	106
12.4.2 Exemplos: Big-Step	107
12.5 Implementação em Haskell	107
12.5.1 Executando o Interpretador	107
12.5.2 Totalidade e Terminação	108
12.6 Conclusão	108
13 Semântica operacional para estado	109
13.1 Introdução	109
13.2 A Linguagem Line	109
13.3 Ambientes	110
13.4 Semântica Small-Step	110
13.4.1 Expressões	110
13.4.2 Comandos	110
13.4.3 Exemplo: Small-Step	111
13.5 Semântica Big-Step	111
13.5.1 Expressões	112
13.5.2 Comandos	112
13.5.3 Exemplos: Big-Step	113
13.6 Implementação em Haskell	113
13.6.1 O Ambiente como Mônada de Estado	113
13.6.2 Interpretador de Expressões	113
13.6.3 Interpretador de Comandos	114
13.6.4 Executando o Interpretador	114
13.7 Conclusão	115
14 Semântica operacional para uma linguagem imperativa simples	116
14.1 Introdução	116
14.2 A Linguagem While	116
14.3 Ambientes	117
14.4 Semântica Small-Step	117
14.4.1 Expressões	117

14.4.2	Comandos	118
14.4.3	Exemplo: Small-Step	118
14.4.4	Extensão TImp: Semântica Small-Step	119
14.5	Semântica Big-Step	120
14.5.1	Expressões	120
14.5.2	Comandos	120
14.5.3	Exemplos: Big-Step	120
14.5.4	Extensão TImp: Semântica Big-Step	121
14.6	Implementação em Haskell	123
14.6.1	Expressões	123
14.6.2	Condicional	124
14.6.3	Repetição	124
14.6.4	Executando o Interpretador	125
14.6.5	Implementação da Extensão TImp	125
14.7	Conclusão	127
V	Sistemas e verificação de tipos	128
15	Introdução aos sistemas de tipos	129
15.1	Introdução	129
15.2	Tipos em Linguagens de Programação	129
15.2.1	Tipagem Estática e Dinâmica	129
15.2.2	Tipagem Forte e Fraca	129
15.3	A Linguagem de Expressões Tipadas	130
15.4	O Sistema de Tipos	130
15.4.1	Regras de Tipagem	130
15.4.2	Exemplos de Derivações de Tipo	131
15.4.3	Unicidade de Tipos	131
15.5	Semântica Small-Step	132
15.5.1	Regras de Redução para Booleanos	132
15.5.2	Regras de Redução para Naturais	132
15.5.3	Formas Presas	132
15.5.4	Exemplo: Small-Step	133
15.6	Semântica Big-Step	133
15.6.1	Regras para Booleanos	133
15.6.2	Regras para Naturais	133
15.6.3	Exemplo: Big-Step	133
15.7	Corretude do Sistema de Tipos	134
15.7.1	Lema de Preservação	134
15.7.2	Lema de Progresso	135
15.7.3	Da Preservação e do Progresso à Corretude	136
15.8	Sistema de Tipos para TLine	136

15.8.1	Regras de Tipagem para Expressões	136
15.8.2	Regras de Tipagem para Comandos	137
15.9	Sistema de Tipos para TWhile	137
15.9.1	Regras de Tipagem para os Novos Comandos	137
15.10	Sistema de Tipos para Funções e Registros	137
15.10.1	Tipos, Contextos e Ambientes	137
15.10.2	Regras de Tipagem para Registros	138
15.10.3	Regras de Tipagem para Funções	138
15.10.4	Exemplo de Derivação de Tipo	139
15.10.5	Extensão da Corretude	139
15.11	Conclusão	140
16	Verificação de tipos	141
16.1	Introdução	141
16.2	Verificação de Tipos para TExp	141
16.2.1	A Linguagem e o seu Sistema de Tipos	141
16.2.2	Da Regra à Equação	141
16.2.3	O Verificador Completo	142
16.3	Verificação de Tipos para TLine	143
16.3.1	A Linguagem TLine	143
16.3.2	O Contexto de Tipos	143
16.3.3	O Algoritmo de Verificação	144
16.4	Verificação de Tipos para TWhile	145
16.4.1	Extensões à Linguagem	145
16.4.2	Escopo de Variáveis	145
16.4.3	As Novas Equações do Verificador	145
16.5	Verificação de Tipos para TImp	146
16.5.1	A Mônada de Verificação	146
16.5.2	Regras e Implementação: Registros	147
16.5.3	Regras e Implementação: Funções	148
16.6	Conclusão	149
VI	Geração de código intermediário	150
17	Geração de Código Intermediário	151
17.1	Introdução	151
17.2	Propriedades Desejáveis	151
17.3	Abordagens Principais	151
17.4	A Representação Intermediária em Árvore (IRT)	152
17.4.1	Sintaxe das Expressões	152
17.4.2	Sintaxe dos Comandos	153
17.5	Semântica Operacional Big-Step da IRT	154

17.5.1	Estado e Valores	154
17.5.2	Semântica das Expressões	154
17.5.3	Semântica dos Comandos	155
17.6	Tradução Dirigida por Sintaxe	156
17.6.1	Tradução de Expressões	156
17.6.2	Tradução de Comandos	157
17.6.3	Tradução de Estruturas de Controle	158
17.6.4	Tradução de Funções	159
17.7	Exemplo Completo	159
17.8	Formas Canônicas	159
17.9	Implementação em Haskell: Geração de Código para TWhile	160
17.9.1	Mônada de Geração de Código	160
17.9.2	Ponto de Entrada	161
17.9.3	Geração de Comandos	161
17.9.4	Geração de Expressões	163
17.9.5	Exemplo: Fatorial Iterativo	163
17.9.6	Integração no Pipeline	164
17.10	Geração de Código para TImp: Funções e Registros	164
17.10.1	Representação de Registros na Memória	164
17.10.2	Tradução Dirigida por Sintaxe para Registros	165
17.10.3	Tradução de Funções	165
17.10.4	Implementação em Haskell	166
17.10.5	Exemplo Completo	168
17.11	Conclusão	169

Parte I

Introdução

Capítulo 1

Introdução à construção de compiladores

1.1 Introdução

Neste capítulo apresentamos uma visão geral da construção de compiladores, focando em suas diferentes etapas e nos diferentes resultados produzidos por cada uma delas.

1.2 O desafio da compilação

Um compilador é um programa que traduz uma linguagem, denominada origem, para outra, a linguagem de destino. A tradução não é uma tarefa fácil quando as linguagens de origem e de destino são muito diferentes. Em um compilador clássico, a linguagem de origem é uma linguagem de programação projetada para ser compreensível por humanos, enquanto a linguagem de destino é uma linguagem assembly ou linguagem de máquina projetada para ser executada de forma eficiente pelo hardware. No entanto, outras traduções são úteis e utilizadas na prática. Por exemplo, o compilador Java compila o código-fonte Java para código da Máquina Virtual Java (JVM) (chamado *bytecode*), e o sistema de tempo de execução da JVM possui um segundo compilador, *just-in-time* (JIT), que traduz o bytecode executado com frequência em código de máquina que o processador pode executar diretamente.

Precisamos de um compilador para executar programas? Não, os programas podem ser executados usando um interpretador. Por exemplo, o bytecode Java é executado por um interpretador JVM que simula a execução de uma máquina hipotética. No entanto, os interpretadores de software são muito menos eficientes do que o hardware de um processador, portanto, o código compilado tende a ser executado pelo menos dez vezes mais rápido do que o código interpretado. (Na verdade, um processador é simplesmente uma implementação de hardware de um interpretador.) Essa vantagem de desempenho também significa que o código compilado consome dez vezes menos energia e gera dez vezes menos calor. Claro, a etapa de compilação do código em si pode exigir muito poder computacional, então, se você for executar um programa apenas uma vez, é melhor interpretá-lo. A compilação faz sentido para programas que serão executados muitas vezes.

O código-fonte e o código-alvo são geralmente bastante diferentes. O código-fonte é otimizado para que humanos o leiam e modifiquem. Ele tenta, até certo ponto, corresponder às noções humanas de linguagem e possui redundância para ajudar a prevenir erros de programação. O código-fonte suporta análise e raciocínio, seja por humanos ou por algoritmos. O código-alvo, por outro lado, geralmente é otimizado para interpretação eficiente. Ele é compacto, pouco legível e não desperdiça espaço com redundância. O código-alvo frequentemente descarta grande parte da estrutura de alto nível do código-fonte, tornando-o difícil de interpretar.

Mais importante para os compiladores do que o desempenho do código gerado é a sua correção. Depurar código já é bastante difícil quando se pode confiar no compilador; se os erros podem ter sido inseridos pelo próprio compilador, a depuração torna-se verdadeiramente árdua. As diferenças entre as linguagens de origem e de destino também dificultam demonstrar que o compilador está a executar a sua função corretamente. Curiosamente, é possível definir com precisão matemática

o que significa um compilador estar correto, desde que exista uma semântica definida tanto para a linguagem de origem como para a de destino: uma descrição matemática do comportamento de ambas as linguagens. De facto, vários compiladores foram desenvolvidos juntamente com provas de correção, inclusive provas que podem ser verificadas automaticamente para garantir que foram construídas corretamente. Desenvolver compiladores com este elevado grau de garantia não é o nosso objetivo aqui, mas a abordagem que adotamos, de especificar precisamente diferentes partes do processo de compilação, é útil para atingir esse objetivo.

1.3 Exemplo de passos feitos por um compilador

Para ter uma ideia do trabalho que um compilador realiza, considere a seguinte função curta implementada na linguagem de origem que usaremos neste curso. Sua sintaxe é semelhante à de várias linguagens populares, portanto, deve ser fácil de entender.

```
findIndex(a:int[]): int {
    n:int = length a
    i:int = 0
    while i < n {
        if a[i] == i+1 {
            return i
        }
        i = i + 1
    }
    return -1
}
```

Um compilador não otimizador poderia traduzir o código acima para código assembly x86-64, como mostrado abaixo. As variáveis do programa são armazenadas na pilha de execução e acessadas por meio de operandos de memória. Por exemplo, a variável `|a|` é armazenada no endereço de memória `[|rbp - 16|]`, onde `|rbp|` é um registrador que aponta para o topo do frame de pilha atual. Também é importante notar que a estrutura de controle do código-fonte de alto nível, como a instrução `|while|`, foi substituída por instruções de desvio de baixo nível, como `[|jne|]` e `[|jmp|]`.

```
findIndex:
    push    rbp
    mov rbp, rsp
    mov [rbp - 16], rdi
    mov rax, [rbp - 16]
    mov rax, [rax - 8]
    mov [rbp - 24], rax
    mov qword ptr [rbp - 32], 0
L1:    mov rax, [rbp - 32]
    cmp rax, [rbp - 24]
    jge L6
    mov rax, [rbp - 16]
    mov rcx, [rbp - 32]
    mov rax, [rax + 8*rcx]
    mov rcx, [rbp - 32]
    add rcx, 1
    cmp rax, rcx
    jne L4
    mov rax, [rbp - 32]
    mov [rbp - 8], rax
    jmp L7
L4:    jmp L5
```

```

L5:    mov rax, [rbp - 32]
        add rax, 1
        mov [rbp - 32], rax
        jmp L1
L6:    mov qword ptr [rbp - 8], -1
L7:    mov rax, [rbp - 8]
        pop rbp
        ret

```

O código não otimizado é mais ineficiente do que o código que um programador de assembly experiente poderia produzir, mas um bom compilador otimizador geralmente consegue igualar ou melhorar o código escrito manualmente, gerando um código como o mostrado a seguir. As variáveis são armazenadas inteiramente em registradores, o compilador fez um trabalho melhor ao escolher instruções eficientes e as instruções associadas ao gerenciamento da pilha foram removidas, pois a pilha não é utilizada.

```

findIndex:
    mov rcx, [rdi - 8]
    xor edx, edx
L1:    cmp rcx, rdx
        je L2
        lea rax, [rdx + 1]
        cmp rax, [rdi + 8*rdx]
        mov rdx, rax
        jne L1
        dec rax
        ret
L2:    mov rax, -1
        ret

```

O compilador geralmente não termina a tarefa de gerar o código de máquina; essa etapa é deixada para o montador (assembler). O resultado da montagem do código assembly otimizado acima é a seguinte sequência de bytes de código de máquina, expressa em hexadecimal, com suas posições mostradas em hexadecimal na coluna da esquerda e o código assembly correspondente mostrado à direita. Observe que os rótulos no código assembly foram substituídos por deslocamentos hexadecimais.

0: 48 8b 4f f8	mov rcx, [rdi - 8]
4: 31 d2	xor edx, edx
6: 48 39 d1	cmp rcx, rdx
9: 74 11	je 0x1c
b: 48 8d 42 01	lea rax, [rdx + 1]
f: 48 3b 04 d7	cmp rax, [rdi + 8*rdx]
13: 48 89 c2	mov rdx, rax
16: 75 ee	jne 0x6
18: 48 ff c8	dec rax
1b: c3	ret
1c: 48 c7 c0 ff ff ff ff	mov rax, -1
23: c3	ret

1.4 Estrutura de um compilador

Pode parecer quase mágico que seja possível traduzir código-fonte para código de baixo nível eficiente e correto, e ainda mais surpreendente que isso possa ser feito com relativa rapidez. A principal ideia que torna a tradução de alta qualidade viável é não fazê-la de uma só vez. A maneira de resolver um

problema complexo geralmente é dividi-lo em problemas menores e mais fáceis, e com os compiladores não é diferente.

Em vez de traduzir diretamente do código-fonte (um fluxo de bytes) para o código-alvo (outro fluxo de bytes), definimos uma sequência de fases de compilação, cada uma representando o programa original de uma maneira diferente. Essencialmente, entre as linguagens de origem e destino, definimos uma sequência de linguagens intermediárias que se situam entre a origem e o destino, e são projetadas para a conveniência da fase de compilação atual.

Para entender melhor como isso funciona, vejamos como um trecho simples de um programa é representado em cada fase do compilador:

```
if (val >= 0) pos = val
```

1.5 Análise léxica

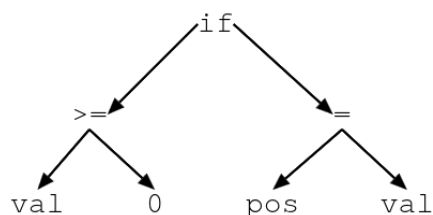
O primeiro passo na compilação é transformar o programa de entrada, representado como uma sequência de bytes ou caracteres, em uma estrutura de dados em árvore que suporte as fases posteriores do compilador. A análise léxica divide a entrada em uma sequência de tokens (ou lexemas). Essas são as “palavras” do programa. Espaços em branco e outros separadores de tokens são descartados.

A imagem abaixo ilustra o resultado da análise léxica para o trecho de programa apresentado anteriormente.



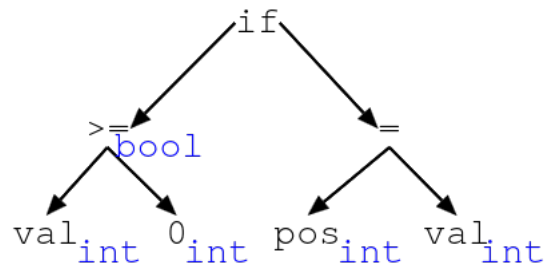
1.6 Análise sintática

O próximo passo é a análise sintática, que cria uma estrutura em árvore que captura a sintaxe do programa, descartando elementos sintáticos como parênteses que não afetam o significado do programa. O resultado desta fase é uma árvore sintática abstrata. Na figura seguinte, apresentamos a árvore de sintaxe para nosso programa de exemplo.



1.7 Análise semântica

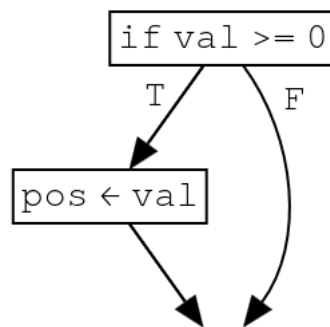
A análise semântica completa a tarefa de determinar se o código-fonte representa um programa válido. Frequentemente, envolve a verificação de tipos. Informações adicionais sobre o programa podem ser coletadas durante essa fase, como os tipos de expressões do programa. Essas informações podem ser usadas para enriquecer a árvore sintática abstrata. Na figura seguinte, apresentamos a árvore de sintaxe contendo informação de tipos sobre todos os elementos do código.



1.8 Geração de código

Após as etapas de análise, o compilador gera novas representações do programa. Normalmente, há uma fase inicial de geração de código que produz uma representação intermediária (RI) do programa. Uma fase posterior do compilador gera o código alvo. Quando a linguagem alvo é código assembly, essa fase é chamada de seleção de instruções.

Uma representação intermediária comum é um *grafo de fluxo de controle*. A representação como grafo de fluxo de controle do trecho de código exemplo é apresentada na figura abaixo.



Uma fase posterior da geração de código é a *seleção de instruções*, na qual a representação intermediária é traduzida em instruções assembly. As instruções podem ser geradas como código abstrato, no qual as variáveis são utilizadas como se fossem registradores de hardware. Para o nosso exemplo, o resultado da tradução para código de abstrato (x86-64) poderia ser o seguinte:

```

    cmp val, 0
    jl L1
    mov pos, val
L1:

```

1.9 Otimização de código

A otimização de código é uma etapa cujo objetivo é melhorar a eficiência de programas. Normalmente, “melhor” significa “mais rápido”, embora outros objetivos também sejam buscados, como reduzir o uso de memória ou o consumo de energia do programa. A otimização pode ser aplicada em praticamente todas as fases do compilador.

A otimização mais importante é a alocação de registradores, que atribui variáveis à registradores de hardware. Aplicar a alocação de registradores ao código assembly acima poderia gerar o seguinte código assembly x86-64:

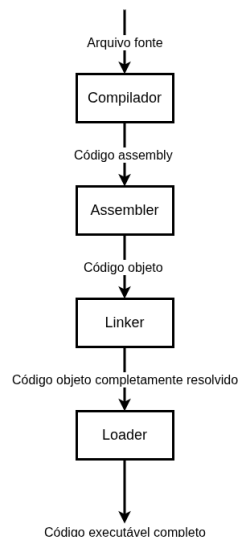
```
    cmp rax, 0
    jl L1
    mov rcx, rax
L1:
```

Outra otimização que pode ser feita é usar uma sequência de instruções, mais eficiente, que evite desvios.

```
cmp rax,0
cmovge rcx, rax
```

1.10 Visão geral de um compilador

O compilador geralmente executa apenas uma parte da tarefa de produzir código executável. Após gerar o código assembly para cada uma das unidades de compilação (arquivos-fonte), um assembler é utilizado para produzir módulos de código objeto que possuem dependências entre si. Um linker resolve as dependências entre os arquivos objeto e determina como modificar ou estender o código assembly para que essas dependências sejam satisfeitas. Um loader, frequentemente integrado ao sistema operacional, carrega então a saída do linker na memória e produz uma imagem executável que o processador pode interpretar.



1.11 Conclusão

Neste capítulo, apresentamos uma visão geral do que é um compilador e do desafio fundamental que ele enfrenta: traduzir programas de uma linguagem de origem, projetada para seres humanos, para uma linguagem de destino, otimizada para execução eficiente por hardware. Vimos que, embora os compiladores sejam sistemas complexos, é possível dominar essa complexidade dividindo o problema em fases menores e mais gerenciáveis: análise léxica, sintática, semântica, geração de código e otimização. Cada uma transformando gradualmente o programa de sua representação original até o código de máquina final. Ao longo deste material, exploraremos cada uma dessas fases em detalhe, sempre buscando descrevê-las com precisão para que possamos não apenas compreender seu funcionamento, mas também implementá-las corretamente.

Parte II

Análise léxica

Capítulo 2

Introdução à análise léxica

2.1 Introdução

A análise léxica é a primeira fase do processo de compilação, responsável por ler o código-fonte como uma sequência de caracteres e agrupá-los em tokens significativos - palavras reservadas, identificadores, operadores, números e símbolos - enquanto remove elementos irrelevantes como espaços em branco e comentários. Sua importância está em transformar o texto bruto do programa em uma representação estruturada que simplifica as fases subsequentes (análise sintática e semântica), além de detectar erros léxicos precocemente, como caracteres inválidos ou identificadores mal formados, funcionando como a “porta de entrada” que organiza e valida o string de entrada antes das demais etapas de um compilador / interpretador.

Damos o nome de **token** a um componente indivisível da sintaxe de uma linguagem. De maneira simples, o conjunto de tokens de uma linguagem pode ser entendido como o seu alfabeto.

Antes de apresentarmos a primeira técnica de análise léxica, vamos descrever a primeira linguagem que iremos utilizar durante o curso.

2.2 A linguagem Exp

Durante o curso de compiladores vamos implementar interpretadores e compiladores para diversas linguagens de programação. A primeira destas linguagens será chamada de **Exp** e está é definida pela seguinte gramática livre de contexto.

$$E \rightarrow n \mid E + E \mid E * E \mid (E)$$

De maneira simples, a linguagem **Exp** é formada por expressões envolvendo adição, multiplicação, parêntesis e constantes numéricas (representadas por **n**).

O conjunto de tokens desta linguagem é dado por $\{n, +, *, (,)\}$. A próxima seção vamos apresentar a implementação de um analisador léxico ad-hoc para esta linguagem.

2.3 Analisadores léxicos ad-hoc

Um analisador léxico ad-hoc é uma implementação manual e simplificada de um analisador léxico, construída especificamente para um problema ou linguagem particular, sem utilizar geradores automáticos como o Alex. Diferentemente de analisadores baseados em expressões regulares e autômatos (tema de um próximo capítulo), o analisador ad-hoc é tipicamente implementado através de código que processa caracteres sequencialmente, utilizando estruturas como casamento de padrão para identificar tokens.

O primeiro passo de nossa implementação é a definição de tipos para representação de tokens.

```
type Line = Int
type Column = Int
```

```
data Token
  = Token {
    pos :: (Line, Column)
    , lexeme :: Lexeme
  }
```

```
data Lexeme
  = TNumber Int
  | TLParen
  | TRParen
  | TPlus
  | TTimes
  | TEOF
```

Os sinônimos **Line** e **Column** representam a informação de linha e coluna inicial de um token. Por sua vez, tipo **Token** é definido como um registro contendo dois campos: um para sua posição inicial e outro que identifica o seu **lexema**. O tipo **Lexeme** representa os diferentes tokens da linguagem com seus construtores: **TNumber** representa números, **TLParen** o abre parêntesis, **TRParen** o fecha parêntesis, **TPlus** o operador de adição, **TTimes** o operador de multiplicação e **TEOF** o final da entrada.

O próximo passo para definirmos o analisador léxico para a linguagem **EXP** é determinar o tipo desta função. A função deverá receber uma string de entrada e retornar uma lista de tokens. Um possível tipo para essa função seria:

```
lexer :: String -> [Token]
```

Porém, como lidar com situações de erro? Por exemplo, qual seria o resultado se a string possuir caracteres inválidos (por exemplo, **a**), que não fazem parte dos possíveis tokens da linguagem? O ideal é que o analisador retorne uma mensagem de erro apropriada. Para isso, vamos modificar o tipo de **lexer** para:

```
lexer :: String -> Either String [Token]
```

para permitir a emissão de mensagens de erro.

A ideia do analisador léxico é percorrer a string de entrada enquanto produz uma lista contendo os tokens encontrados até o momento. Durante o caminhar da entrada, vamos atualizar o *estado* do analisador léxico a cada caractere lido. O estado é definido pelo seguinte sinônimo:

```
type State = (Line, Column, String, [Token])
```

O tipo **State** é uma quádrupla formada pela linha atual, coluna atual, string de dígitos consecutivos encontrados até o momento e a lista de tokens já produzidos. A cada novo caractere processado, devemos atualizar o estado de forma apropriada. Para isso, utilizamos a função **transition**, apresentada a seguir.

```
transition :: State -> Char -> Either String State
transition state@(l, col, t, ts) c
  | c == '\n' = mkDigits state c
  | isSpace c = mkDigits state c
  | c == '+' = Right (l, col + 1, "", mkToken state (Token TPlus (l,col)) ++ ts)
  | c == '*' = Right (l, col + 1, "", mkToken state (Token TMul (l,col)) ++ ts)
  | c == '(' = Right (l, col + 1, "", mkToken state (Token TLParen (l,col)) ++ ts)
  | c == ')' = Right (l, col + 1, "", mkToken state (Token TRParen (l,col)) ++ ts)
  | isDigit c = Right (l, col + 1, c : t, ts)
  | otherwise = unexpectedCharError l col c
```

Quando encontramos parêntesis, operadores ou dígitos simplesmente atualizamos o estado criando um token ou adicionando o caractere de dígito na sequência de dígitos encontrados até o momento atual. Caso o caractere atual seja um espaço ou uma quebra de linha, tentamos construir um token de dígitos consecutivos e atualizamos a respectiva posição de linha / coluna. A função `mkDigits` realiza essa tarefa. Primeiro, verificamos se a string de dígitos encontrados é vazia, nesta situação retornamos o estado com a informação de linha / coluna devidamente modificada. Caso a lista de dígitos seja formada apenas por dígitos, criamos um novo token de dígitos e atualizamos o estado apropriadamente. Finalmente, caso nenhuma destas situações se aplique, temos um erro.

```
mkDigits :: State -> Char -> Either String State
mkDigits state@(l, col, s, ts) c
  | null s = let l' = if c == '\n' then l + 1 else l
              col' = if isSpace c then col + 1 else col
              in (l', col', s, ts)
  | all isDigit s = let t = Token (TNumber (VInt (read $ reverse s))) (l,col)
                    l' = if c == '\n' then l + 1 else l
                    col' = if c /= '\n' && isSpace c then col + 1 else col
                    in Right (l', col', "", t : ts)
  | otherwise = unexpectedCharError l col c
```

De posse da função de transição entre estados do analisador léxico, podemos definir o analisador léxico como:

```
lexer :: String -> Either String [Token]
lexer = finish . foldl step base
  where
    base = Right (1, 1, "", [])

    finish = either Left (Right . extract)

    step ac@(Left _) _ = ac
    step (Right state) c = transition state c

    extract (l, col, s, ts)
      | null s = reverse ts
      | otherwise = let n = read (reverse s)
                    t = Token (TNumber (VInt n))
                      (l, col - (length s))
                    in reverse (t : ts)
```

O valor `base` é definido como a tupla `(1, 1, "", [])` como valor do estado inicial do analisador léxico formado por valores iniciais de linha, coluna e as listas de tokens e dígitos iniciais é vazia.

A função `extract` é aplicada ao estado final do analisador léxico para tentar construir um token de dígitos a partir da sequência de dígitos consecutivos encontrados. Finalmente, a função `finish` realiza o casamento de padrão no resultado do analisar léxico: caso o resultado seja um erro, este é propagado e caso seja o estado final do analisador, aplicamos a função `extract` para obter a lista final de tokens da entrada.

2.4 Analisadores léxicos ad-hoc: vantagens e desvantagens

Analisadores léxicos ad-hoc possuem a vantagem de possuir uma implementação simples para linguagens pequenas e maior controle sobre erros encontrados. Porém, essa técnica possui diversas desvantagens. A primeira é que tais analisadores são difíceis de manter e estender, além de não serem eficientes para linguagens com uma estrutura léxica complexa. Por exemplo, como estender

a implementação do analisador ad-hoc para prover suporte a identificadores? Como dar suporte a comentários de linha ou mesmo de blocos?

Dessa forma, esta técnica é aplicável apenas em situações em que a simplicidade da implementação supera as vantagens de um analisadores léxicos produzidos automaticamente a partir de expressões regulares.

2.5 Conclusão

Neste capítulo apresentamos uma introdução à análise léxica e descrevemos uma implementação de um analisador léxico ad-hoc para uma linguagem de programação simples. Concluimos apresentando argumentos a favor e contra o uso deste tipo de analisadores léxicos. Nos próximos capítulos veremos como analisadores léxicos podem ser produzidos automaticamente a partir da descrição de tokens como expressões regulares.

Capítulo 3

Análise léxica baseada em expressões regulares

3.1 Introdução

No capítulo anterior, apresentamos um analisador léxico ad-hoc para uma linguagem simples. Ao tentarmos estender a linguagem considerada com construções simples com identificadores ou comentários, encontraremos problemas: temos que modificar a estrutura do estado do analisador léxico para acomodar uma string para acumular caracteres que formam um possível identificador e modificar toda a estrutura do analisador para lidar com comentários de linha / blocos.

Diante desta situação, temos a seguinte pergunta: existe uma forma extensível e eficiente para a construção de analisadores léxicos? Neste capítulo, veremos que a teoria de expressões regulares e autômatos finitos provêem uma solução eficiente e automatizável para a criação de analisadores léxicos.

Neste capítulo, vamos revisar a teoria de autômatos, expressões regulares, a implementação de algoritmos e como estes podem ser combinados para a criação de analisadores léxicos para uma especificação de tokens usando expressões regulares.

3.2 Autômatos finitos determinísticos

Nesta seção, vamos apresentar uma implementação simples de autômatos finitos determinísticos e de operações sobre estes.

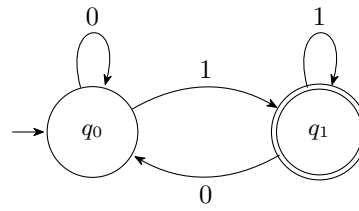
Definição 3.1. *Um autômato finito determinístico M pode ser entendido como uma quintupla $M = (E, \Sigma, \delta, i, F)$, em que:*

- E : conjunto finito de estados;
- Σ : alfabeto;
- $\delta : E \times \Sigma \rightarrow E$, função de transição;
- $i \in E$: estado inicial;
- $F \subseteq E$: conjunto de estados finais.

De maneira simples, o tipo **DFA** é parametrizado pelo tipo de estados e possui 3 campos para representar o estado inicial (campo **start**), a função de transição (campo **delta**) e uma função para determinar se um estado é final ou não (campo **finals**).

```
data DFA a
= DFA {
    start :: a
    , delta :: a -> Char -> a
    , finals :: a -> Bool
}
```

Como um exemplo, considere o seguinte autômato que aceita palavras que terminam em 1.



Esse mesmo autômato pode ser representado pelo seguinte valor em Haskell:

```

endsWithOne :: DFA Bool
endsWithOne = DFA False transition id
  where
    transition False '0' = False
    transition False '1' = True
    transition True  '0' = False
    transition True  '1' = True
    transition _ _ = False
  
```

Aqui representamos o conjunto de estados utilizando o tipo **Bool**. O estado inicial é codificado por **False** e o estado final por **True**. A função de transição é representada por casamento de padrão sobre o estado e o caractere processado. A função para identificar o estado final é a função identidade, **id**.

O cálculo do estado em que a computação de uma string termina em um AFD é feita percorrendo a string utilizando a função de transição de forma direta:

```

deltaStar :: DFA a -> String -> a
deltaStar m = foldl (delta m) (start m)
  
```

que permite a definição direta de uma função para verificar se uma palavra pertence ou não a linguagem descrita por um autômato por testar se o estado retornado por **deltaStar** é final.

```

accept :: DFA a -> String -> Bool
accept m s = finals m (deltaStar m s)
  
```

Evidentemente, em analisadores léxicos devemos utilizar **autômatos mínimos**, visto que estes evitam a utilização de espaço desnecessário por utilizar a menor quantidade possível para seu conjunto de estados. A implementação de algoritmos de minimização está disponível no repositório de código associado a este material. O leitor interessado pode consultar esses detalhes on-line.

3.3 Autômatos finitos não determinísticos

Nesta seção, vamos apresentar uma implementação simples de autômatos finitos não determinísticos e de operações sobre estes. A seguir, apresentamos a descrição formal de autômatos não determinísticos.

Definição 3.2. *Um autômato finito não determinístico M pode ser entendido como uma quintupla $M = (E, \Sigma, \delta, I, F)$, em que:*

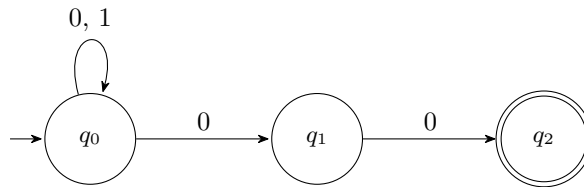
- E : conjunto finito de estados;
- Σ : alfabeto;
- $\delta : E \times \Sigma \rightarrow \mathcal{P}(E)$, função de transição;

- $I \subseteq E$: conjunto de estados iniciais;
- $F \subseteq E$: conjunto de estados finais.

Para representação de um autômato não determinístico, vamos representar de maneira explícita o conjunto de estados do autômato. A função de transição retorna um conjunto de estados para representar a possibilidade dos diversos resultados de uma transição em um AFN.

```
data NFA a
= NFA {
    numberOfStates :: Int
  , nfaStart :: Set a
  , nfaDelta :: a -> Char -> Set a
  , nfaFinals :: Set a
}
```

Como um exemplo, considere o seguinte AFN que aceita a linguagem de palavras que terminam em 00:



este é representado pelo seguinte valor do tipo `NFA Int`:

```
endsWith00 :: NFA Int
endsWith00
= NFA 3 (Set.singleton 0)
  (\ s c -> case (s,c) of
    (0, '0') -> Set.fromList [0,1]
    (0, '1') -> Set.singleton 0
    (1, '0') -> Set.singleton 2
    _         -> Set.empty)
  (Set.singleton 2)
```

Note que a função de transição é codificada por um par formado por um estado e o caractere a ser processado pela transição. Os conjuntos de estados iniciais e finais são representados utilizando a função `Set.singleton` que cria um conjunto unitário com o elemento fornecido como argumento. A definição de aceitação de uma palavra é similar à de AFDs e está implementada no código de suporte deste material.

A definição seguinte revisa a equivalência entre AFNs e AFDs.

Definição 3.3. *Seja $M = (E, \Sigma, \delta, I, F)$ um autômato finito não determinístico qualquer. O AFD equivalente a M é $M' = (\mathcal{P}(E), \Sigma, \delta', I, F')$, em que:*

- $\delta' : \mathcal{P}(E) \times \Sigma \rightarrow \mathcal{P}(E)$, função de transição;
- $I \in \mathcal{P}(E)$: estado inicial;
- $F' = \{X \mid X \cap F \neq \emptyset\}$: conjunto de estados finais.

De maneira simples, no AFD equivalente cada estado representa um conjunto de estados do AFN. O estado inicial do AFD é apenas o conjunto de estados iniciais do AFN e o conjunto de estados finais é qualquer conjunto de estados que possui algum estado final do AFN.

A partir da definição anterior, podemos codificar uma função que transforma um AFN qualquer em um AFD equivalente.

```
subset :: Ord a => NFA a -> DFA (Set a)
subset m
  = DFA {
    start  = nfaStart m
    , delta = \ es c ->
        Set.unions (map (flip (nfaDelta m) c) (Set.elems es))
    , finals = \ es -> not (disjoint es (nfaFinals m))
  }
```

Observe que a função mostra que se o AFN possui estados de tipo `a`, o AFD equivalente terá estados de tipo `Set a`, isto é, conjuntos de estados de tipo `a`. O estado inicial do AFD é definido como o estado inicial do AFN, `nfaStart m`. Estados finais são definidos por uma função que verifica se o argumento possui algum estado final do AFN¹.

3.4 Expressões regulares

Uma abordagem mais declarativa para análise léxica consiste em definir os tokens válidos usando expressões regulares e, em seguida, sintetizar automaticamente o analisador léxico a partir dessas expressões regulares. Essa abordagem é simples e tem maior probabilidade de resultar em código eficiente e de fácil manutenção.

Para iniciar, vamos rever a definição da sintaxe de expressões regulares e de sua semântica.

Definição 3.4. *Expressões regulares são definidas pela seguinte gramática livres de contexto:*

$$e \rightarrow \emptyset \mid \lambda \mid a \mid ee \mid e + e \mid e^*$$

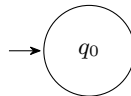
Uma expressão regular e denota um conjunto de strings chamado linguagem de e , escrita $L(e)$, que é definida recursivamente como:

$$\begin{aligned} L(\emptyset) &= \emptyset \\ L(\lambda) &= \{\lambda\} \\ L(a) &= \{a\} \\ L(e_1 e_2) &= L(e_1) L(e_2) \\ L(e_1 + e_2) &= L(e_1) \cup L(e_2) \\ L(e^*) &= L(e)^* \end{aligned}$$

Existem diversas maneiras para converter uma expressão regular em um autômato equivalente. Primeiro, vamos revisar uma abordagem que produz um AFN equivalente por recursão sobre a estrutura da expressão regular.

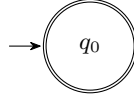
Definição 3.5 (Construção de Thompson). *Seja e uma expressão regular arbitrária. O AFN equivalente a e , M , é definido por recursão sobre a estrutura de e :*

Caso $e = \emptyset$: O AFN M equivalente é

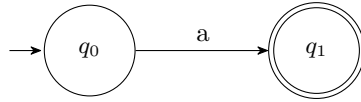


Caso $e = \lambda$: O AFN M equivalente é

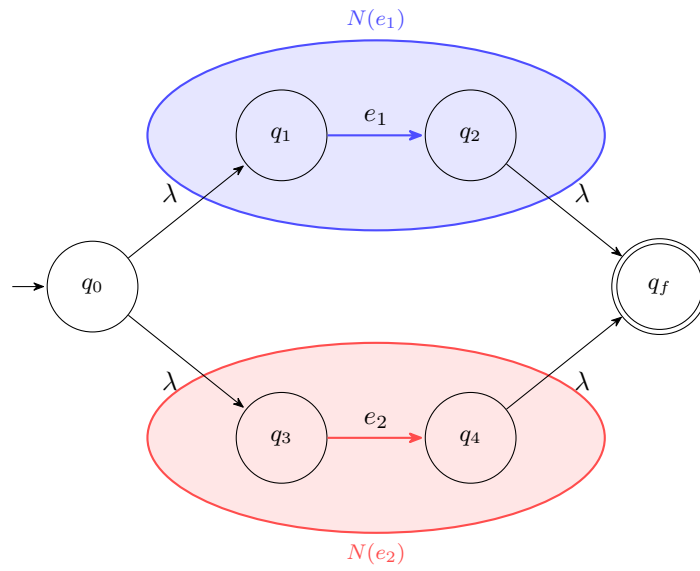
¹A função `disjoint` retorna verdadeiro se a interseção entre dois conjuntos é vazia.



Caso $e = a$: O AFN M equivalente é

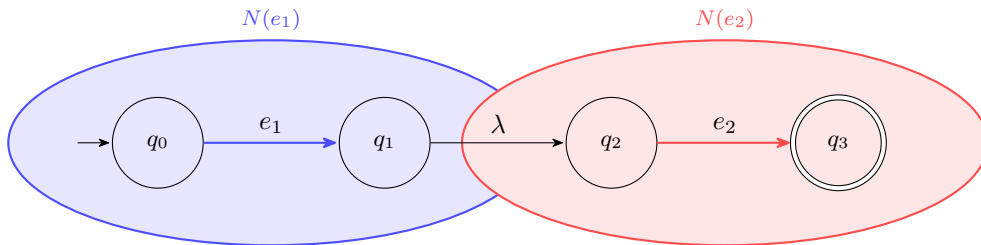


Caso $e = e_1 + e_2$: O AFN M equivalente é



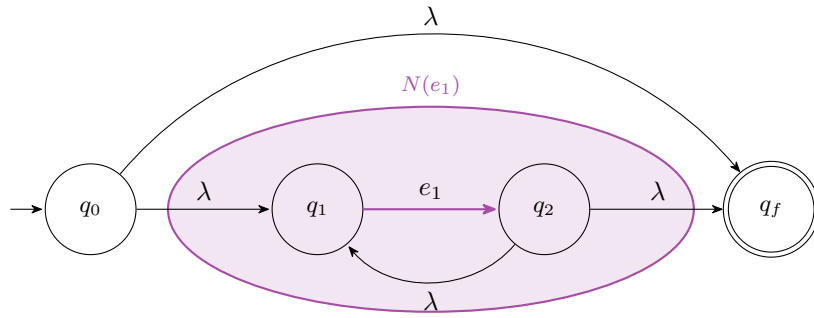
em que $N(e_1)$ é o AFN equivalente a e_1 e $N(e_2)$ é o AFN equivalente a e_2 .

Caso $e = e_1 e_2$: O AFN M equivalente é



em que $N(e_1)$ é o AFN equivalente a e_1 e $N(e_2)$ é o AFN equivalente a e_2 .

Caso $e = e_1^*$: O AFN M equivalente é



em que $N(e_1)$ é o AFN equivalente a e_1 .

3.5 Gerando AFNs a partir de ERs.

Nesta seção, vamos apresentar uma implementação Haskell da construção de Thompson. O primeiro passo é representar expressões regulares utilizando um tipo de dados Haskell, conforme a seguir:

```
data Regex
= Empty
| Lambda
| Chr Char
| Regex :+: Regex
| Regex :@: Regex
| Star Regex
```

Para a criação de AFNs a partir da expressão regular, temos que definir qual tipo representará os estados do AFN. Por questão de simplicidade, vamos utilizar o tipo `Int` para representar estados. Dessa forma, os AFNs produzidos terão tipo `NFA Int`.

Iniciamos apresentando o AFN equivalente a expressão regular `Empty`:

```
emptyNFA :: NFA Int
emptyNFA
= NFA 0 Set.empty (\ _ _ -> Set.empty) Set.empty
```

que consiste de um AFN que não possui estados e nem transições. Dessa forma, aceita a linguagem vazia². O AFN que aceita a linguagem formada apenas pela palavra vazia é:

```
lambdaNFA :: NFA Int
lambdaNFA
= NFA 1 one (\ _ _ -> Set.empty) one
  where
    one = Set.singleton 1
```

que denota um AFN de um único estado inicial e final, sem transições. A função `chrNFA` constrói um AFN que aceita um caractere recebido como argumento.

```
chrNFA :: Char -> NFA Int
chrNFA c
= NFA 2 zero f one
  where
```

²Optamos por não representar como um AFN de um único estado não final para reduzir o número de estados que seriam considerados inúteis posteriormente.

```

zero = Set.singleton 0
one  = Set.singleton 1
err  = Set.empty
f 0 x = if c == x then one
      else err
f _ _ = err

```

Este AFN possui dois estados: 0, que é inicial e não final e 1 que é final. A função de transição produz como resultado um conjunto vazio, exceto quando a origem da transição é o estado 0 e o caractere processado é igual a `c`, que é o argumento de `chrNFA`.

Antes de apresentar funções para produzir AFNs para união, concatenação e fecho de Kleene, devemos garantir que AFNs não possuam estados com o mesmo **nome** (i.e. mesmo valor de tipo `Int`). Para resolver esse problema, vamos definir uma função que soma um valor `n` a cada elemento de um conjunto de inteiros.

```

shift :: Int -> Set Int -> Set Int
shift n = Set.fromAscList . map (+ n) . Set.toAscList

```

A função `shift` converte um conjunto de entrada (tipo `Set Int`) para uma lista de inteiros, usando `Set.toAscList`, e então soma `n` a cada elemento da lista produzida para convertê-la em um conjunto novamente utilizando a função `Set.fromAscList`.

A função que calcula o AFN que aceita a união da linguagem de dois AFNs fornecidos como argumentos é definida como:

```

unionNFA :: NFA Int -> NFA Int -> NFA Int
unionNFA m1 m2
  = NFA {
    numberOfStates = n1 + n2
  , nfaStart = Set.union (nfaStart m1) (shift n1 (nfaStart m2))
  , nfaDelta = f
  , nfaFinals = Set.union (nfaFinals m1) (shift n1 (nfaFinals m2))
  }
  where
    n1 = numberOfStates m1
    n2 = numberOfStates m2
    f s c = if s < n1 then nfaDelta m1 s c
            else shift n1 (nfaDelta m2 (s - n1) c)

```

A função `unionNFA` implementa a união de dois AFNs combinando seus conjuntos de estados de forma disjunta. Para evitar colisões, ela preserva a estrutura do primeiro autômato `m1` e utiliza uma função de deslocamento `shift` para renomear os estados do segundo (`m2`), garantindo que todos os novos estados sejam únicos. O resultado é um novo NFA onde os estados iniciais, as transições e os estados de aceitação são a união das partes correspondentes de ambos os autômatos originais, permitindo que o AFN resultante reconheça a linguagem composta pela união das linguagens aceitas por `m1` e `m2`.

A seguir, apresentamos a função que calcula o AFN que aceita concatenação das linguagens aceitas por dois AFNs fornecidos como argumento.

```

concatNFA :: NFA Int -> NFA Int -> NFA Int
concatNFA m1 m2
  = NFA {
    numberOfStates = n1 + n2
  , nfaStart = newStart
  , nfaDelta = newDelta
  , nfaFinals = newFinals
  }

```

```

}
where
  n1 = numberOfStates m1
  n2 = numberOfStates m2
  start1 = nfaStart m1
  final1 = nfaFinals m1
  final2 = nfaFinals m2
  newStart = if disjoint start1 final1
              then start1
              else Set.union start1 (shift n1 final1)
  newFinals = shift n1 final2
  newDelta e c = if e < n1 then
                    if disjoint (nfaDelta m1 e c) final1
                    then nfaDelta m1 e c
                    else Set.union (nfaDelta m1 e c) (shift n1 start2)
                  else shift n1 (nfaDelta m2 (e - n1) c)

```

Assim como na construção da união de dois AFNs, utilizamos a função `shift` para garantir que estados de cada um dos AFNs de entrada não possuam o mesmo nome. Para isso, utilizamos o renomeamento nas definições da função de transição e nos conjuntos de estados iniciais e finais. A implementação da função de transição `newDelta` decide o fluxo da computação com base no índice do estado atual: se o estado pertence ao primeiro AFN ($e < n1$), a função executa a transição original de `m1`, mas utiliza a verificação `disjoint` para identificar se o destino atinge um estado de aceitação; em caso positivo, ela funde esse destino aos estados iniciais para permitir o início imediato do processamento da segunda linguagem, simulando a conexão entre os autômatos sem recorrer a transições vazias. Caso o estado já pertença ao segundo AFN ($e \geq n1$), a função simplesmente associa a transição original de `m2` para o novo intervalo de índices utilizando `shift`, garantindo que, uma vez iniciada a segunda parte da cadeia, o autômato permaneça restrito a `m2`.

A função `starNFA` implementa o fecho de Kleene ao produzir um AFN, a partir de `m1`, que aceita zero ou mais repetições da linguagem original, sem adicionar novos estados. Para permitir a aceitação da string vazia e o reinício do ciclo, a função redefine os estados de aceitação (`nfaFinals`) como sendo os próprios estados iniciais (`nfaStart m1`) e modifica a função de transição (`newDelta`): sempre que uma transição original de `m1` atingiria um estado final, ela é incluída nos estados iniciais usando `Set.union`. Esse detalhe cria um laço que permite ao autômato processar múltiplas palavras consecutivas, tratando o alcance de um estado final como um gatilho para reiniciar a computação a partir de algum dos estados iniciais.

```

starNFA :: NFA Int -> NFA Int
starNFA m1
  = NFA {
      numberOfStates = numberOfStates m1
    , nfaStart = nfaStart m1
    , nfaDelta = newDelta
    , nfaFinals = nfaStart m1
  }
  where
    newDelta e c
      = let r = nfaDelta m1 e c
        in if disjoint r (nfaFinals m1)
            then r
            else Set.union r (nfaStart m1)

```

De posse de todas essas funções auxiliares, a conversão de uma expressão regular em um AFN equivalente é definida por recursão sobre a estrutura da expressão:

```

thompson :: Regex -> NFA Int

```

```

thompson Empty = emptyNFA
thompson Lambda = lambdaNFA
thompson (Chr c) = chrNFA c
thompson (e1 :+: e2)
  = unionNFA (thompson e1) (thompson e2)
thompson (e1 :@: e2)
  = concatNFA (thompson e1) (thompson e2)
thompson (Star e1)
  = starNFA (thompson e1)

```

Especificações léxicas de uma linguagem são feitas utilizando diversas expressões regulares. A função a seguir, combina as implementações descritas até o momento para produzir o AFD equivalente a uma lista de expressões regulares:

```

lexer :: [Regex] -> DFA (Set Int)
lexer = subset . foldr unionNFA emptyNFA . map thompson

```

Primeiro, obtemos um AFN equivalente para cada expressão usando a função `thompson`. Em seguida, realizamos a união de todos os AFNs produzidos utilizando a função `unionNFA`. Finalmente, obtemos o AFD equivalente chamando a função `subset` sobre o resultado dos passos anteriores.

3.6 Critério do maior prefixo

Autômatos fornecem um maneira eficiente e fundamentada para a criação de analisadores léxicos. Porém, a mera aceitação de um prefixo da entrada é suficiente para que um analisador produza um token? Para entender considere o seguinte trecho de código:

```

if (x > 0) ift = x + 1;

```

Como diferenciar o identificador `ift` da palavra reservada `if`? A ideia é que analisadores léxicos sempre busquem aceitar o **maior prefixo** possível. Esse critério é necessário para evitar ambiguidades semânticas e garantir que a análise reflita a intenção real da linguagem. Sem esse critério, o analisador léxico poderia fragmentar precocemente sequências de caracteres que possuem um significado específico quando combinadas, gerando múltiplos tokens genéricos em vez de um único token. Com isso, o compilador assegura que identificadores, palavras-reservadas e operadores compostos sejam interpretados de forma atômica e correta dentro do contexto gramatical.

Mas isso nos leva a seguinte pergunta: Como reconhecer o maior prefixo utilizando um AFD? A função a seguir, permite que um AFD de argumento reconheça o maior prefixo de uma string de entrada.

```

longest :: DFA a -> String -> Maybe String
longest m = combine . foldl step (Just "", Nothing, start m)
  where
    step (Just pre, Nothing, e) c
      | finals m (delta m e c)
        = ( Just (c : pre)
          , Just (c : pre)
          , delta m e c )
      | otherwise
        = ( Just (c : pre)
          , Nothing
          , delta m e c )
    step (Just pre, Just pre', e) c
      | finals m (delta m e c)

```

```

    = ( Just (c : pre)
      , Just (c : pre')
      , delta m e c)
  | otherwise = ( Nothing
                , Just pre'
                , delta m e c)
step (Nothing, val, e) c = ( Nothing
                          , val
                          , delta m e c)

combine (_, val, _) = reverse <$> val

```

A função `longest` busca identificar o maior prefixo de uma string aceita por AFD, percorrendo a entrada enquanto mantém um acumulador representado como uma tripla formada por:

- Prefixo atual sendo processado
- Último prefixo válido (aceito pelo DFA) encontrado até o momento
- Estado atual do autômato.

A lógica de atualização do acumulador é feita pela função `step` que lida com três situações: ela acumula caracteres no prefixo atual enquanto for possível estendê-lo, salva essa string como o “melhor resultado” sempre que atinge um estado final (`finals`) e, caso o caminho se torne inválido para futuras aceitações (indicado pelo valor `Nothing` no primeiro elemento da tupla), interrompe a atualização do prefixo em progresso mas preserva a maior correspondência já registrada. Ao final, a função `combine` recupera esse maior prefixo armazenado e inverte a ordem da string (corrigindo a inversão causada pela construção da lista) para retornar a string resultante.

3.7 Derivadas de expressões regulares

A geração de analisadores léxicos baseados na construção de Thompson é útil para o reconhecimento de expressões regulares em um compilador, visto que a especificação do token não se altera. No entanto, expressões regulares são frequentemente usadas em outros contextos em que a pré-compilação em um AFD não compensa o custo. Infelizmente, bibliotecas comuns de expressões regulares, como as utilizadas por Java, Perl e Javascript, apresentam desempenho exponencial no pior caso, pois empregam backtracking para lidar com o operador de união.

Uma técnica alternativa consiste em interpretar diretamente a expressão regular à medida que a entrada é analisada, usando derivadas de expressões regulares, uma técnica elegante desenvolvida por Janus Brzozowski. A ideia é que, para qualquer expressão regular e símbolo de entrada, podemos calcular a expressão regular que aceita o restante da entrada após o símbolo de entrada ter sido consumido. Usamos a notação $\partial(e, a)$ para representar a derivada da expressão regular e com respeito ao símbolo de entrada a .

Antes de apresentar a definição de derivadas propriamente ditas, precisamos de um conceito auxiliar: a noção de anulabilidade. Dizemos que uma expressão regular é **anulável** se $\lambda \in L(e)$. Esse conceito é definido a seguir.

Definição 3.6. Dizemos que uma expressão regular é **anulável** se $\nu(e) = \top$, em que $\nu(e)$ é definido por recursão sobre a estrutura da expressão da seguinte maneira:

$$\begin{aligned}
 \nu(\emptyset) &= \perp \\
 \nu(\lambda) &= \top \\
 \nu(a) &= \perp \\
 \nu(e_1 e_2) &= \nu(e_1) \wedge \nu(e_2) \\
 \nu(e_1 + e_2) &= \nu(e_1) \vee \nu(e_2) \\
 \nu(e_1^*) &= \top
 \end{aligned}$$

As três primeiras equações mostram que o conjunto vazio não é anulável, que λ é anulável e que um símbolo não é considerado anulável. Uma concatenação é anulável somente se ambas as suas sub-expressões são anuláveis e a união é considerada anulável se qualquer uma de suas sub-expressões o for anulável. Finalmente, uma expressão envolvendo o fecho de Kleene é sempre anulável.

Utilizando o conceito de anulabilidade, podemos definir a derivada de expressões regulares.

Definição 3.7 (Derivada de uma expressão regular). *Seja e uma expressão regular qualquer e $a \in \Sigma$. A derivada de e com respeito a a é definida como:*

$$\begin{aligned} \partial(\emptyset, a) &= \emptyset \\ \partial(\lambda, a) &= \emptyset \\ \partial(b, a) &= \begin{cases} \lambda & \text{se } a = b \\ \emptyset & \text{caso contrário} \end{cases} \\ \partial(e_1 + e_2, a) &= \partial(e_1, a) + \partial(e_2, a) \\ \partial(e_1 e_2, a) &= \begin{cases} \partial(e_1, a)e_2 + \partial(e_2, a) & \text{se } \nu(e_1) = \top \\ \partial(e_1, a)e_2 & \text{caso contrário} \end{cases} \\ \partial(e_1^*, a) &= \partial(e_1, a)e_1^* \end{aligned}$$

A definição da derivada de uma expressão regular em relação a um símbolo a é apresentada de forma recursiva. Para os casos base: a derivada da expressão vazia $\emptyset$ ou da palavra vazia λ em relação a qualquer símbolo resulta em $\emptyset$, pois não é possível consumir um caractere a partir delas. No caso de um símbolo b , a derivada retorna λ se o símbolo consumido a for igual a b (indicando que o caractere esperado foi encontrado e a expressão se reduz à palavra vazia), ou $\emptyset$ caso contrário. Para a união $e_1 + e_2$, a derivada é simplesmente a união das derivadas de cada subexpressão. No caso da concatenação $e_1 e_2$, a definição se desdobra: se e_1 for anulável (ou seja, pode reconhecer a palavra vazia), a derivada é a união entre a concatenação da derivada de e_1 com e_2 e a derivada de e_2 ; caso contrário, é apenas a derivada de e_1 concatenada com e_2 . Por fim, para o fecho de Kleene e^* , a derivada é a concatenação da derivada de e com a própria estrela e^* , permitindo que a expressão continue reconhecendo múltiplas repetições.

Para obter um autômato finito determinístico (AFD) a partir de uma expressão regular utilizando derivadas, podemos construir os estados do autômato como as próprias expressões regulares que surgem durante o processo de derivação. A ideia fundamental é que, partindo da expressão regular original e_0 , podemos calcular sua derivada em relação a cada símbolo do alfabeto, obtendo novas expressões que representam o estado após consumir aquele símbolo. Este processo é repetido para cada nova expressão gerada, até que nenhuma nova expressão seja produzida, garantindo que o conjunto de estados seja finito.

Definição 3.8. *Seja e uma expressão regular qualquer. O AFD equivalente a e é $M = (E, \Sigma, \delta, i, F)$, em que:*

- E : é o conjunto de todas as derivadas para a expressão e .
- Σ : alfabeto de entrada
- δ : função de transição, representada pela própria derivada: $\delta(e, a) = \partial(e, a)$.
- $F = \{e_1 \mid \nu(e_1) = \top\}$, isto é, consideramos como estados finais as expressões regulares anuláveis.

Um ponto crucial nesta construção é o fato de que o conjunto de derivadas de uma expressão é **finito**. Porém, esse fato é verdade apenas se considerarmos algumas simplificações, apresentadas a seguir:

$$\begin{array}{lll} e_1 + \emptyset \equiv e_1 & \emptyset + e_2 \equiv e_2 & e_1 \emptyset \equiv \emptyset \\ \emptyset e_2 \equiv \emptyset & e_1 \lambda \equiv e_1 & \lambda e_2 \equiv e_2 \\ \lambda^* \equiv \lambda & \emptyset^* \equiv \lambda & (e^*)^* \equiv e^* \end{array}$$

Considerando essas simplificações, é demonstrado que sempre obteremos um conjunto finito de derivadas a partir de uma expressão regular qualquer.

3.8 Implementação em Haskell

De posse das definições, podemos implementar a obtenção de um AFD a partir de uma expressão regular de forma direta. Primeiro, vamos definir uma função para determinar se uma expressão é ou não anulável.

```
nullable :: Regex -> Bool
nullable Empty = False
nullable Lambda = True
nullable (Chr _) = False
nullable (e1 :+: e2) = nullable e1 || nullable e2
nullable (e1 :@: e2) = nullable e1 && nullable e2
nullable (Star _) = True
```

Observe que a implementação é uma tradução direta da notação matemática para código Haskell. Em seguida, apresentamos funções para realizar as simplificações da união, concatenação e fecho de Kleene. Cada uma destas funções realiza as simplificações apresentadas anteriormente.

```
(.+. ) :: Regex -> Regex -> Regex
Empty .+. e' = e'
e .+. Empty = e
e .+. e' = e :+: e'

(.@.) :: Regex -> Regex -> Regex
Empty .@. _ = Empty
_ .@. Empty = Empty
Lambda .@. e' = e'
e .@. Lambda = e
e .@. e' = e :@: e'

star :: Regex -> Regex
star Empty = Lambda
star (Star e) = e
star e = Star e
```

De posse destas simplificações e do teste de anulabilidade, a implementação da operação de derivada é apenas a tradução da notação matemática para código Haskell.

```
deriv :: Char -> Regex -> Regex
deriv _ Empty = Empty
deriv _ Lambda = Empty
deriv a (Chr b)
  | a == b = Lambda
  | otherwise = Empty
deriv a (e1 :+: e2)
  = deriv a e1 .+. deriv a e2
deriv a (e1 :@: e2)
  | nullable e1 = deriv a e1 .@. e2 .+. deriv a e2
  | otherwise = deriv a e1 .@. e2
deriv a (Star e1)
  = deriv a e1 .@. (star e1)
```

Para construção do AFD utilizando derivadas, precisamos obter o conjunto de todas as derivadas de uma expressão regular. A função `enumerateDerivs` implementa um algoritmo para enumerar todas as expressões regulares alcançáveis a partir de uma expressão inicial (`initial`) através da aplicação sucessiva da operação de derivada em relação aos símbolos do alfabeto.

```

enumerateDerivs :: Regex -> [Regex]
enumerateDerivs initial = go [initial] [initial]
  where
    go [] acc = acc
    go (state:rest) acc =
      let chars = symbols initial
          newStates = nub [deriv c state | c <- chars]
          unseen = filter (`notElem` acc) newStates
      in go (rest ++ unseen) (acc ++ unseen)

```

A função `enumerateDerivs` realiza uma busca sistemática em largura sobre o espaço de estados (expressões regulares), utilizando duas listas: a primeira (`state:rest`) funciona como uma fila de estados a serem processados, enquanto a segunda (`acc`) acumula todos os estados já visitados. Para cada estado atual, a função obtém todos os símbolos do alfabeto a partir da expressão inicial (usando `symbols initial`), calcula a derivada para cada um deles com `deriv c state`, e remove duplicidades com `nub`. As novas expressões geradas que ainda não estão no acumulador

```
(filter (notElem acc) newStates)
```

são então adicionadas tanto ao final da fila de processamento quanto ao acumulador, garantindo que todos os estados alcançáveis sejam explorados. O processo continua até que a fila se esgote, retornando a lista completa de expressões regulares que formam os estados do autômato.

Finalmente, a construção do autômato a partir da expressão regular é feita pela função `derivDFA`:

```

derivDFA :: Regex -> DFA Regex
derivDFA e
  = DFA {
    start = initial
    , delta = delta'
    , finals = nullable
  }
  where
    initial = simp e
    states = enumerateDerivs initial

    delta' state char =
      let e' = deriv char state
      in if e' `elem` states then e' else Empty

```

O estado inicial é dado pela simplificação da expressão regular de entrada e os estados finais são exatamente as expressões regulares anuláveis. Finalmente, a função de transição é a derivada com respeito ao caractere considerado.

3.9 Conclusão

Neste capítulo, apresentamos os fundamentos teóricos e implementações práticas para a construção automatizada de analisadores léxicos. Exploramos autômatos finitos determinísticos e não determinísticos, a construção de Thompson para conversão de expressões regulares em AFNs, e o algoritmo para converter AFNs em AFDs. Discutimos o critério do maior prefixo como solução para ambiguidades na identificação de tokens. Por fim, introduzimos a abordagem de derivadas de expressões regulares, que permite construir AFDs diretamente a partir de expressões regulares de forma elegante e eficiente.

Capítulo 4

Geradores de analisadores léxicos

4.1 Introdução

Neste capítulo, veremos como utilizar um gerador para construir um analisador léxico a partir de uma especificação. Em Haskell, o gerador de analisadores mais popular é o Alex. Este capítulo apresenta uma introdução ao uso do Alex em um exemplo prático: a construção do analisador léxico para uma linguagem de expressões.

O código discutido neste capítulo está disponível em

`Exp/Frontend/Lexer/Alex/ExpLexer.x`

4.2 Construindo um analisador léxico utilizando o Alex

Nesta seção vamos apresentar a especificação de um analisador léxico utilizando a linguagem do Alex. Primeiro, vamos definir um tipo para representação de tokens e, em seguida, descreveremos a especificação do analisador e como este pode ser criado utilizando o Alex.

4.2.1 Definindo os tokens

Antes de criar o lexer, precisamos definir o que é um **token** na nossa linguagem de expressões. Os tipos a seguir apresentam a definição de token que utilizaremos.

```
data Token = Token {
    pos      :: (Int, Int) -- (Linha, Coluna)
    , lexeme :: Lexeme
} deriving (Eq, Show)

data Lexeme
= TNumber Int | TLParen | TRParen | TPlus | TTimes | TEOF
deriving (Eq, Show)
```

O tipo **Lexeme** representa as diferentes categorias de tokens presentes na linguagem: números, abre e fecha parêntesis, adição, multiplicação e um valor especial para fim da entrada (**EOF**). Um token é formado por um **Lexeme** e um par de inteiros que representam a linha e coluna inicial daquele token na entrada.

4.2.2 Sintaxe de expressões regulares no Alex

Alex usa expressões regulares para definir padrões. A seguir, vamos apresentar alguns exemplos de expressões que são utilizadas na especificação léxica de linguagens.

A sintaxe `[0-9]` denota que qualquer caractere numérico entre 0 e 9 (inclusive) será aceito. Da mesma forma, `[a-zA-Z]` denota uma expressão para qualquer caractere minúsculo ou maiúsculo.

Caracteres também podem ser excluídos; por exemplo, `[\\^\\"]` é entendido como “qualquer coisa, exceto aspas duplas”.

O operador Kleene denota que uma expressão pode ser casada zero ou mais vezes com a entrada. Da mesma forma, o fecho positivo, `+` denota que uma expressão pode ser casada uma ou mais vezes. Por exemplo, `[0-9]+` aceita a palavra `1234`, pois denota uma sequência formada por um ou mais dígitos. Outro operador comum é `?` que significa que uma expressão pode ser casada opcionalmente, isto é, zero ou uma vez.

Um ponto, `.`, casa com qualquer caractere diferente de quebra de linha. Uma string entre aspas duplas, como `">="`, corresponderá exatamente à string dentro das aspas. Um caractere de escape, como `\n`, corresponderá exatamente a esse caractere.

Você pode agrupar expressões regulares concatenando-as. Ao concatenar expressões regulares, como em `[a-z][A-Za-z0-9]*`, significa que o Alex deve corresponder a cada conjunto de strings em sequência. Um exemplo de lexema que pode ser aceita por essa expressão regular é `myVariable1`.

O operador de barra vertical, `|`, indica que o Alex pode alternar entre opções, como

```
0(x[0-9a-fA-F]+ | o[0-7]+)
```

que é uma expressão regular que pode corresponder a números hexadecimais ou octais.

4.2.3 Estrutura de especificações Alex

A estrutura de um arquivo de especificação Alex (com extensão `.x`) é dividida em quatro seções principais. O arquivo inicia com um bloco de código Haskell, delimitado por chaves `{...}`, onde são inseridos o nome do módulo e as importações necessárias. No exemplo, `ExpLexer.x`, temos o seguinte bloco inicial:

```
{
{-# OPTIONS_GHC -Wno-name-shadowing #-}
module Exp.Frontend.Lexer.Alex.ExpLexer where

import Control.Monad
import Exp.Frontend.Lexer.Token
}
```

Logo abaixo, encontram-se as definições de macros e diretivas de configuração, como o `%wrapper`, que instrui o Alex sobre qual o “template” de analisador léxico deve ser gerado.

```
%wrapper "monadUserState"
```

```
$digit = 0-9
@number = $digit+
```

O Alex suporta dois tipos de macros para facilitar a escrita de expressões regulares: os conjuntos de caracteres (como `$digit`), que representam classes de caracteres, e as macros de expressões regulares (como `@number`), que funcionam como atalhos para padrões mais complexos. Quanto à infraestrutura de código gerada, o Alex utiliza wrappers que definem a interface do lexer; os mais comuns incluem o **basic**, que processa uma String de forma simples, o **posn**, que adiciona automaticamente informações de linha e coluna, e o **monadUserState**, que permite a execução dentro de uma mônada para manipulação de estados complexos. A principal diferença entre eles reside no controle: enquanto os wrappers simples são puramente funcionais e diretos, o wrapper **monadUserState** (utilizado no exemplo) oferece maior flexibilidade ao permitir que o desenvolvedor armazene dados personalizados, como o nível de aninhamento de comentários, diretamente no estado do analisador.

A especificação léxica é inicializada pelo símbolo `tokens :-` e contém as regras de tradução que associam expressões regulares a ações específicas em Haskell, que podem ser utilizadas para criação de tokens da linguagem. A seguir, apresentamos a especificação para a linguagem de expressões.

```

tokens :-
  -- whitespace and line comments
  <0> $white+      ;
  <0> "//" .*      ;
  -- other tokens
  <0> @number      {mkNumber}
  <0> "("          {simpleToken TLParen}
  <0> ")"          {simpleToken TRParen}
  <0> "+"          {simpleToken TPlus}
  <0> "*"          {simpleToken TTimes}
  -- multi-line comment
  <0> "\\*"        { nestComment `andBegin` state_comment }
  <0> "*/"         { \ _ _ -> alexError "Error! Unexpected close comment!" }
  <state_comment> "\\*" { nestComment }
  <state_comment> "*/" { unnestComment }
  <state_comment> .    ;
  <state_comment> \n   ;

```

O código anterior define as regras de reconhecimento para cada elemento da linguagem, utilizando **start codes** para alternar entre o processamento normal e o de comentários. No estado inicial <0>, o analisador ignora espaços em branco (`$white+`) e comentários de linha (começam com `//`). Para tokens significativos como números (`@number`), parênteses e operadores, o Alex executa ações Haskell como `mkNumber` ou `simpleToken`, que geram os tokens correspondentes. Caso o analisador encontre um token para fechar um comentário, `*/` enquanto ainda está no estado inicial, ele dispara um erro imediato, pois não há um bloco aberto para ser fechado.

A transição para o estado de comentários multilinha ocorre quando o padrão `\\*` é identificado no estado <0>, acionando a função `nestComment` e mudando o estado para `state_comment` através da função `andBegin`. Uma vez dentro do estado `state_comment`, o léxico ignora caracteres genéricos (usando `.`) e quebras de linha (`\\n`), mas permanece atento a novos símbolos de abertura ou fechamento de blocos de comentário. O uso de `nestComment` e `unnestComment` dentro desse estado permite que o analisador incremente ou decmente um contador de nível de aninhamento. Somente quando o nível de aninhamento retorna a zero é que a função `unnestComment` retorna para o estado <0>, permitindo que a análise continue normalmente.

Em especificações Alex, quando utilizamos o wrapper `monadUserState`, ganhamos a capacidade de carregar um estado personalizado durante toda a análise, definido pelo tipo `AlexUserState`. No exemplo, esse tipo contém apenas o campo `nestLevel` para rastrear o nível de blocos de comentários aninhados. Para definir o estado inicial, devemos codificar a função `alexInitUserState` que inicializa esse contador de nível de comentários em zero.

A manipulação desse estado ocorre através de três funções fundamentais: `get`, `put` e `modify`. A função `get` extrai o estado atual de dentro da mônada Alex, enquanto `put` sobrescreve o estado existente por um novo. Já a função `modify` aplica uma transformação direta ao estado. Por exemplo, na função `nestComment`, `modify` é usada para incrementar o nível de comentário de forma concisa.

4.3 Conclusão

Este capítulo apresentou como usar o gerador de analisadores léxicos para Haskell, Alex. Apresentamos a definição de tokens, como tipos de dados Haskell. Utilizando a linguagem de especificação do Alex, definimos quais padrões (definidos como expressões regulares) e quais tokens são associados a eles. Apresentamos os conceitos de wrappers e como estados podem ser utilizados para modificar o comportamento do analisador léxico. Mostramos como esse mecanismo é utilizado para descartar comentários em bloco. Com isso, temos a base necessária para a próxima fase do compiladores: a análise sintática.

Parte III

Análise sintática

Capítulo 5

Introdução à análise sintática

5.1 Introdução

Após a análise léxica, o programa de entrada foi convertido em uma lista de tokens. O próximo passo da compilação é a análise sintática, também chamada de **parsing**, na qual o compilador verifica se os tokens formam uma sintaxe válida da linguagem. Para além de uma resposta booleana, analisadores sintáticos constroem uma representação do programa como uma árvore para uma uso em fases posteriores do compilador.

É importante ressaltar que o analisador sintático não realiza todas as verificações restantes sobre o programa; por exemplo, não é responsabilidade do analisador sintático determinar se tipos são compatíveis ou se todas as variáveis estão declaradas. Essas verificações são feitas em outra etapa, a análise semântica.

A análise sintática é um tópico bem estabelecido na ciência da computação e seu estudo pode fornecer ideias para aplicações em outras áreas visto que não apenas compiladores precisam analisar informações.

5.2 Gramáticas livres de contexto

Neste curso, adotamos a abordagem de construir implementações a partir de especificações precisas. Como podemos especificar a sintaxe válida de programas em uma linguagem de programação? Vimos que, para o problema de especificar tokens válidos, as expressões regulares nos fornecem uma linguagem de especificação suficientemente expressiva. Infelizmente, as expressões regulares não são suficientes para representar a sintaxe da maioria das linguagens de programação.

Uma maneira fácil de perceber isso é considerar a necessidade de verificar se os parênteses e outras construções de agrupamento estão devidamente balanceados. Para balancear os parênteses, é necessário contar o número de parênteses abertos que ainda não foram fechados. No entanto, as expressões regulares podem ser reconhecidas por autômatos finitos, enquanto que contar um número ilimitado de parênteses exigiria um número ilimitado de estados. Nenhum autômato finito consegue controlar o número de parênteses desbalanceados, portanto, as expressões regulares não são suficientemente poderosas para especificar a sintaxe da linguagem.

Essa limitação das expressões regulares motiva o uso de gramáticas livres de contexto (GLCs) para especificar a sintaxe da linguagem.

Uma gramática livre de contexto é caracterizada por um conjunto de produções. Cada produção explica como reescrever um símbolo não terminal em alguma sequência de símbolos terminais (tokens) ou símbolos não terminais. Como exemplo, temos a seguir uma gramática para uma linguagem de expressões aritméticas:

$$E \rightarrow n \mid E + E \mid E * E \mid (E)$$

em que $\Sigma = \{+, *, (,), n\}$ são **tokens** desta linguagem.

Dada uma gramática $G = (V, \Sigma, R, P)$, uma palavra pertence à sua linguagem, $L(G)$, se existir alguma derivação desta string usando as regras de G . Uma derivação é uma sequência de reescritas

que começa com a variável inicial e eventualmente termina na palavra considerada. Cada passo na sequência aplica alguma produção da gramática: realiza uma reescrita

$$\alpha A \beta \rightarrow \alpha \gamma \beta$$

em que $A \rightarrow \gamma$ é uma regra de G .

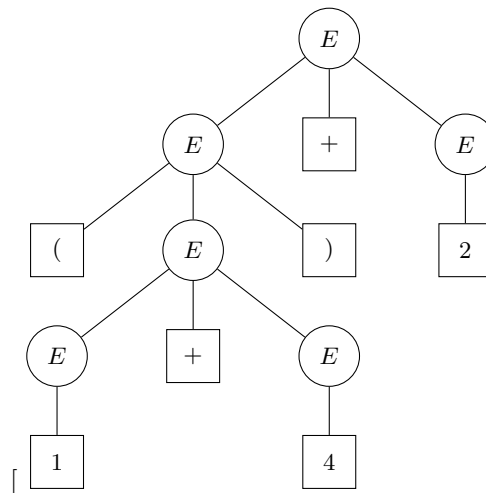
Por exemplo, a palavra $(1+4)+2$ pertence à linguagem da gramática. Podemos verificar isso construindo uma derivação:

$$\begin{aligned} E &\Rightarrow \\ E + E &\Rightarrow \\ (E) + E &\Rightarrow \\ (E + E) + E &\Rightarrow \\ (1 + E) + E &\Rightarrow \\ (1 + 4) + E &\Rightarrow \\ (1 + 4) + 2 & \end{aligned}$$

Essa derivação é uma derivação mais à esquerda: a cada passo, o não terminal expandido é o não terminal mais à esquerda. Como veremos em breve, as derivações mais à esquerda são construídas por analisadores sintáticos descendentes. Também é possível construir uma derivação mais à direita para qualquer string, expandindo o não terminal mais à direita a cada passo. Por exemplo, a derivação mais à direita da mesma string procede da seguinte forma:

$$\begin{aligned} E &\Rightarrow \\ E + E &\Rightarrow \\ E + 2 &\Rightarrow \\ (E) + 2 &\Rightarrow \\ (E + E) + 2 &\Rightarrow \\ (E + 4) + 2 &\Rightarrow \\ (1 + 4) + 2 & \end{aligned}$$

Uma derivação corresponde a uma árvore sintática, na qual a raiz é o símbolo inicial e as folhas foram a string do programa processado. Ambas as derivações acima correspondem à seguinte árvore de análise sintática:



Embora essas duas derivações gerem a mesma árvore de análise sintática, é possível, em geral, que uma string tenha mais de uma árvore sintática. Quando isso acontece, diz-se que a gramática é ambígua. Uma gramática ambígua é aquela para a qual existe alguma string com múltiplas árvores sintáticas.

Nossa gramática de exemplo é, de fato, ambígua. Por exemplo, considere a string $2 + 3 * 4$. Ela possui uma árvore de análise sintática na qual a produção $E \rightarrow E + E$ é usada na raiz, mas também

outra árvore na qual a produção $E \rightarrow E * E$ é usada na raiz. Considerando-as como expressões aritméticas, a primeira árvore de análise sintática corresponde ao valor esperado da expressão, 14. A segunda árvore de análise sintática corresponde ao cálculo de 20: possivelmente, a resposta errada. O problema com a gramática é que ela não impõe precedência ou associatividade aos operadores $+$ e $*$.

5.2.1 Representando precedência e associatividade

Garantir precedência e associatividade é um problema comum no projeto de gramáticas. Felizmente, existem algumas maneiras padronizadas de lidar com isso.

O problema com a gramática original é que ela permite que a produção $E \rightarrow E + E$ apareça na árvore de análise sintática diretamente abaixo da produção $E \rightarrow E * E$. Podemos evitar essas árvores de análise sintática introduzindo não-terminais separados para representar diferentes níveis de precedência. Por exemplo, podemos dizer que uma expressão aritmética é a soma de um ou mais termos (T), onde cada termo é o produto de um ou mais fatores (F), conforme a gramática a seguir:

$$\begin{aligned} E &\rightarrow E + T \mid T \\ T &\rightarrow T * F \mid F \\ F &\rightarrow n \mid (E) \end{aligned}$$

Esta gramática impede que uma soma apareça como argumento de um produto, a menos que esteja entre parênteses. Esta gramática também impõe a associatividade à esquerda dos operadores $+$ e $*$. Essa restrição surge porque a produção $E \rightarrow E + T$ é recursiva à esquerda: o símbolo E aparece nas produções apenas como o primeiro símbolo. Por outro lado, se quisermos que um operador binário seja associativo à direita, usaríamos uma produção recursiva à direita como $E \rightarrow E + T$.

5.2.2 Ambiguidade do else vazio

A maioria das linguagens de programação possui uma estrutura condicional if-then-else, o que naturalmente leva à ambiguidade. Considere a seguinte gramática simplificada para instruções:

$$\begin{aligned} S &\rightarrow \text{if } E \text{ then } S \\ &\quad \mid \text{if } E \text{ then } S \text{ else } S \\ &\quad \mid \text{other} \end{aligned}$$

Se usarmos essa gramática para analisar uma string da forma **if** **E1** **then** **if** **E2** **then** **S1** **else** **S2**, existem dois resultados possíveis: uma em que a cláusula **else** se liga ao primeiro **if** e outra em que se liga ao segundo. A maioria das linguagens implementa a regra do **if** mais próximo: um **else** se liga ao **if** precedente mais próximo.

Para resolver esse problema, observamos que existem dois tipos de instruções: instruções não correspondentes que terminam em uma sequência da forma **then S**, e instruções correspondentes, como **if E then S1 else S2**, que não terminam dessa forma. A ideia principal é que não podemos permitir que uma instrução não correspondente seja seguida por um **else**, porque, nesse caso, a regra do closest-if exigiria que a cláusula **else** fizesse parte do **if** não correspondente.

Essa intuição pode ser capturada introduzindo não terminais separados para representar instruções correspondentes (**M**) e não correspondentes (**U**):

$$\begin{aligned} M &\rightarrow \text{if } E \text{ then } M \text{ else } M \\ &\quad \mid \text{other} \\ U &\rightarrow \text{if } E \text{ then } S \\ &\quad \mid \text{if } E \text{ then } S \text{ else } U \end{aligned}$$

A questão principal é que ambas as produções que incluem **else** exigem que a instrução precedente seja uma instrução correspondente.

5.2.3 Remoção de Recursão à Esquerda

Um analisador sintático descendente tenta expandir o não-terminal mais à esquerda a cada passo. Se uma gramática contém **recursão à esquerda**; isto é, um não-terminal A tal que $A \Rightarrow^+ A\alpha$ para alguma sequência α ; um analisador descendente entrará em loop infinito ao tentar expandir A , pois sempre escolherá a mesma produção sem consumir nenhum token. Por isso, gramáticas com recursão à esquerda precisam ser transformadas antes de serem usadas por analisadores descendentes.

Recursão direta à esquerda

A forma mais simples de recursão à esquerda é a **direta**: o não-terminal aparece imediatamente como o primeiro símbolo do corpo de sua própria produção. Por exemplo, as produções da gramática de expressões:

$$\begin{aligned} E &\rightarrow E + T \mid T \\ T &\rightarrow T * F \mid F \end{aligned}$$

são diretamente recursivas à esquerda. O padrão geral de uma produção diretamente recursiva à esquerda é:

$$A \rightarrow A\alpha_1 \mid A\alpha_2 \mid \dots \mid A\alpha_m \mid \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$$

em que nenhum β_i começa com A . A transformação consiste em introduzir um novo não-terminal A' e reescrever as produções como:

$$\begin{aligned} A &\rightarrow \beta_1 A' \mid \beta_2 A' \mid \dots \mid \beta_n A' \\ A' &\rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \dots \mid \alpha_m A' \mid \lambda \end{aligned}$$

A ideia é converter a recursão à esquerda em recursão à **direita**: A' acumula os sufixos α_i à direita, e a produção $A' \rightarrow \lambda$ encerra a recursão.

Aplicando essa transformação à gramática de expressões:

$$\begin{aligned} E &\rightarrow T E' \\ E' &\rightarrow + T E' \mid \lambda \\ T &\rightarrow F T' \\ T' &\rightarrow * F T' \mid \lambda \\ F &\rightarrow n \mid (E) \end{aligned}$$

A gramática resultante é equivalente à original: gera a mesma linguagem. No entanto, agora é adequada para análise descendente, pois nenhuma produção é recursiva à esquerda.

Observação: a transformação preserva a linguagem gerada, mas altera a estrutura das árvores de derivação. Em particular, a associatividade à esquerda dos operadores é capturada pela recursão à direita de A' , que ao ser expandida produz uma cadeia de operações associativas à esquerda.

Recursão indireta à esquerda

A recursão à esquerda também pode ser **indireta**, ocorrendo por meio de uma cadeia de derivações. Por exemplo:

$$\begin{aligned} A &\rightarrow B a \\ B &\rightarrow A b \mid c \end{aligned}$$

Aqui $A \Rightarrow B a \Rightarrow A b a$, caracterizando recursão à esquerda indireta. O algoritmo geral para eliminar toda recursão à esquerda (direta e indireta) funciona ordenando os não-terminais $A_1, A_2, \dots, A_n$ e executando os seguintes passos:

Algorithm 1 Eliminação de recursão à esquerda

```

1: for  $i = 1$  até  $n$  do
2:   for  $j = 1$  até  $i - 1$  do
3:     Substituir cada produção da forma  $A_i \rightarrow A_j \alpha$  pelas produções  $A_i \rightarrow \delta_1 \alpha \mid \delta_2 \alpha \mid \dots$ , onde
4:      $A_j \rightarrow \delta_1 \mid \delta_2 \mid \dots$  são as produções atuais de  $A_j$ .
5:   Eliminar a recursão direta à esquerda entre as produções de  $A_i$ .

```

Após executar esse algoritmo, a gramática resultante não possui recursão à esquerda (direta ou indireta), supondo que a gramática original não tenha ciclos $A \Rightarrow^+ A$ nem produções λ problemáticas.

5.2.4 Fatoração à Esquerda

Mesmo sem recursão à esquerda, uma gramática pode ser inadequada para análise LL(1) se duas produções de um mesmo não-terminal começam com o mesmo símbolo. Nesse caso, com apenas um símbolo de lookahead, o analisador não consegue determinar qual produção aplicar.

Por exemplo, considere as produções:

$$\begin{array}{l} S \rightarrow \text{if } E \text{ then } S \text{ else } S \\ \quad \mid \text{if } E \text{ then } S \end{array}$$

Ao ver o token **if**, o analisador não sabe qual das duas produções escolher. A **fatoração à esquerda** resolve esse problema: o prefixo comum é “fatorado” para uma produção separada, e as alternativas restantes são agrupadas em um novo não-terminal.

O padrão geral é: dadas as produções

$$A \rightarrow \alpha \beta_1 \mid \alpha \beta_2 \mid \dots \mid \alpha \beta_k \mid \gamma_1 \mid \gamma_2 \mid \dots$$

onde α é o prefixo comum mais longo, transformá-las em:

$$\begin{array}{l} A \rightarrow \alpha A' \mid \gamma_1 \mid \gamma_2 \mid \dots \\ A' \rightarrow \beta_1 \mid \beta_2 \mid \dots \mid \beta_k \end{array}$$

O analisador expande A consumindo α e depois decide entre $\beta_1, \beta_2, \dots$ baseado no próximo token, que agora é suficiente para distingui-los (desde que os β_i comecem com tokens distintos).

Aplicando ao exemplo do if-then-else:

$$\begin{array}{l} S \rightarrow \text{if } E \text{ then } S' \\ S' \rightarrow \text{else } S \mid \lambda \end{array}$$

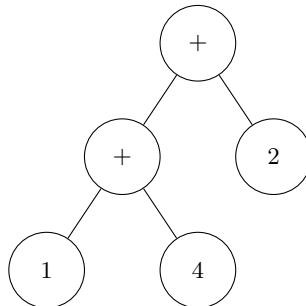
O prefixo **if** E **then** S é fatorado e o sufixo opcional **else** S é tratado pelo novo não-terminal S' . Com essa gramática, ao ver **else** o analisador expande $S' \rightarrow \text{else } S$; caso contrário, expande $S' \rightarrow \lambda$.

Nota: fatoração à esquerda pode precisar ser aplicada repetidamente se, após uma primeira rodada, novas produções passarem a compartilhar prefixos. Além disso, nem toda gramática pode ser tornada LL(1) apenas com remoção de recursão à esquerda e fatoração: algumas gramáticas são inerentemente ambíguas ou requerem lookahead maior.

5.2.5 Árvores de sintaxe abstrata

Uma árvore sintática abstrata (AST) é uma representação intermediária de alto nível de um programa, na qual as construções da linguagem de programação são os nós da árvore. De fato, um teórico de linguagens de programação provavelmente consideraria a AST como o programa verdadeiro, sendo sua representação original analisada sintática uma necessidade infeliz para obtê-lo.

É importante distinguir entre a árvore sintática abstrata de um programa e a árvore de análise sintática. A árvore de análise sintática descreve como usar as produções da gramática para derivar a entrada, mas inclui nós que são desnecessários para os estágios posteriores do compilador. Por exemplo, se estiver usando uma gramática aritmética simples, a AST para a expressão $(1+4)*2$, apresentada na figura a seguir



não incluiria nós para representar parênteses ou nós para não terminais. A melhor forma de representar uma AST depende da linguagem de programação. Em uma linguagem funcional como Haskell, é conveniente projetar a AST como um tipo de dados algébrico que usa um construtor para cada regra presente na gramática. A seguir, apresentamos o tipo **Exp** que representa a árvore de sintaxe abstrata para a gramática de expressões.

```

data Exp
  = EInt Int
  | Exp :+: Exp
  | Exp **: Exp

```

5.2.6 Outros problemas na análise sintática

Algumas linguagens de programação apresentam desafios adicionais para a análise sintática. Por exemplo, o uso significativo de espaços em branco (após um hiato de várias décadas) voltou a ser comum em linguagens como Python, Haskell e JavaScript. Os níveis de indentação em Python e Haskell sinalizam implicitamente o agrupamento de expressões. A estratégia usual para analisar linguagens com essa característica é o analisador léxico inserir tokens adicionais nos pontos onde o nível de indentação muda. Esses tokens são conhecidos como tokens de indentação/desindentação. Eles podem então ser usados na gramática da linguagem como se fossem parênteses ou chaves invisíveis.

Outro desafio são as linguagens que distinguem diferentes categorias de identificadores. O analisador léxico deve gerar diferentes tipos de tokens para diferentes tipos de identificadores. Em algumas linguagens, como OCaml, essa distinção não é um problema, pois a categoria do identificador pode ser determinada na análise léxica. No entanto, as linguagens C e C++ são mais difíceis de lidar. Por exemplo, a análise sintática correta para a declaração

```
HashTable<K,V> x;
```

depende se `HashTable` é o nome de uma classe. Se for o nome de uma variável inteira, a instrução descreve, na verdade, duas comparações entre quatro variáveis!

Para gerar diferentes tipos de tokens para diferentes tipos de identificadores, são necessárias informações de estágios posteriores do compilador durante a análise léxica. Esse feedback do analisador léxico torna o compilador mais complexo e frágil, já que a compilação deixa de ser uma simples sequência linear de transformações.

Outra técnica para lidar com análises sintáticas ambíguas é adiar o problema para fases posteriores do compilador, quando mais informações estiverem disponíveis, como a árvore sintática abstrata completa do programa. Essa abordagem funciona bem quando não é necessário resolver a ambiguidade

para obter a estrutura da árvore sintática correta (o que não é o caso em C++, como mostra o exemplo).

5.3 Conclusão

Esse capítulo apresentou uma introdução à análise sintática. Primeiro, demonstramos por que as expressões regulares são insuficientes para lidar com a estrutura recursiva e balanceada das linguagens de programação. Através do uso de Gramáticas Livres de Contexto (GLCs), foi possível formalizar a sintaxe de expressões e resolver problemas de ambiguidade, precedência e associatividade. Discutimos o conceito de Árvore de Sintaxe Abstrata (AST), que serve como uma representação simplificada e essencial para as fases subsequentes de análise semântica e geração de código.

Em seguida, apresentamos duas transformações fundamentais sobre gramáticas livres de contexto: a **remoção de recursão à esquerda**, que elimina produções que causariam loops infinitos em analisadores descendentes, e a **fatoração à esquerda**, que elimina ambiguidades de escolha entre alternativas com prefixo comum. Essas transformações são pré-requisitos para a construção de analisadores LL(1), que estudaremos em capítulos posteriores.

Por fim, discutiu-se como desafios modernos, como a indentação significativa e identificadores dependentes de contexto, exigem estratégias sofisticadas para sua solução.

Capítulo 6

Análise descendente recursiva

6.1 Introdução

Neste capítulo vamos estudar a primeira técnica para construção de analisadores sintáticos: a análise descendente recursiva. Essa técnica permite a construção de um analisador sintático como um conjunto de funções, uma para cada não terminal da gramática da linguagem considerada.

Em linguagens funcionais, como Haskell, uma abordagem elegante para construção de analisadores descendentes recursivos utiliza os chamados **combinadores de parsing**, que consistem de um funções para construir analisadores simples e outras funções para combinar analisadores existentes. Essa técnica permite descrever a gramática da linguagem de forma direta. O código discutido neste capítulo está disponível nos arquivos

```
Parsing/Descendent/Combinators.hs,  
Parsing/Descendent/Expr.hs e  
Parsing/Descendent/ExpExample.hs.
```

6.2 Combinadores de parsing

A ideia central por trás dos combinadores de parsing é tratar um analisador sintático como um **valor de primeira classe**: um analisador é simplesmente uma função que recebe uma lista de símbolos e retorna os possíveis resultados da análise. Essa representação permite combinar analisadores simples para construir analisadores mais complexos usando funções de ordem superior.

6.2.1 O tipo Parser

O tipo fundamental da biblioteca é definido como:

```
newtype Parser s a =  
  Parser { runParser :: [s] -> [(a,[s])] }
```

Um valor do tipo `Parser s a` representa um analisador que:

- Consome símbolos do tipo `s` (por exemplo, `Char` ou algum tipo de token);
- Produz resultados do tipo `a` (por exemplo, uma árvore de sintaxe abstrata).

A função `runParser` extrai a função subjacente, que recebe a entrada como uma lista de símbolos `[s]` e devolve uma lista de pares `[(a, [s])]`. Cada par contém um possível resultado da análise e o restante da entrada ainda não consumida.

O uso de uma lista de resultados é o que torna esta representação poderosa: a lista vazia, `[]`, sinaliza **falha** (nenhum resultado possível), enquanto uma lista com múltiplos elementos representa um analisador **ambíguo**, com mais de uma interpretação possível para a entrada. Um analisador determinístico retornará sempre no máximo um elemento na lista.

6.2.2 Instâncias de classes de tipos

A estrutura algébrica do tipo `Parser` é rica: ele pode ser instanciado em várias classes de tipos de Haskell, o que nos dá gratuitamente um conjunto de combinadores expressivos.

Functor. Um parser é um functor: dado um parser que produz valores do tipo `a` e uma função `f :: a -> b`, podemos construir um parser que produz valores do tipo `b`, aplicando `f` a cada resultado.

```
instance Functor (Parser s) where
  fmap f p
    = Parser (\ s -> [(f x, s') |
                      (x,s') <- runParser p s])
```

Applicative. Um parser é um functor aplicativo. O combinador `pure` cria um functor que sempre tem sucesso sem consumir entrada, retornando o valor fornecido. O operador `<*>` sequencia dois parsers: o primeiro produz uma função e o segundo produz um argumento; os resultados são combinados.

```
instance Applicative (Parser s) where
  pure a = Parser (\ts -> [(a, ts)])
  p1 <*> p2 = Parser (\ s -> [(f x, s2) |
                              (f, s1) <- runParser p1 s,
                              (x, s2) <- runParser p2 s1])
```

Alternative. A instância de `Alternative` nos fornece o combinador de escolha. O parser `empty` falha sempre. O operador `<|>` tenta o primeiro parser e o segundo parser. Porém, dada a lazy evaluation de Haskell, o segundo parser só será realmente executado se o primeiro falhar. Isso se deve devido a concatenação dos resultados de ambos os parsers.

```
instance Alternative (Parser t) where
  empty = Parser (\ _ -> [])
  p1 <|> p2 = Parser
    (\ s -> runParser p1 s ++
            runParser p2 s)
```

Monad. A instância de `Monad` permite que o segundo parser dependa do resultado do primeiro.

```
instance Monad (Parser t) where
  return = pure
  p >>= f = Parser (\ts -> concat [ runParser (f a) cs' |
                                    (a,cs') <- runParser p ts ])
```

Com essa instância, é possível escrever analisadores em notação `do`, tornando o código mais legível para parsers com múltiplos estágios sequenciais.

MonadPlus. Essa instância espelha a instância de `Alternative`, fornecendo `mzero` (falha) e `mplus` (escolha) para uso no contexto monádico.

6.2.3 Combinadores primitivos

A biblioteca oferece dois parsers primitivos que servem como base para todos os demais.

A função `item` consome exatamente um símbolo da entrada, falhando se a entrada estiver vazia:

```
item :: Parser t t
item = Parser (\ ts ->
  case ts of
    []      -> []
    (c:cs) -> [(c,cs)])
```

O combinador `sat` consome um símbolo somente se ele satisfaz um predicado:

```
sat :: (t -> Bool) -> Parser t t
sat p = do
    t <- item
    if p t then return t else mzero
```

A partir de `sat`, derivamos diretamente os combinadores para reconhecer símbolos e sequências específicas:

```
symbol :: Eq s => s -> Parser s s
symbol c = sat (c ==)

token :: Eq s => [s] -> Parser s [s]
token = mapM symbol
```

`symbol c` consome exatamente o símbolo `c`; `token ts` consome exatamente a sequência `ts`, aplicando `symbol` a cada elemento.

6.2.4 Combinadores de alto nível

Utilizando as funções primitivas, podemos construir um conjunto de combinadores de alto nível.

Repetição. A classe `Alternative` fornece `many` (zero ou mais repetições) e `many1` (uma ou mais):

```
many1 :: Parser s a -> Parser s [a]
many1 p = list <$> p <*> many p
```

Opção. O combinador `option` executa um parser opcionalmente, retornando um valor padrão em caso de falha:

```
option :: Parser s a -> a -> Parser s a
option p v = p <|> succeed v
```

Pack. O combinador `pack` sequencia três parsers, descartando o resultado do primeiro e do terceiro e retornando apenas o resultado do segundo. É útil para reconhecer construções entre delimitadores:

```
pack :: Parser s a -> Parser s b -> Parser s c -> Parser s b
pack p r q = pi32 <$> p <*> r <*> q
```

Por exemplo, `pack (symbol '(') p (symbol ')')` reconhece `p` entre parênteses.

Separadores. O combinador `listOf` reconhece uma lista de itens separados por um delimitador:

```
listOf :: Parser s a -> Parser s b -> Parser s [a]
listOf p s = list <$> p <*> many (pi22 <$> s <*> p)
```

Repetição gulosa. O combinador `determ` trunca a lista de resultados ao primeiro, tornando o parser determinístico. Sobre ele, `greedy` e `greedy1` implementam versões determinísticas de `many` e `many1`:

```
determ :: Parser s b -> Parser s b
determ p = Parser (\ ts ->
    case (runParser p ts) of
        [] -> []
        (x:_) -> [x] )

greedy, greedy1 :: Parser s b -> Parser s [b]
greedy = determ . many
greedy1 = determ . many1
```


6.2.5 Aplicações para tipos concretos

Com os combinadores acima, a biblioteca define analisadores para construções léxicas comuns sobre `Parser Char`:

```
digit :: Parser Char Int
digit = f <$> sat isDigit
  where
    f c = ord c - ord '0'

natural :: Parser Char Int
natural = foldl (\a b -> a*10 + b) 0 <$> many1 digit

integer :: Parser Char Int
integer = (const negate <$> (symbol '-')) `option` id <*> natural

identifier :: Parser Char String
identifier = list <$> sat isAlpha <*> greedy (sat isAlphaNum)

parens :: Parser Char a -> Parser Char a
parens p = pack (symbol '(') p (symbol ')')

spaces :: Parser Char String
spaces = greedy (sat isSpace)
```

6.2.6 Combinadores para encadeamento de operadores

Um padrão muito comum em gramáticas de linguagens de programação é a expressão com operadores infixos associativos. Para não-terminais da forma

$$E \rightarrow E \oplus T \mid T$$

(associatividade à esquerda) ou

$$E \rightarrow T \oplus E \mid T$$

(associatividade à direita), a biblioteca oferece os combinadores `chainl` e `chainr`:

```
chainr :: Parser s a -> Parser s (a -> a -> a) -> Parser s a
chainr pe po = h <$> many (j <$> pe <*> po) <*> pe
  where j x op = (x `op`)
        h fs x = foldr ($) x fs

chainl :: Parser s a -> Parser s (a -> a -> a) -> Parser s a
chainl pe po = h <$> pe <*> many (j <$> po <*> pe)
  where j op x = (`op` x)
        h x fs = foldl (flip ($)) x fs
```

O argumento `pe` é o analisador para os operandos (expressões de nível mais alto) e `po` é o analisador para os operadores, que devolve uma **função binária** a ser aplicada aos operandos.

Por exemplo, `chainl termParser opParser` reconhece a gramática

$$E \rightarrow E + T \mid T$$

produzindo uma árvore associativa à esquerda: a expressão `1 + 2 + 3` é interpretada como `(1 + 2) + 3`.

6.3 O parser de expressões

Para ilustrar o uso da biblioteca, vamos examinar o analisador sintático para uma linguagem simples de expressões aritméticas com variáveis. A linguagem é dada pela seguinte gramática:

$$\begin{aligned} E &\rightarrow E + T \mid T \\ T &\rightarrow T * F \mid F \\ F &\rightarrow n \mid x \mid (E) \end{aligned}$$

onde n representa literais inteiros e x representa identificadores. A implementação está dividida em dois estágios, como é habitual: a **análise léxica**, que converte a string de entrada em uma lista de tokens, e a **análise sintática**, que consome os tokens e produz uma AST.

6.3.1 Tokens e análise léxica

Os tokens da linguagem de expressões são definidos pelo tipo:

```
data Token
  = Id String
  | Number Int
  | Add
  | Mult
  | LParen
  | RParen
```

O analisador léxico é construído com os próprios combinadores da biblioteca. O lexer principal elimina espaços em branco e aplica `tokenLexer` repetidamente:

```
lexer' :: Parser Char [Token]
lexer' = (\_ x -> x) <$> spaces
        <*> many ((\x _ -> x) <$> tokenLexer <*> spaces)
```

O analisador `tokenLexer` tenta reconhecer, em ordem, um identificador, um número ou um operador/delimitador. Para os operadores e delimitadores, utiliza-se uma tabela de associação entre caracteres e tokens:

```
tokenLexer :: Parser Char Token
tokenLexer = (Id <$> identifier) <|> number <|> tokenlex
```

```
table :: [(Char, Token)]
table = [('+', Add), ('*', Mult),
        ('(', LParen), (')', RParen)]
```

```
tokenlex :: Parser Char Token
tokenlex = choice $ map (\(c,t) -> const t <$> symbol c) table
```

O uso de `choice` com a tabela é um padrão limpo: para cada par (c, t) na tabela, construímos um parser que reconhece o caractere c e retorna o token t . `choice` combina todos esses parsers com `<|>`.

A função `exprLexer` executa o lexer e extrai o único resultado esperado:

```
exprLexer :: String -> [Token]
exprLexer s
  = case runParser lexer' s of
      [] -> []
      ((x, " "):_) -> x
      v -> error $ show v
```

O padrão `(x, "")` garante que toda a entrada foi consumida; caso haja entrada residual, um erro é sinalizado.

6.3.2 A árvore de sintaxe abstrata

A AST para as expressões é representada pelo tipo algébrico:

```
data Expr
  = Var String
  | Lit Int
  | Expr :+: Expr
  | Expr *: Expr
  deriving (Eq, Show)
```

Os construtores `(:+:)` e `(:*)` são operadores infixos definidos no tipo, o que torna o código do parser notavelmente próximo da gramática.

6.3.3 O combinador gen para gramáticas com operadores

Antes de apresentar o analisador sintático, é necessário introduzir o combinador auxiliar `gen`, definido em `Parsing/Descendent/Expr.hs`:

```
type Op s a = (s, a -> a -> a)

gen :: Eq s => [Op s a] -> Parser s a -> Parser s a
gen ops p = chainl p (choice (map f ops))
  where
    f (s,c) = const c <$> symbol s
```

O tipo `Op s a` associa um símbolo `s` (um token) a uma função binária que combina dois valores do tipo `a`. O combinador `gen` recebe uma lista de tais associações e um parser para os operandos, e constrói um parser para expressões com os operadores dados, com associatividade à esquerda.

Internamente, `gen` constrói o parser do operador com `choice (map f ops)`: para cada par `(s, c)`, cria um parser que consome o token `s` e retorna a função `c`. Em seguida, aplica `chainl` para obter a associatividade à esquerda.

6.3.4 O analisador sintático

O analisador para expressões segue a gramática com três níveis de precedência:

```
exprParser :: Parser Token Expr
exprParser = addtable `gen` termParser
  where
    addtable = [(Add, (:+:))]

termParser :: Parser Token Expr
termParser = multable `gen` factParser
  where
    multable = [(Mult, (:*))]

factParser :: Parser Token Expr
factParser = numParser <|>
  varParser <|>
  parenExpr
```

```

where
  parenExpr = pack lparen exprParser rparen
  lparen = symbol LParen
  rparen = symbol RParen

```

A estrutura do parser espelha diretamente a gramática:

- `exprParser` reconhece somas de termos ($E \rightarrow T + E \mid T$). Utiliza `gen` com a tabela de adição, tendo `termParser` como analisador dos operandos.
- `termParser` reconhece produtos de fatores ($T \rightarrow F * T \mid F$). Utiliza `gen` com a tabela de multiplicação, tendo `factParser` como analisador dos operandos.
- `factParser` reconhece fatores atômicos ($F \rightarrow n \mid x \mid (E)$): literais inteiros, variáveis ou expressões parentesadas.

Os analisadores para variáveis e números usam `sat` para verificar o construtor do token:

```

varParser :: Parser Token Expr
varParser = f <$> sat isVar
  where
    isVar (Id _) = True
    isVar _      = False
    f (Id v) = Var v
    f _      = error "impossible!"

numParser :: Parser Token Expr
numParser = f <$> sat isNum
  where
    isNum (Number _) = True
    isNum _          = False
    f (Number n) = Lit n
    f _          = error "impossible!"

```

A função de entrada do parser converte a lista de tokens em um valor `Either`, retornando a AST ou uma mensagem de erro:

```

expr :: [Token] -> Either String Expr
expr ts
  = case runParser exprParser ts of
    ((t, []) : _) -> Right t
    _              -> Left "Parse error"

```

O padrão `(t, [])` novamente exige que toda a entrada tenha sido consumida.

6.4 O parser de expressões com Megaparsec

A biblioteca desenvolvida nas seções anteriores é didática, mas bibliotecas de produção vão além: oferecem mensagens de erro detalhadas, melhor desempenho e mais funcionalidades. O **Megaparsec** é a biblioteca de combinadores de parsing mais utilizada no ecossistema Haskell e segue os mesmos princípios conceituais que estudamos, porém com uma implementação muito mais robusta. O código discutido nesta seção está disponível em:

```

Exp/Frontend/Lexer/Megaparsec/ExpLexer.hs e
Exp/Frontend/Parser/Megaparsec/ExpParser.hs.

```

6.4.1 O tipo Parser no Megaparsec

Nessa seção, vamos utilizar a biblioteca Megaparsec para definir um analisador descendente recursivo para a linguagem de expressões. Definimos o tipo `Parser` é definido como um alias para `Parsec Void String`:

```
type Parser = Parsec Void String
```

O tipo `Parsec` e `s` a do Megaparsec é parametrizado pelo tipo de **erro** `|e|`, pelo tipo de **entrada** `s` e pelo tipo do **resultado** `a`. Usando `Void` como tipo de erro customizado, indicamos que só utilizamos as mensagens de erro padrão da biblioteca. `String` é o tipo da entrada: uma lista de caracteres.

6.4.2 A análise léxica com Megaparsec

O Megaparsec separa claramente as responsabilidades léxicas das sintáticas. O módulo

```
Exp.Frontend.Lexer.Megaparsec.ExpLexer
```

define os primitivos léxicos que o parser irá consumir.

O primeiro componente é o **consumidor de espaço** `sc` (*space consumer*):

```
sc :: Parser ()
sc = L.space space1 lineCmnt blockCmnt
where
  lineCmnt = L.skipLineComment "//"
  blockCmnt = L.skipBlockComment "/*" "*/"
```

`L.space` recebe três parsers: um para espaços em branco (`space1`, que consome um ou mais caracteres brancos), um para comentários de linha e um para comentários em bloco. Com isso, `sc` descarta silenciosamente espaços e ambas as formas de comentário.

Sobre `sc`, é definido o combinador `lexeme`, que envolve qualquer parser consumindo automaticamente o espaço que o segue:

```
lexeme :: Parser a -> Parser a
lexeme = L.lexeme sc
```

O padrão de uso é: cada parser léxico é definido com `lexeme`, garantindo que o espaço entre tokens seja descartado sem que o parser sintático precise se preocupar com isso.

Os demais primitivos léxicos são diretos:

```
symbol :: String -> Parser String
symbol = L.symbol sc

parens :: Parser a -> Parser a
parens = between (symbol "(") (symbol ")")

rword :: String -> Parser ()
rword w = (lexeme . try) (string w *> notFollowedBy alphaNumChar)

int :: Parser Int
int = lexeme L.decimal
```

- `symbol` reconhece uma string literal e descarta o espaço seguinte.
- `parens p` reconhece `p` entre parênteses, usando `between` do Megaparsec para combinar os dois delimitadores com o conteúdo.

- `rword` reconhece uma **palavra reservada**: usa `try` para retroceder em caso de falha e `notFollowedBy alphaNumChar` para garantir que a palavra não seja prefixo de um identificador maior.
- `int` analisa um literal decimal inteiro, descartando o espaço seguinte.

6.4.3 O analisador sintático com `makeExprParser`

O analisador sintático importa do módulo `Control.Monad.Combinators.Expr` a função `makeExprParser`, que é a versão de nível industrial do nosso `gen` combinado com `chainl/chainr`:

```
import Control.Monad.Combinators.Expr
```

A precedência e associatividade dos operadores são declaradas em uma **tabela de operadores**, organizada do maior para o menor nível de precedência (ou seja, os operadores que ligam mais forte vêm primeiro):

```
opTable :: [[Operator Parser Exp]]
opTable
  = [ [ infixL (:*) " *" ]
    , [ infixL (:+) "+" ]
    ]
where
  infixL op sym = InfixL $ op <$ symbol sym
```

A tabela é uma lista de listas: cada lista interna agrupa operadores de mesma precedência. `(:*)` aparece na primeira lista e `(: +)` na segunda, logo `*` tem maior precedência que `+`. Dentro de cada grupo, os operadores têm a mesma precedência e a associatividade é determinada pelo construtor utilizado: `InfixL` para associatividade à esquerda.

A função auxiliar `infixL` constrói uma entrada na tabela: `op <$ symbol sym` reconhece o símbolo `sym` e retorna a função construtora `op` (como `(:*)`) como resultado, que será aplicada pelos dois operandos adjacentes.

O parser para fatores atômicos é `termP`:

```
termP :: Parser Exp
termP = parens expP
      <|> EInt <$> int
```

`termP` tenta, em ordem: uma expressão entre parênteses (`parens expP`) ou um literal inteiro (`EInt <$> int`). Note que `termP` é mutuamente recursivo com `expP`: parênteses reiniciam o parsing a partir do nível mais alto da gramática.

Finalmente, o parser principal é criado utilizando a tabela e o parser `termP`:

```
expP :: Parser Exp
expP = makeExprParser termP opTable
```

`makeExprParser` recebe o parser `termP` e a tabela de operadores, e constrói automaticamente o parser para toda a hierarquia de precedências, evitando a necessidade de codificar manualmente cada nível como uma função separada.

6.4.4 A função principal de parsing

A função que executa o parser `expParser` sobre uma `String` é:

```

expParser :: String -> Either String Exp
expParser s = case parse expP "" s of
    Left err -> Left (errorBundlePretty err)
    Right e  -> Right e

```

`parse` é a função de execução do Megaparsec: recebe o parser, um nome de arquivo (aqui "", pois não há arquivo físico) e a entrada. Em caso de falha, `errorBundlePretty` formata a mensagem de erro de forma legível, com indicação de linha, coluna e o que era esperado. Em caso de sucesso, a AST é retornada em `Right`.

6.5 Conclusão

Este capítulo apresentou os combinadores de parsing como uma abordagem funcional para a construção de analisadores sintáticos recursivos descendentes. A ideia central, representar um parser como uma função de listas de símbolos para listas de resultados, permite instanciar o tipo `Parser` nas classes `Functor`, `Applicative`, `Alternative` e `Monad`.

A partir dos primitivos `item` e `sat`, construímos combinadores de alto nível como `many`, `option`, `pack`, `chainl` e `chainr`. Esses combinadores foram aplicados para construir um analisador léxico e um analisador sintático para uma linguagem de expressões aritméticas, demonstrando como a estrutura da gramática se reflete diretamente na estrutura do código Haskell.

A combinação de `gen` com `chainl` resolve elegantemente o problema clássico da recursão à esquerda em gramáticas de operadores, sem a necessidade de transformar a gramática original. O resultado é um analisador conciso, legível e corretamente associativo que serve de base para o frontend de um compilador.

Por fim, apresentamos o mesmo parser de expressões reescrito com o **Megaparsec**, uma biblioteca de produção que compartilha os mesmos fundamentos conceituais, mas oferece mensagens de erro detalhadas (com linha, coluna e contexto), tratamento automático de espaço em branco via `lexeme` e o combinador `makeExprParser` para declarar hierarquias de precedência de forma compacta através de uma tabela de operadores. A principal diferença comportamental é a semântica de *committed choice* do operador `<|>`: ao contrário da biblioteca desenvolvida neste capítulo, o Megaparsec só tenta a alternativa direita se a esquerda falhou sem consumir entrada, o que torna o parser eficiente e as mensagens de erro mais precisas.

Capítulo 7

Parsing expression grammars

7.1 Introdução

No capítulo anterior, estudamos os combinadores de parsing como uma forma de implementar analisadores sintáticos descendentes recursivos. Neste capítulo, vamos estudar um formalismo que permite a definição de analisadores sintáticos diretamente, sem a necessidade de usar uma gramática livre de contexto. Esse formalismo são as **Parsing Expression Grammars** (PEGs), introduzidas por Bryan Ford em 2004.

As PEGs compartilham com as gramáticas livres de contexto (GLCs) a ideia de descrever linguagens por meio de regras de reescrita, mas diferem em um ponto fundamental: enquanto as GLCs usam **escolha não-determinística** entre as alternativas de uma produção, as PEGs usam **escolha ordenada**. Em vez de tentar todas as alternativas em paralelo (como faz um autômato não-determinístico), uma PEG experimenta as alternativas em ordem e se compromete com a primeira que tiver sucesso. Essa semântica determinística elimina a ambiguidade presente em gramáticas e expressões regulares.

O código discutido neste capítulo está disponível em

PEG/Semantics/Simple.hs.

7.2 Sintaxe e semântica de PEGs

Uma PEG é composta por um conjunto de **regras de parsing**, cada uma associando um nome (não-terminal) a uma **expressão de parsing**. As expressões de parsing são construídas a partir dos seguintes operadores:

Expressão	Nome	Descrição
ε	vazio	Sempre tem sucesso sem consumir entrada
a	terminal	Consome e casa com o símbolo a
A	não-terminal	Invoca a regra de nome A
$e_1 e_2$	sequência	Tenta e_1 e depois e_2
e_1/e_2	escolha ordenada	Tenta e_1 ; se falhar, tenta e_2
e^*	repetição	Zero ou mais repetições de e (gulosa)
e^+	repetição positiva	Uma ou mais repetições de e
$e?$	opcional	Zero ou uma ocorrência de e
$!e$	predicado negativo	Sucesso se e falhar; não consome entrada
$\&e$	predicado positivo	Sucesso se e tiver sucesso; não consome entrada

Tabela 7.1: Operadores de PEG

Uma PEG é então um conjunto de regras da forma $A \leftarrow e$, em que A é um não-terminal e e é uma expressão de parsing. Uma das regras é designada como **regra inicial**.

Como exemplo, a seguinte PEG descreve a linguagem de expressões aritméticas com precedência e associatividade já embutidas na gramática:

$$\begin{aligned} E &\leftarrow T(+T)^* \\ T &\leftarrow F(*F)^* \\ F &\leftarrow (E) / [0-9]^+ \end{aligned}$$

Comparada à GLC equivalente, a PEG não precisa de produções recursivas à esquerda para expressar associatividade à esquerda: o operador $(\cdot)^*$ captura diretamente a ideia de “zero ou mais ocorrências à direita”, que é processada iterativamente.

A semântica de uma PEG é definida em termos de uma função de parsing que, dado uma expressão e e uma entrada s , retorna ou um **sucesso** que consiste de um par formado com a parte da entrada consumida e o restante ainda não processado; ou uma **falha**.

A diferença crucial em relação às GLCs está na semântica da **escolha ordenada** e_1/e_2 : Tenta-se e_1 sobre a entrada s . Se e_1 **tem sucesso**, o resultado é imediatamente retornado e e_2 nunca é tentada. Se e_1 **falha**, então e_2 é executada sobre a entrada s .

7.2.1 Predicados

Os predicados $!e$ e $\&e$ permitem analisar parte da entrada sem necessariamente consumir parte dela. Temos os seguintes operadores de predicado:

- $!e$ (**predicado negativo**): tem sucesso se e somente se e falha. Nenhuma entrada é consumida em nenhum caso.
- $\&e$ (**predicado positivo**): tem sucesso se e somente se e tem sucesso. Nenhuma entrada é consumida. É equivalente a $!(!e)$.

Os predicados são úteis para especificar restrições contextuais sem avançar na entrada. Por exemplo, para reconhecer uma palavra reservada como `if` sem aceitar um identificador como `iff`, podemos escrever:

`if ! [a-zA-Z0-9]`

Isso garante que `|if|` só é reconhecido quando não seguido de um caractere alfanumérico.

7.2.2 Repetição

O operador e^* é definido recursivamente como:

$$e^* \leftarrow e e^* / \varepsilon$$

Como a escolha é ordenada, a repetição é **gulosa**: o parser sempre tenta consumir mais uma ocorrência de e antes de aceitar o caso vazio. Isso garante que e^* sempre consumirá o maior prefixo possível que case com e .

Ao contrário das GLCs, não há ambiguidade em e^* : o resultado é determinístico e único para qualquer entrada.

7.2.3 Terminação em PEGs

PEGs podem ser vistas como uma representação de analisadores descendentes recursivos e, portanto, possuem limitações similares. A terminação do processo de parsing de um PEG é garantido de terminar se a gramática atende as seguintes restrições:

- Não possui regras recursivas à esquerda, diretas ou indiretas;

- Toda ocorrência de repetição, e^* é tal que e não deve ter sucesso sem consumir símbolos da entrada.

A primeira restrição é óbvia: pois faz com que analisadores descendentes entrem em loop. A segunda é necessária devido à semântica gulosa do operador de repetição: ele sempre tentará casar a expressão e antes de finalizar a repetição. Se e tiver sucesso sem consumir entrada, então e^* pode, a cada iteração, consumir um prefixo vazio da entrada, o que leva a não terminação.

Dizemos que uma PEG é **bem formada** se ela atende as duas restrições mencionadas. Para PEGs bem formadas há uma garantia formal de que o processo de parsing sempre termina.

7.3 Implementação em Haskell

A implementação de PEGs em Haskell presente em `PEG/Semantics/Simple.hs` segue diretamente a semântica apresentada. O ponto central é representar explicitamente se o parser consumiu ou não parte da entrada, pois isso determina quando a escolha ordenada pode retroceder.

7.3.1 O tipo Result

O resultado de um parser é representado pelo tipo:

```
data Result d a
= Pure a           -- sucesso sem consumir entrada
| Commit d a       -- sucesso consumindo entrada; d: o resto da entrada
| Fail String Bool -- falha; Bool indica se houve consumo
```

Os três construtores capturam os três casos possíveis:

- **Pure** a : o parser teve sucesso sem consumir nenhum símbolo. O resultado pode ser descartado e o parser alternativo pode ser tentado sobre a mesma entrada.
- **Commit** $d' a$: o parser teve sucesso e consumiu parte da entrada; d' é o restante. Falhas posteriores não causarão backtracking sobre este ponto.
- **Fail** $s c$: o parser falhou; a **String** s descreve o erro e o **Bool** c indica se houve consumo de entrada antes da falha.

7.3.2 O tipo PExp

Um parser PEG é um valor do tipo:

```
newtype PExp d a
= PExp { runPExp :: d -> Result d a }
```

PExp $d a$ representa um parser que lê de uma entrada do tipo d (tipicamente **String**) e produz um resultado do tipo a . O tipo **PExp** é uma instância de **Functor** que permite modificar o valor de resultado do parser.

7.3.3 Instâncias e combinadores

A instância de **Applicative** define a composição sequencial de dois parsers propagando o estado de consumo:

```
instance Applicative (PExp d) where
  pure a = PExp $ \ _ -> Pure a
  PExp mf <*> PExp ma
    = PExp $ \ d ->
      case mf d of
        Pure f      -> fmap f (ma d)
        Fail s c    -> Fail s c
        Commit d' f ->
          case ma d' of
            Pure a      -> Commit d' (f a)
            Fail s _    -> Fail s True
            Commit d'' a -> Commit d'' (f a)
```

O caso `Commit d' f` é o mais importante: se o primeiro parser consumiu entrada, qualquer falha do segundo parser é marcada como `Fail s True`, que indica que o consumo ocorreu, sem possibilidade de retrocesso.

A instância de `Alternative` implementa a semântica da **escolha**:

```
instance Alternative (PExp d) where
  PExp ma <|> PExp mb
    = PExp $ \ d ->
      case ma d of
        Fail _ False -> mb d -- falhou SEM consumir: tenta alternativa
        x             -> x    -- sucesso OU falhou COM consumo: retorna
  empty = PExp $ \ _ -> Fail "empty" False
```

O padrão `Fail \_ False` é a chave: o segundo parser `mb` só é tentado se o primeiro falhou **sem consumir entrada** (`False`). Se o primeiro consumiu entrada e falhou (`Fail \_ True`), o erro é propagado imediatamente, sem retrocesso.

7.3.4 O combinador try

Para permitir retrocesso mesmo quando já houve consumo, a biblioteca oferece o combinador `try`:

```
try :: PExp d a -> PExp d a
try (PExp m)
  = PExp $ \d ->
    case m d of
      Fail s _ -> Fail s False -- sempre marca como "sem consumo"
      x        -> x
```

`try p` executa `p` normalmente. Se `p` falha, com ou sem consumo, `try` marca o resultado como `Fail s False`, sinalizando que nenhum consumo ocorreu para fins de retrocesso. Isso efetivamente “cancela” qualquer consumo parcial de `p`, permitindo que outras alternativas, caso existam, sejam tentadas.

7.3.5 A escolha ordenada

Combinando `try` e `<|>`, a biblioteca define o operador de escolha ordenada:

```
infixl 3 </>

(</>) :: PExp d a -> PExp d a -> PExp d a
p </> q = try p <|> q
```

`p </> q` tenta `p` com backtracking garantido: se `p` falha por qualquer razão, `q` é tentado sobre a entrada original.

7.3.6 O predicado not

O combinador `not` implementa o predicado negativo `!`:

```
not :: PExp d a -> PExp d ()
not (PExp m)
  = PExp $ \d ->
    case m d of
      Fail{} -> Pure ()      -- e falhou: sucesso sem consumir
      _      -> Fail "unexpected" False -- e teve sucesso: falha
```

`not p` tem sucesso se e somente se `p` falha, e nunca consome entrada. Sobre ele, `eof` é definido como:

```
eof :: Stream d => PExp d ()
eof = not anyChar
```

`eof` tem sucesso quando não há mais entrada disponível.

7.3.7 Combinadores léxicos

A biblioteca oferece combinadores para reconhecimento de tokens comuns:

```
satisfy :: Stream d => (Char -> Bool) -> PExp d Char
satisfy p = try $ do
  x <- anyChar
  x <$ guard (p x)

char :: Stream d => Char -> PExp d Char
char c = satisfy (c ==)

lexeme :: Stream d => PExp d a -> PExp d a
lexeme m = m <*> whiteSpace

symbol :: Stream d => Char -> PExp d Char
symbol c = lexeme (char c)

digit :: Stream d => PExp d Int
digit = f <$> satisfy isDigit
  where f c = ord c - ord '0'
```

O combinador `satisfy` usa `try` para garantir que, se o predicado falha (o caractere foi consumido por `anyChar` mas não satisfaz `p`), o parser retrocede e marca a falha como sem consumo. Isso é necessário para que `satisfy` se comporte corretamente como primitivo: uma falha em `satisfy` não deve comprometer a entrada.

O combinador `lexeme` descarta espaços em branco após o item reconhecido, e `symbol c` combina o reconhecimento de um caractere específico com descarte de espaço, o mesmo padrão que vimos no Megaparsec.

7.4 Exemplo: parser de expressões

Para ilustrar o uso da biblioteca, vamos construir um parser de expressões aritméticas. A gramática PEG é:

$$\begin{aligned} E &\leftarrow T (+T)^* \\ T &\leftarrow F (*F)^* \\ F &\leftarrow (E) / [0-9]^+ \end{aligned}$$

Em Haskell, usando os combinadores da biblioteca:

```
data Exp = Lit Int | Exp :+: Exp | Exp **: Exp

expP :: Stream d => PExp d Exp
expP = foldl1 (:+:) <$> termP <*> many (symbol '+' *> termP)

termP :: Stream d => PExp d Exp
termP = foldl1 (:**:) <$> factP <*> many (symbol '*' *> factP)

factP :: Stream d => PExp d Exp
factP = between (symbol '(') (symbol ')') expP
      </> Lit . foldl1 (\a b -> a * 10 + b) 0 <$> many1 digit
```

A estrutura do parser segue diretamente a gramática PEG. Em `factP`, o operador `</>` garante que, se a tentativa de reconhecer uma expressão com parêntesis falhar, o parser retroceda e tente reconhecer um literal inteiro.

Note que `expP` e `termP` usam `many` com `foldl1` para construir a AST com associatividade à esquerda, sem precisar de recursão à esquerda nas regras. Isso é uma vantagem das PEGs sobre as GLCs: associatividade à esquerda é expressa naturalmente por repetição, sem introduzir recursão problemática para parsers descendentes.

7.5 PEGs e GLCs: principais diferenças

As PEGs e as GLCs são formalismos distintos para descrever linguagens. As principais diferenças práticas são:

Ambiguidade. Toda PEG é, por definição, não-ambígua: a escolha ordenada garante que exista no máximo um resultado para qualquer entrada. As GLCs podem ser ambíguas, e resolver ambiguidades muitas vezes exige reformular a gramática ou usar anotações de precedência externas (como os mecanismos `%left/%right` do yacc/bison).

Recursão à esquerda. GLCs admitem recursão à esquerda naturalmente; por exemplo, $E \rightarrow E + T$ é uma produção válida e útil. Parsers PEG descendentes não suportam recursão à esquerda direta, ela causaria um laço infinito. A solução é reescrever usando repetição $(\cdot)^*$ ou usar técnicas especializadas de eliminação de recursão à esquerda.

Poder expressivo. PEGs reconhecem linguagens que as GLCs não conseguem (por exemplo, $\{a^n b^n c^n \mid n \geq 0\}$, usando predicados) e há conjectura de que algumas linguagens livres de contexto não podem ser descritas por PEGs, embora isso não esteja formalmente provado.

Mensagens de erro. A semântica de PEGs facilita a geração de mensagens de erro precisas: quando o parser se compromete com uma alternativa e depois falha, sabe exatamente onde e o que era esperado.

7.6 Conclusão

Este capítulo apresentou as Parsing Expression Grammars como um formalismo para especificar analisadores sintáticos determinísticos. As PEGs se distinguem das GLCs pela **escolha ordenada** /, que elimina a ambiguidade e corresponde diretamente ao comportamento de parsers.

Apresentamos a implementação em Haskell da biblioteca `PEG.Semantics.Simple`, cujo tipo `Result` da representa explicitamente os três estados possíveis de um parser: sucesso sem consumo (`Pure`), sucesso com consumo (`Commit`) e falha (`Fail`).

Os combinadores `<|>` (escolha sem backtracking) e `</>` (escolha com backtracking via `try`) dão ao programador controle explícito sobre quando o parser pode recuar, tornando as PEGs uma ferramenta poderosa tanto para especificação formal quanto para implementação prática de analisadores sintáticos.

Capítulo 8

Análise sintática LL(1)

8.1 Introdução

Nos capítulos anteriores, apresentamos duas técnicas para análise sintática descendente: combinadores de parsing e Parsing Expression Grammars. Ambas constroem uma derivação iniciando pelo símbolo inicial da gramática e, a cada passo, escolhem uma regra para expandir o não-terminal mais à esquerda.

Neste capítulo, estudamos outro algoritmo descendente: o **analisador LL(1)**. O nome tem três componentes:

- O primeiro **L** indica que a entrada é lida da **esquerda para a direita** (*left-to-right*).
- O segundo **L** indica que o analisador produz uma **derivação mais à esquerda** (*leftmost derivation*).
- O **1** indica que o analisador usa apenas **um símbolo de lookahead** para decidir qual produção aplicar.

A ideia central é substituir a recursão por um **algoritmo dirigido por tabela**: uma tabela de parsing $M[A, a]$ indica, para cada não-terminal A e cada terminal a (incluindo o marcador de fim de entrada $\$$), qual produção deve ser aplicada, ou sinaliza um erro sintático.

O código discutido neste capítulo está em

Parsing/LL/BuildTable.hs e
Parsing/LL/Parser.hs

8.2 Exemplo Informal

Considere a gramática de expressões aritméticas a seguir (sem recursão à esquerda, fatorada à esquerda):

$$\begin{aligned} E &\rightarrow T E' \\ E' &\rightarrow + T E' \mid \lambda \\ T &\rightarrow F T' \\ T' &\rightarrow * F T' \mid \lambda \\ F &\rightarrow (E) \mid \mathbf{n} \end{aligned}$$

e a entrada $\mathbf{n} + \mathbf{n}$. O analisador LL(1) mantém uma **pilha** de símbolos que representa o que ainda falta reconhecer. Inicialmente a pilha contém o símbolo inicial seguido de $\$$. A tabela a seguir mostra os passos para reconhecer a string $\mathbf{n} + \mathbf{n}$:

Em cada passo, o analisador observa o topo da pilha e o símbolo corrente da entrada:

- Se o topo é um **terminal** igual ao símbolo corrente, ele é consumido da entrada e removido da pilha (casamento).

Pilha	Entrada	Ação
$E \$$	$\mathbf{n} + \mathbf{n} \$$	Consultar $M[E, \mathbf{n}]$: aplicar $E \rightarrow T E'$
$T E' \$$	$\mathbf{n} + \mathbf{n} \$$	Consultar $M[T, \mathbf{n}]$: aplicar $T \rightarrow F T'$
$F T' E' \$$	$\mathbf{n} + \mathbf{n} \$$	Consultar $M[F, \mathbf{n}]$: aplicar $F \rightarrow \mathbf{n}$
$\mathbf{n} T' E' \$$	$\mathbf{n} + \mathbf{n} \$$	Casar: consumir $\mathbf{n}$
$T' E' \$$	$+ \mathbf{n} \$$	Consultar $M[T', +]$: aplicar $T' \rightarrow \lambda$
$E' \$$	$+ \mathbf{n} \$$	Consultar $M[E', +]$: aplicar $E' \rightarrow + T E'$
$+ T E' \$$	$+ \mathbf{n} \$$	Casar: consumir $+$
$T E' \$$	$\mathbf{n} \$$	Consultar $M[T, \mathbf{n}]$: aplicar $T \rightarrow F T'$
$F T' E' \$$	$\mathbf{n} \$$	Consultar $M[F, \mathbf{n}]$: aplicar $F \rightarrow \mathbf{n}$
$\mathbf{n} T' E' \$$	$\mathbf{n} \$$	Casar: consumir $\mathbf{n}$
$T' E' \$$	$\$$	Consultar $M[T', \$]$: aplicar $T' \rightarrow \lambda$
$E' \$$	$\$$	Consultar $M[E', \$]$: aplicar $E' \rightarrow \lambda$
$\$$	$\$$	Aceitar

Tabela 8.1: Simulação do analisador preditivo para $\mathbf{n} + \mathbf{n}$

- Se o topo é um **não-terminal** A , consulta-se $M[A, a]$ para obter a produção a aplicar. O não-terminal é removido e os símbolos do corpo da produção são empilhados em ordem (o primeiro símbolo da produção fica no topo).
- Se o topo e a entrada são ambos $\$$, a entrada foi aceita.
- Qualquer outra situação é um erro sintático.

Note que o problema central em um analisador LL(1) consiste em construir a tabela. Esse será o assunto das próximas seções.

8.3 Construção da tabela LL(1)

Para construir a tabela de parsing, precisamos de três conceitos, definidos a seguir.

8.3.1 Anulável

Um não-terminal A é **anulável** se pode derivar a cadeia vazia em um número qualquer de passos:

$$A \Rightarrow^* \lambda$$

Uma sequência de símbolos $\alpha = X_1 X_2 \cdots X_n$ é anulável se **todos** os X_i são anuláveis. A sequência vazia é trivialmente anulável.

8.3.2 Conjunto FIRST

O conjunto $\text{FIRST}(\alpha)$ de uma sequência de símbolos α é o conjunto de terminais que podem iniciar uma cadeia derivada de α :

$$\text{FIRST}(\alpha) = \{ a \in \Sigma \mid \alpha \Rightarrow^* a \beta \}$$

Adicionalmente, incluímos λ em $\text{FIRST}(\alpha)$ se α é anulável.

As regras de cálculo para uma sequência $X_1 X_2 \cdots X_n$ são:

1. Se a sequência é vazia: $\text{FIRST}(\varepsilon) = \{\lambda\}$.

2. Se X_1 é um terminal a : $\text{FIRST}(X_1 \cdots X_n) = \{a\}$.
3. Se X_1 é um não-terminal:
 - Incluir $\text{FIRST}(X_1) - \{\lambda\}$.
 - Se $\lambda \in \text{FIRST}(X_1)$, incluir também $\text{FIRST}(X_2 \cdots X_n)$.

Os conjuntos FIRST para cada não-terminal são calculados iterativamente até atingir um ponto fixo, isto é, até não haver modificações no conjunto.

8.3.3 Conjunto FOLLOW

O conjunto $\text{FOLLOW}(A)$ de um não-terminal A é o conjunto de terminais (e o marcador $\$$) que podem aparecer imediatamente à direita de A em alguma forma sentencial:

$$\text{FOLLOW}(A) = \{a \in \Sigma \cup \{\$\} \mid S \Rightarrow^* \alpha A a \beta\}$$

As regras de cálculo são:

1. Se A é o símbolo inicial: $\$ \in \text{FOLLOW}(A)$.
2. Para cada regra $B \rightarrow \alpha A \beta$:
 - Adicionar $\text{FIRST}(\beta) - \{\lambda\}$ a $\text{FOLLOW}(A)$.
 - Se $\lambda \in \text{FIRST}(\beta)$, adicionar $\text{FOLLOW}(B)$ a $\text{FOLLOW}(A)$.

Os conjuntos FOLLOW também são calculados iterativamente até ponto fixo.

8.3.4 Algoritmo para construção da tabela de parsing

A tabela M é uma matriz indexada por pares (A, a) , onde A é um não-terminal e a é um terminal ou $\$$. Cada entrada contém uma produção ou indica erro. O algoritmo de construção é:

Algorithm 2 Construção da tabela de análise preditiva M

```

1: for all produção  $A \rightarrow \alpha$  da gramática do
2:   for all terminal  $a \in \text{FIRST}(\alpha) - \{\lambda\}$  do
3:      $M[A, a] \leftarrow A \rightarrow \alpha$ 
4:   if  $\lambda \in \text{FIRST}(\alpha)$  then
5:     for all  $b \in \text{FOLLOW}(A)$  do
6:        $M[A, b] \leftarrow A \rightarrow \alpha$ 
7: Todas as entradas não preenchidas indicam erro

```

A gramática é **LL(1)** se e somente se nenhuma entrada da tabela receber mais de uma produção. Quando isso ocorre, diz-se que há um **conflito** na tabela.

8.4 Algoritmo de Análise Sintática LL(1)

Com a tabela construída, o algoritmo de parsing é:

A sequência de produções aplicadas corresponde à **derivação mais à esquerda** da entrada.

Algorithm 3 Analisador sintático preditivo dirigido por tabela**Require:** tabela M , símbolo inicial S , sequência de tokens $w\$$

```

1: pilha  $\leftarrow [S, \$]$ 
2: loop
3:   Seja  $X$  o topo da pilha e  $a$  o símbolo corrente da entrada
4:   if  $X = \$$  e  $a = \$$  then
5:     aceitar e parar
6:   else if  $X$  é um terminal then
7:     if  $X = a$  then
8:       Desempilhar  $X$  e avançar na entrada
9:     else
10:      erro
11:   else if  $X$  é um não-terminal then
12:     if  $M[X, a] = X \rightarrow \alpha$  then
13:       Substituir  $X$  na pilha pelos símbolos de  $\alpha$ 
          (empilhados da direita para a esquerda)
14:     else
15:      erro

```

8.5 Exemplo de Construção de Tabela

8.5.1 Gramática LL(1): Expressões Aritméticas

Retomamos a gramática de expressões apresentada no exemplo informal:

$$\begin{aligned}
 E &\rightarrow T E' \\
 E' &\rightarrow + T E' \mid \lambda \\
 T &\rightarrow F T' \\
 T' &\rightarrow * F T' \mid \lambda \\
 F &\rightarrow (E) \mid \mathbf{n}
 \end{aligned}$$

Conjuntos FIRST:

Não-terminal	FIRST
E	$\{ (, \mathbf{n} \}$
E'	$\{ +, \lambda \}$
T	$\{ (, \mathbf{n} \}$
T'	$\{ *, \lambda \}$
F	$\{ (, \mathbf{n} \}$

Tabela 8.2: Conjuntos FIRST

Conjuntos FOLLOW:

Não-terminal	FOLLOW
E	$\{ \$,) \}$
E'	$\{ \$,) \}$
T	$\{ +, \$,) \}$
T'	$\{ +, \$,) \}$
F	$\{ *, +, \$,) \}$

Tabela 8.3: Conjuntos FOLLOW

Tabela de parsing LL(1):

	n	+	*	(	)	\$
E	$T E'$	$\emptyset$	$\emptyset$	$T E'$	$\emptyset$	$\emptyset$
E'	$\emptyset$	$+ T E'$	$\emptyset$	$\emptyset$	λ	λ
T	$F T'$	$\emptyset$	$\emptyset$	$F T'$	$\emptyset$	$\emptyset$
T'	$\emptyset$	λ	$* F T'$	$\emptyset$	λ	λ
F	n	$\emptyset$	$\emptyset$	(E)	$\emptyset$	$\emptyset$

Tabela 8.4: Tabela de análise preditiva M

Cada entrada tem **no máximo uma produção**, confirmando que a gramática é LL(1). A seguir, apresentamos alguns exemplos de gramáticas que não são LL(1).

8.5.2 Gramática não-LL(1): Conflito FIRST/FIRST

Considere a gramática para comandos condicionais com o problema do *dangling else*:

$$\begin{array}{lcl}
 S & \rightarrow & \text{if } E \text{ then } S \text{ else } S \\
 & | & \text{if } E \text{ then } S \\
 & | & \text{other}
 \end{array}$$

Para o não-terminal S com lookahead **if**:

- $\text{FIRST}(\text{if } E \text{ then } S \text{ else } S) = \{\text{if}\}$
- $\text{FIRST}(\text{if } E \text{ then } S) = \{\text{if}\}$

Ambas as produções contribuem para $M[S, \text{if}]$, gerando um **conflito FIRST/FIRST**. A gramática não é LL(1) porque, com apenas um símbolo de lookahead, não é possível distinguir dentre as duas alternativas.

8.5.3 Gramática não-LL(1): Conflito FIRST/FOLLOW

Outro tipo comum de conflito ocorre quando uma produção pode derivar λ e os conjuntos FIRST e FOLLOW do não-terminal não são disjuntos. Considere:

$$\begin{array}{lcl}
 S & \rightarrow & A \mathbf{a} \\
 A & \rightarrow & \mathbf{a} \mid \lambda
 \end{array}$$

Aqui $\text{FIRST}(A) = \{\mathbf{a}, \lambda\}$ e $\text{FOLLOW}(A) = \{\mathbf{a}\}$. Para a entrada $M[A, \mathbf{a}]$ devemos registrar tanto $A \rightarrow \mathbf{a}$ (pois $\mathbf{a} \in \text{FIRST}(A)$) quanto $A \rightarrow \lambda$ (pois $\mathbf{a} \in \text{FOLLOW}(A)$). Há um **conflito FIRST/FOLLOW** e, portanto, a gramática não é LL(1).

A seguir, vamos discutir a implementação deste algoritmo em Haskell.

8.6 A Implementação em Haskell

A implementação está dividida em três módulos principais:

```

Parsing.Grammar,
Parsing.LL.BuildTable e
Parsing.LL.Parser.

```

8.6.1 Representação da Gramática

Uma gramática é representada como um mapa de não-terminais para listas de produções:

```
data Symbol = Terminal String | NonTerminal String
type Production = [Symbol]
type Grammar = Map String [Production]
```

O terminal especial λ representa a produção vazia. Funções auxiliares como `nonTerminals`, `terminals`, `isLambda` e `symbolString` facilitam a manipulação das estruturas.

8.6.2 Conjuntos FIRST (Parsing.First)

Os conjuntos FIRST são computados por iteração até ponto fixo, usando a função `fixedPoint` do módulo `Utils.Fixpoint`:

```
type FirstSet = Map String [String]

computeFirstSets :: Grammar -> FirstSet
computeFirstSets g = fixedPoint step initial
  where
    initial = Map.fromList [(nt, []) | nt <- nonTerminals g]
    step cur = Map.mapWithKey update cur
      where
        update nt _ = unions $ map (firstForSequence g cur)
          (fromMaybe [] (Map.lookup nt g))
```

A função `firstForSequence` implementa diretamente as regras da definição:

```
firstForSequence :: Grammar -> FirstSet -> [Symbol] -> [String]
firstForSequence _ _ [] = ["λ"]
firstForSequence g fs (sym:syms)
  | isTerminal sym =
    if isLambda sym
    then firstForSequence g fs syms
    else [symbolString sym]
  | otherwise =
    let nt = symbolString sym
        firstOfSym = fromMaybe [] (Map.lookup nt fs)
        withoutLam = firstOfSym \\ ["λ"]
    in if "λ" `elem` firstOfSym
    then withoutLam `union` firstForSequence g fs syms
    else withoutLam
```

O caso base retorna `["λ"]` para a sequência vazia. Para um terminal, retorna o próprio terminal (ou continua com o resto da sequência se for λ). Para um não-terminal, inclui seu FIRST sem λ e, se ele é anulável, continua com o restante da sequência.

8.6.3 Conjuntos FOLLOW (Parsing.Follow)

Os conjuntos FOLLOW também são calculados por ponto fixo. O símbolo inicial começa com `"$"` em seu conjunto FOLLOW, e os demais começam vazios:

```

computeFollowSets :: Grammar -> FirstSet -> FollowSet
computeFollowSets g fs = fixedPoint step initial
  where
    startSymbol = head (nonTerminals g)
    initial = Map.fromList
      [ (nt, if nt == startSymbol then ["$"] else [])
      | nt <- nonTerminals g ]
    step cur = foldl update cur allProductions

```

Para cada ocorrência de um não-terminal B numa produção $A \rightarrow \alpha B \beta$, o código calcula $\text{FIRST}(\beta)$ e adiciona à $\text{FOLLOW}(B)$; se β é anulável, adiciona também $\text{FOLLOW}(A)$:

```

updatePos lhs' follow' pos
| NonTerminal b <- prod !! pos =
  let beta      = drop (pos + 1) prod
      firstBeta = firstForSequence g fs beta
      followB   = fromMaybe [] (Map.lookup b follow')
      newFollow = followB
                  `union` (firstBeta \> ["λ"])
                  `union` (if "λ" `elem` firstBeta
                          then fromMaybe [] (Map.lookup lhs' cur)
                          else [])
  in Map.insert b newFollow follow'
| otherwise = follow'

```

8.6.4 Construção da Tabela (Parsing.LL.BuildTable)

A tabela é um mapa de pares $(\text{String}, \text{String})$ para ParseAction :

```

type ParseTable = Map (String, String) ParseAction

data ParseAction
= Derive Production
| Accept
| Error

```

A tabela é inicializada com **Error** em todas as entradas (nt, t) para cada par (não-terminal, terminal). Em seguida, cada produção preenche as entradas correspondentes:

```

buildParseTable :: Grammar -> ParseTable
buildParseTable g =
  foldl (addProduction g firstSets followSets) (initialTable g) (allProductions g)
  where
    firstSets = computeFirstSets g
    followSets = computeFollowSets g firstSets

```

A função **addProduction** determina os terminais para os quais a produção deve ser registrada e detecta conflitos:

```

addProduction :: Grammar -> FirstSet -> FollowSet
               -> ParseTable -> (String, Production) -> ParseTable
addProduction g firstSets followSets table (lhs, prod) =
  foldl insertEntry table entries
  where
    firstAlpha = firstForSequence g firstSets prod

```

```

followLhs = fromMaybe [] (Map.lookup lhs followSets)

entries :: [String]
entries = (firstAlpha \\ ["λ"])
  ++ if "λ" `elem` firstAlpha then followLhs else []

insertEntry tbl a =
  case Map.lookup (lhs, a) tbl of
    Just Error -> Map.insert (lhs, a) (Derive prod) tbl
    Just _      -> error $ "Grammar is not LL(1): conflict at ("
      ++ lhs ++ ", " ++ a ++ ")"
    Nothing     -> tbl

```

A lista `entries` reúne exatamente os terminais do algoritmo de construção: $\text{FIRST}(\alpha) \setminus \{\lambda\}$ mais, se α é anulável, $\text{FOLLOW}(A)$. Se uma entrada já tem uma ação diferente de `Error`, a gramática não é LL(1) e uma exceção é lançada imediatamente.

8.6.5 O Parser (Parsing.LL.Parser)

O analisador é uma função pura que recebe a gramática, a tabela e a sequência de tokens, e retorna as produções aplicadas ou uma mensagem de erro:

```

data ParseResult
  = ParseOk [Production]
  | ParseError String

parse :: Grammar -> ParseTable -> [String] -> ParseResult
parse grammar table tokens =
  go initialStack (tokens ++ ["$"]) []
  where
    startSym      = head (nonTerminals grammar)
    initialStack = [NonTerminal startSym, Terminal "$"]

go (Terminal "$" : _) (" $" : _) acc =
  ParseOk (reverse acc)

go (Terminal t : stack') (a : inp') acc
  | t == a      = go stack' inp' acc
  | otherwise   = ParseError $
    "Syntax error: expected '" ++ t ++ "', found '" ++ a ++ "'"

go (NonTerminal nt : stack') inp@(a : _) acc =
  case Map.lookup (nt, a) table of
    Just (Derive prod) ->
      let pushed = filter (not . isLambda) prod
      in go (pushed ++ stack') inp (prod : acc)
    Just Accept ->
      ParseOk (reverse acc)
    _ ->
      ParseError $
        "Syntax error: no rule for (" ++ nt ++ ", " ++ a ++ ")"

go stack' inp' _ =
  ParseError "Syntax error: unexpected state"

```

Quando uma produção λ é aplicada, os símbolos λ são filtrados antes de ser empilhados (`filter (not . isLambda)` pois λ representa a cadeia vazia e não deve ocupar espaço na pilha. O acumulador `acc` coleta as produções em ordem reversa; ao aceitar, `reverse acc` devolve a derivação mais à esquerda na ordem correta.

8.7 Conclusão

Neste capítulo, estudamos a análise sintática LL(1) do ponto de vista teórico e prático.

Do lado teórico, definimos os conceitos de **anulável**, **FIRST** e **FOLLOW** e vimos como eles se combinam para construir a tabela de parsing preditivo. A tabela encapsula toda a decisão do analisador: dado um não-terminal e um símbolo de lookahead, ela indica exatamente qual produção aplicar, ou que a entrada é inválida.

Do lado prático, a implementação em Haskell mostrou que o algoritmo se traduz diretamente em código. Os conjuntos FIRST e FOLLOW são calculados por iteração até ponto fixo, a tabela é preenchida com um único percurso sobre as produções da gramática, e o parser em si é uma função recursiva com três casos simples.

Uma limitação importante da abordagem LL(1) é que nem toda gramática livre de contexto pode ser analisada desta forma: gramáticas com **recursão à esquerda** ou que não sejam **fatoradas à esquerda** geram conflitos na tabela. Nesses casos, é necessário transformar a gramática ou usar um algoritmo mais poderoso, como os analisadores **LR** que estudaremos nos próximos capítulos.

Capítulo 9

Introdução à análise ascendente

9.1 Introdução

Até agora, estudamos técnicas **top-down** para a análise sintática. Um analisador sintático top-down é limitado nas gramáticas que pode processar, pois precisa ser capaz de decidir qual regra deve ser usada com base em relativamente pouca informação. Por exemplo, ele precisa ser capaz de escolher a produção correta a partir do símbolo inicial com base no primeiro token da entrada.

Essa limitação motiva a análise sintática **bottom-up**, na qual o analisador sintático pode escolher produções após analisar um prefixo maior da entrada. Por exemplo, os analisadores sintáticos de bottom-up podem lidar com produções recursivas à esquerda, enquanto os analisadores sintáticos top-down não. De fato, os analisadores sintáticos bottom-up funcionam melhor quando as produções são recursivas à esquerda, embora também possam lidar com produções recursivas à direita.

Neste capítulo, vamos apresentar dois algoritmos bottom-up: o projetado por Earley e os algoritmos LR(0) e SLR.

9.2 Algoritmo de Earley

Um analisador sintático bottom-up expressivo e simples é o desenvolvido por Jay Earley. Ele consegue analisar todas as gramáticas livres de contexto, incluindo as ambíguas. Seu tempo de execução no pior caso é cúbico em relação ao comprimento da entrada, mas para a maioria das gramáticas encontradas na prática, leva tempo linear se implementado com cuidado.

A ideia do analisador Earley é manter o controle de todas as análises possíveis à medida que a entrada é lida. Podemos pensar no analisador como aquele que, diferentemente de um analisador LL, não tenta prever apenas uma produção; em vez disso, ele prevê cegamente todas as possíveis produções, independentemente da decisão anterior, criando uma nova thread concorrente para lidar com cada possível decisão. Um problema dessa abordagem baseada em threads é o número exponencial de threads criadas. Além disso, uma enorme quantidade de passos seria repetido por diferentes threads. Portanto, o analisador Earley simula o trabalho que essas threads fariam, garantindo que o trabalho seja feito apenas uma vez.

O analisador Earley processa um token da entrada por vez, mantendo o controle de todas as informações necessárias para simular as threads de análise. Para cada posição de entrada j , um analisador Earley constrói um conjunto de itens I_j representando o estado das produções que podem ser usadas na derivação. Um item Earley tem a forma $[A \rightarrow \beta.\gamma, k]$. A primeira parte do item é uma produção $A \rightarrow \beta\gamma$ da gramática, com um ponto localizado em algum lugar à direita da produção para indicar quanto dessa regra de produção foi concluído. Para uma produção com o lado direito vazio, como $X \rightarrow \lambda$, a primeira parte do item contém apenas o ponto: $[X \rightarrow ., k]$. A segunda parte do item, k , é a posição na entrada onde a análise do não-terminal X começou, em termos do número de tokens lidos até aquele ponto. Podemos pensar em k como o registro da posição onde a thread para este item foi iniciada.

O algoritmo começa inicializando o conjunto I_0 como $[S' \rightarrow .S, 0]$, onde S é o símbolo inicial da gramática e S' é um novo símbolo de partida introduzido para simplificar o algoritmo. O algoritmo analisa com sucesso a sequência de n tokens de entrada $a_0a_1\dots a_{n-1}$ se o item $[S' \rightarrow .S, 0]$ estiver no

conjunto I_n .

O algoritmo considera cada posição de entrada j de 0 até n . Em cada posição de entrada, ele constrói o conjunto I_j aplicando as três regras a seguir quantas vezes forem necessárias até que nenhuma delas tenha efeito:

- **Verificar.** Se o conjunto I_j contém um item $[A \rightarrow \beta.c\delta, k]$ onde c é igual ao próximo símbolo de entrada a_j , então o item $[A \rightarrow \beta.c.\delta, k]$ é adicionado a I_{j+1} . Todas as etapas de verificação podem ser realizadas como as ações finais em uma determinada posição de entrada, já que não afetam I_j .
- **Predição.** Se o conjunto I_j contém um item $[A \rightarrow \beta.C\delta, k]$, então, para todas as produções $C \rightarrow \gamma$ na gramática, um item $[C \rightarrow .\gamma, j]$ é adicionado a I_j . Observe que as etapas de predição podem habilitar etapas de predição adicionais, quando γ começa com um não terminal.
- **Completar.** Se o conjunto I_j contém um item completo $[C \rightarrow \gamma., k]$, então para cada item da forma $[A \rightarrow \beta.C\delta, l]$ no conjunto I_k , um item $[A \rightarrow \beta.C.\delta, l]$ é adicionado a I_j .

Essas são as mesmas três ações que vimos no analisador LL baseado em tabelas, mas, diferentemente daquele analisador, o analisador Earley não precisa se comprometer com suas decisões. Em vez disso, ele lida com múltiplas decisões, e até mesmo conclusões, em paralelo. As previsões que não se confirmam são efetivamente descartadas, pois em algum momento levam a itens incompatíveis com os tokens de entrada encontrados.

9.2.1 Exemplo

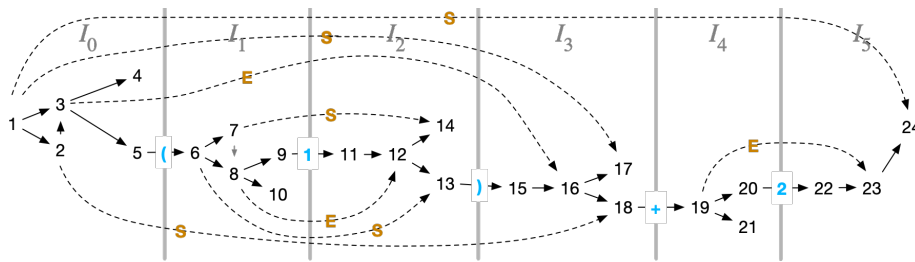
Considere a seguinte gramática, que não é LL(1):

$$\begin{array}{lcl} S & \rightarrow & S + E \mid E \\ E & \rightarrow & (E) \mid n \end{array}$$

Dado a entrada $(1) + 2$, o analisador Earley procede da seguinte forma:

Conjunto	Item	Pos	Motivo	Etapa #
I0	$S' \rightarrow . S$	0	item inicial	1
	$S \rightarrow . S + E$	0	prever S	2 (a partir de 1)
	$S \rightarrow . E$	0	prever S	3 (a partir de 1 e também de 2)
	$E \rightarrow . n$	0	prever E	4 (a partir de 3)
	$E \rightarrow . (S)$	0	prever E	5 (de 3)
I1	$E \rightarrow (. S)$	0	examinar (	6 (de 5)
	$S \rightarrow . S + E$	1	prever S	7 (de 6)
	$S \rightarrow . E$	1	prever S	8 (de 6 e também de 7)
	$E \rightarrow . n$	1	prever E	9 (de 8)
	$E \rightarrow . (S)$	1	prever E	10 (de 8)
I2	$E \rightarrow n .$	1	consumir 1	11 (de 9)
	$S \rightarrow E .$	1	completar E	12 (de 11, 8)
	$E \rightarrow (S .)$	0	completar S	13 (de 12, 6)
	$S \rightarrow S . + E$	1	completar S	14 (de 12, 7)
I3	$E \rightarrow (S) .$	0	consumir)	15 (de 13)
	$S \rightarrow E .$	0	completo E	16 (de 15, 3)
	$S' \rightarrow S .$	0	completo S	17 (de 16, 1)
	$S \rightarrow S . + E$	0	completo S	18 (de 16, 2)
I4	$S \rightarrow S + . E$	0	consumir +	19 (de 18)
	$E \rightarrow . n$	4	prever E	20 (de 19)
	$E \rightarrow . (S)$	4	prever E	21 (de 19)
I5	$E \rightarrow n .$	4	consumir 2	22 (de 20)
	$S \rightarrow S + E .$	0	completo E	23 (de 22, 19)
	$S' \rightarrow S .$	0	completo S: sucesso!	24 (de 23, 1)
	$S \rightarrow S . + E$	0	completo S	25 (de 23, 2)

Podemos visualizar a ação do analisador sintático desenhando um gráfico que mostre como as várias etapas dependem umas das outras:



Aqui, as ações de consumir tokens são anotadas com o token escaneado em azul. As setas tracejadas mostram o estado e são rotuladas com o não terminal que está sendo completado. Como este gráfico mostra, o analisador explora alguns estados que não levam à análise bem-sucedida, mas esses estados (4, 10, 14, 17 e 21) rapidamente ficam **presos** porque não conseguem encontrar uma entrada correspondente. Removendo esses estados e observando apenas a sequência de produções completadas que levam ao sucesso na etapa 24, vemos que elas descrevem uma derivação da string:

$$\begin{array}{ccccccc}
 S' & \rightarrow & S & \rightarrow & S + E & \rightarrow & S + 2 & \rightarrow \\
 24 & & 23 & & 18-22 & & 16 & \\
 \\
 E + 2 & \rightarrow & (S) + 2 & \rightarrow & (E) + 2 & \rightarrow & (1) + 2 \\
 13-15 & & 12 & & 1-11 & &
 \end{array}$$

Note que, a cada passo, o não terminal mais à direita é expandido usando alguma produção. No entanto, o analisador sintático completa essas produções de trás para frente, gerando a derivação mais à direita invertida, começando pela entrada e terminando com o símbolo inicial. Essa construção reversa de uma derivação mais à direita é o que os analisadores sintáticos bottom-up fazem.

Os analisadores de Earley têm complexidade de tempo $O(n^3)$ no pior caso quando implementados cuidadosamente; em gramáticas não ambíguas, a complexidade de tempo é $O(n^2)$ no pior caso. Intuitivamente, a razão pela qual os analisadores de Earley são polinomiais em vez de exponenciais no pior caso é que eles compartilham o trabalho entre as diferentes “linhas de processamento”. Por exemplo, observe que a conclusão única da produção $S \rightarrow E$, na etapa 12 realiza trabalho para duas linhas de processamento que se ramificam na etapa 6 para os estados 7 e 8. A linha de processamento “7” acaba ficando presa porque não encontra o sinal “+” esperado na entrada, mas a outra linha de processamento continua até o final.

Em gramáticas LL ou LR, os analisadores Earley são ainda mais úteis, executando em tempo linear quando implementados cuidadosamente. Nossa gramática de exemplo é uma gramática LR não ambígua, razão pela qual o grafo de estados acima é predominantemente linear e pouco trabalho é desperdiçado.

Uma otimização simples e útil é completar as produções somente quando o token de lookahead estiver no conjunto FOLLOW do não terminal em questão. Também pode ser útil memorizar etapas de predição em cascata.

Uma melhoria adicional que evita computação redundante é sugerida por Ayrcock e Horspool [Practical Earley Parsing](#). Na etapa de predição, quando o símbolo C é anulável, o item $[A \rightarrow \beta C \gamma, k]$ é imediatamente adicionado ao estado I_j . Eles relatam que, com essa e outras otimizações, a análise Earley é apenas 50% mais lenta que o analisador LALR Happy, uma compensação razoável dada a melhoria no poder de análise.

9.3 Analisadores LR(0) e SLR

A análise LR, incluindo a análise LALR, é um método popular de análise sintática. Esses algoritmos são a tecnologia subjacente em diversos geradores de analisadores sintáticos, como o Happy. Podemos considerar a análise LR como uma otimização da análise Earley, na qual todas as previsões são pré-computadas. A análise LR consiste em duas ações: shift e reduce, portanto, os analisadores LR são chamados de analisadores de shift-reduce. A ação de shift corresponde à ação de consumo do algoritmo de Earley, seguida pelo máximo de previsões possível; a ação de reduce corresponde à ação completa, novamente seguida por previsões.

Em vez de manter o controle de todas as possíveis análises sintáticas mais à direita, um analisador sintático de shift-reduce mantém o controle apenas de um conjunto de análises sintáticas que correspondem a uma única análise sintática mais à direita. Essa limitação permite que o estado do analisador seja mais simples do que em um analisador de Earley: o estado do analisador é baseado em uma pilha de símbolos. Em qualquer ponto durante uma análise sintática de shift-reduce, o estado atual da derivação é a concatenação da pilha com a entrada não consumida. Por exemplo, podemos escrever a análise sintática da expressão de exemplo $(1)+2$ como uma série de operações de shift e reduce que constroem exatamente a mesma derivação mais à direita que o analisador de Earley fez acima:

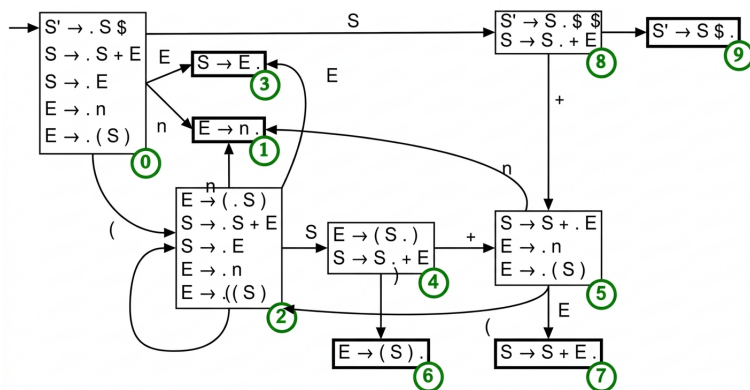
Pilha de Derivação	Ações	Entrada não consumida
(1) + 2	(1) + 2	
(1) + 2	shift	(1) + 2
(1) + 2	shift	(1) + 2
(E) + 2	reduce $E \rightarrow n$	(E) + 2
(S) + 2	reduce $S \rightarrow E$	(S) + 2
(S) + 2	shift	(S) + 2
E + 2	reduce $E \rightarrow (S)$	E + 2
S + 2	reduce $S \rightarrow E$	S + 2
S + 2	shift	S + 2
S + 2	shift	S + 2
S + E	reduce $E \rightarrow n$	S + E
S	reduce $S \rightarrow S+E$	S

Para determinar qual ação deve ser tomada em cada etapa, o analisador sintático precisa saber quais são as possíveis produções futuras a serem reduzidas. As possíveis produções futuras são um conjunto de itens, fechado sob predição; esse conjunto de itens nos diz qual ação faz sentido em um determinado ponto da análise sintática. Para mapear uma pilha em um conjunto de itens, esses conjuntos de itens podem ser interpretados como um autômato finito determinístico que lê a pilha e cujos estados são exatamente os conjuntos de itens. Esses estados são semelhantes aos conjuntos de Earley, mas têm a propriedade de que, ignorando a predição, apenas uma ação significativa pode ser tomada em cada estado: shift ou reduce.

O estado inicial do autômato LR(0) corresponde ao conjunto de Earley I_0 , exceto que adicionamos o símbolo de fim de entrada à produção de nível superior. Assim como o conjunto de Earley, ele é fechado sob predição.

$$\begin{aligned}
 S' &\rightarrow .S\$ \\
 S &\rightarrow .S + E \\
 S &\rightarrow .E \\
 E &\rightarrow .n \\
 E &\rightarrow .(S)
 \end{aligned}$$

O autômato é construído repetindo-se o seguinte procedimento até que nenhuma alteração adicional seja possível. Escolha um estado e um símbolo que esteja à direita do ponto em uma ou mais produções. Desloque esse símbolo para a direita nesses itens e feche o conjunto de itens sob previsão. O conjunto de itens resultante pode ser um estado existente do autômato ou um novo. Em ambos os casos, adicione uma transição no símbolo escolhido do estado antigo para o estado correspondente a este conjunto de itens.



Por exemplo, ao realizar uma transição no token (do estado inicial, deslocamos apenas o último item para a direita, obtendo um estado contendo a produção $E \rightarrow (.S)$, sobre o qual então realizamos

um fechamento de predição que adiciona itens para S e, em seguida, para E . Ao realizar uma transição a partir do mesmo estado no não terminal S , deslocamos os dois primeiros itens para a direita, obtendo os itens $S' \rightarrow S.\$$ e $S \rightarrow S. + E$. O fechamento de predição não adiciona nenhum item a este estado. A figura a seguir mostra os passos que você pode seguir para completar o autômato (pressione o botão para avançar a animação). Na figura, os estados estão numerados em verde e os estados nos quais uma ação de redução deve ser realizada têm uma borda mais espessa.

Podemos resumir este autômato em duas tabelas que podem ser usadas para controlar um analisador sintático. A primeira é a tabela action, que informa ao analisador o que fazer em cada estado, dado o símbolo de previsão: shift e ir para um novo estado, ou reduce por alguma produção. A segunda tabela é a tabela goto, que informa ao analisador para qual estado ir quando ele reduz uma produção. Abaixo estão essas duas tabelas do autômato que acabamos de construir, lado a lado. Entradas vazias nas tabelas correspondem a erros de sintaxe. Entradas numéricas na tabela action correspondem a ações de shift; uma produção na tabela action indica uma ação de reduce.

	n	+	(	)	\$	S	E
0	1		2			8	3
1	$E \rightarrow n$	$E \rightarrow n$	$E \rightarrow n$	$E \rightarrow n$	$E \rightarrow n$		
2	1		2			4	3
3	$S \rightarrow E$	$S \rightarrow E$	$S \rightarrow E$	$S \rightarrow E$	$S \rightarrow E$		
4		5		6			
5	1		2				7
6	$E \rightarrow (S)$	$E \rightarrow (S)$	$E \rightarrow (S)$	$E \rightarrow (S)$	$E \rightarrow (S)$		
7	$S \rightarrow S+E$	$S \rightarrow S+E$	$S \rightarrow S+E$	$S \rightarrow S+E$	$S \rightarrow S+E$		
8		5			9		
9	$S' \rightarrow S.$	$S' \rightarrow S.$	$S' \rightarrow S.$	$S' \rightarrow S.$	$S' \rightarrow S\$$		

Durante sua execução, um analisador sintático shift-reduce utiliza uma tabela (como a apresentada) e uma pilha de estados. O algoritmo inicia com a pilha contendo o estado inicial e a entrada a ser processada. À medida que lê cada token de entrada, ele consulta na tabela action qual é a ação atual para o estado no topo da pilha, dado o próximo token. Se a ação for um shift, ele adiciona o estado especificado à pilha. Se a ação for um reduce, ele remove da pilha o número de itens na produção correspondente e usa a tabela goto do estado agora no topo da pilha para determinar o próximo estado a ser adicionado à pilha.

Vamos testar este analisador sintático no exemplo anterior, $(1)+2$. A pilha contém apenas o estado 0. Para tornar mais claro, os estados na pilha são mostrados em **negrito** e são precedidos pelo símbolo que foi seguido para chegar a esse estado. Esses símbolos correspondem à parte da derivação que foi construída até agora pelo analisador sintático.

Pilha	Entrada	Ação
0	(1)+2	shift 2
0(2	1)+2	shift 1
0(21	) +2	reduce $E \rightarrow n$
0(2E3	) +2	reduce $S \rightarrow E$
0(2S4	) +2	shift 6
0(2S4)6	+2	reduce $E \rightarrow (S)$
0E3	+2	reduce $S \rightarrow E$
0S8	+2	shift 5
0S8+5	2	shift 1
0S8+51	\$	reduce $E \rightarrow n$
0S8+5E7	\$	reduce $S \rightarrow S+E$
0S8	\$	shift 9
0S8\$9		reduce $S' \rightarrow S$

Note que a construção LR que vimos até agora não faz nada além de reduzir a uma única produção fixa, independentemente do próximo token na entrada. Como resultado, os estados que contêm reduções (estados 1, 3, 6, 7, 9 no exemplo) ignoram o token de lookahead. A esse algoritmo damos o nome de analisador LR(0). Preenchendo as tabelas de action e goto de uma maneira menos restritiva, poderíamos obter um analisador LR(1), que é mais expressivo. No entanto, precisaremos modificar a construção da tabela LR(0) que acabamos de ver para obter essa expressividade.

Uma extensão simples, mas surpreendentemente eficaz, para LR(0) é adicionar uma ação de redução à tabela de ações apenas no caso em que o token de lookahead esteja no conjunto FOLLOW do não terminal que está sendo reduzido, já que, caso contrário, a produção claramente não leva a uma análise válida. Quando esse refinamento na construção LR(0) produz um analisador sintático shift-reduce válido, dizemos que a gramática é SLR (Simple LR).

9.4 Implementação

Nesta seção comentamos os principais detalhes das implementações Haskell dos algoritmos apresentados neste capítulo.

9.4.1 Algoritmo de Earley

A implementação está no módulo `Parsing.Earley.EarleyParser`. Os tipos principais refletem diretamente as definições do algoritmo apresentadas acima.

Itens e conjuntos

Um item Earley $[A \rightarrow \beta.\gamma, k]$ é representado pelo tipo:

```
data EarleyItem = EarleyItem
  { itemLhs :: String      -- não-terminal do lado esquerdo
  , itemBefore :: [Symbol] -- símbolos antes do ponto
  , itemAfter  :: [Symbol] -- símbolos depois do ponto
  , itemOrigin :: Int      -- posição de início
  }
```

O campo `itemOrigin` corresponde ao índice k do item $[A \rightarrow \beta.\gamma, k]$: ele registra em qual conjunto Earley este item se originou, o que é usado na etapa de completar. O chart completo é uma lista de conjuntos:

```
type EarleySet = [EarleyItem]
type Chart    = [EarleySet]
```

O chart tem $n + 1$ conjuntos, I_0 a I_n , onde n é o comprimento da entrada.

As três operações

As três operações do algoritmo são funções puras:

```
-- predição
predictItems :: Grammar -> Int -> EarleyItem -> [EarleyItem]
-- verificação
scanItems :: String -> EarleySet -> [EarleyItem]
-- completar
completeItems :: Chart -> EarleySet -> Int -> EarleyItem -> [EarleyItem]
```

A função `completeItems` precisa de acesso ao chart anterior para buscar os itens que esperavam o não-terminal B completado. Quando o item completado tem `itemOrigin = j < k`, a busca é feita em "`chart !! j`" (já fechado); quando $j = k$, busca-se em `currentSet`, o conjunto sendo construído, para lidar com completações em cadeia no mesmo conjunto.

Fechamento de um conjunto

A função `closeSet` aplica predição e completar repetidamente usando um algoritmo de worklist: novos itens gerados vão para o final da fila e são processados em ordem, evitando elementos duplicados:

```
closeSet :: Grammar -> Chart -> Int -> [EarleyItem] -> EarleySet
closeSet grammar prevSets k = go []
  where
    go current [] = current
    go current (item : rest)
      | item `elem` current = go current rest
      | otherwise =
          let current' = current ++ [item]
              new      = predictItems grammar k item
                      ++ completeItems prevSets current' k item
          in go current' (rest ++ new)
```

Esta estrutura garante que cada item seja processado apenas uma vez, assim como o algoritmo descrito na seção anterior assegura que as três regras são aplicadas “quantas vezes forem necessárias até que nenhuma delas tenha efeito”.

Construção do chart e aceitação

A função `buildChart` constrói I_0 fechando os itens iniciais do símbolo de partida, depois aplica `scanItems` para obter o conjunto inicial de cada I_{k+1} e fecha-o:

```
buildChart :: Grammar -> [String] -> Chart
buildChart grammar tokens = go [s0] tokens
  where
    s0 = closeSet grammar [] 0 startItems
    go chart [] = chart
    go chart (tok : toks) =
        let sk1 = closeSet grammar chart (length chart) (scanItems tok (last chart))
        in go (chart ++ [sk1]) toks
```

A entrada é aceita se I_n contém um item completo para o símbolo inicial com origem em 0, exatamente a condição $[S' \rightarrow S., 0] \in I_n$ descrita na introdução do algoritmo:

```
earleyParse :: Grammar -> [String] -> EarleyResult
earleyParse grammar tokens
  | any accept (chart !! n) = EarleyAccept
  | otherwise               = EarleyReject
  where
    n      = length tokens
    chart  = buildChart grammar tokens
    accept item = itemLhs item == start && itemOrigin item == 0
                && null (itemAfter item)
```

9.4.2 LR(0) e SLR

A implementação está distribuída em três módulos:

```
Parsing.LR.LR0.BuildTable,
Parsing.LR.LR0.LR0Parser e
Parsing.LR.SLR.BuildTable.
```

Itens e estados LR(0)

Um item LR(0) $A \rightarrow \alpha.\beta$ é mais simples que um item Earley: não carrega a posição de origem, pois os estados do autômato já capturam essa informação implicitamente:

```
data Item = Item
  { itemLHS :: String      -- não-terminal
  , itemBefore :: [Symbol] -- símbolos antes do ponto
  , itemAfter  :: [Symbol] -- símbolos depois do ponto
  }

type State = Set Item
```

Um estado LR(0) é um conjunto de itens, exatamente como os conjuntos de itens apresentados na seção anterior. O autômato cujos estados são esses conjuntos é o DFA que lê a pilha do analisador.

Fechamento e transições

A função `closure` implementa o fechamento de predição descrito no capítulo: para cada item com um não-terminal C após o ponto, adiciona todos os itens $C \rightarrow \gamma$:

```
closure :: Grammar -> State -> State
closure g = fixedPoint step
  where
    step current = Set.union current $ Set.fromList
      [ Item b [] prod
      | item <- Set.toList current
      , NonTerminal b <- take 1 (itemAfter item)
      , prod <- fromMaybe [] (Map.lookup b g)
      ]
```

A função `goto` computa a transição do autômato ao consumir um símbolo: avança o ponto nos itens que têm esse símbolo imediatamente após ele e fecha o resultado:

```
goto :: Grammar -> State -> Symbol -> State
goto g items sym = closure g $ Set.fromList
  [ Item lhs (before ++ [sym]) rest
  | Item lhs before (s:rest) <- Set.toList items
  , s == sym
  ]
```

A função `buildStates` constrói a coleção canônica de estados LR(0) por busca em largura, partindo do estado inicial que contém o item $S' \rightarrow .S$ da gramática aumentada:

```
buildStates :: Grammar -> [State]
buildStates g = buildFrom [initial] [initial]
  where
    initial = closure g' (Set.singleton $ initialItem g')
    buildFrom seen [] = seen
    buildFrom seen (cur : queue) =
      let newStates = [goto g' cur sym | sym <- nextSymbols cur]
          unseen     = filter (`notElem` seen) newStates
      in buildFrom (seen ++ unseen) (queue ++ unseen)
```


Tabela de parsing LR(0)

A tabela de parsing tem duas partes: a tabela `action`, que mapeia (`estado`, `terminal`) para uma ação, e a tabela `goto`, que mapeia (`estado`, `não-terminal`) para o próximo estado após uma redução:

```
data Action = Shift Int | Reduce String [Symbol] | Accept

data ParsingTable = ParsingTable
{ actionTable :: Map (Int, String) Action
, gotoTable :: Map (Int, String) Int
, states :: [State]
}
```

O preenchimento da tabela segue as regras descritas na seção de LR(0):

- **Shift**: para cada terminal t após o ponto em algum item de um estado i , $\text{action}[i, t] = \text{Shift } j$, onde j é o índice do estado `goto(i, t)`.
- **Reduce (LR(0))**: para cada item completo $A \rightarrow \alpha$. em um estado i , $\text{action}[i, t] = \text{Reduce}(A \rightarrow \alpha)$ para **todos** os terminais t da gramática. Esta é a característica do LR(0): ele ignora o lookahead nas reduções.
- **Accept**: quando o item completo é $S' \rightarrow S$, $\text{action}[i, \$] = \text{Accept}$.

O algoritmo de parsing LR(0)

O algoritmo de parsing mantém uma pilha de estados e processa a entrada símbolo a símbolo. O tipo `ParserState` encapsula o estado completo do analisador:

```
data ParserState = ParserState
{ stateStack :: [Int] -- pilha de estados
, symbolStack :: [Symbol] -- pilha de símbolos (para diagnóstico)
, inputBuffer :: [String] -- entrada restante
}
```

A função `parseStep` executa um único passo: consulta `action[estado_topo, token]` e aplica a ação correspondente:

- **Shift n** : empilha o estado n e avança na entrada.
- **Reduce $A \rightarrow \alpha$** : desempilha α estados e empilha o estado dado por `goto(novo_topo, A)`. Depois chama `parseStep` recursivamente pois a entrada não é consumida em reduções.
- **Accept**: sinaliza sucesso.

```
parseLR0 :: Grammar -> ParsingTable -> [String] -> ParseResult
parseLR0 g table tokens = go (ParserState [0] [] (tokens ++ ["$"]))
```

A diferença do SLR

O módulo `Parsing.LR.SLR.BuildTable` reutiliza toda a infraestrutura de LR(0): `buildStates`, `closure`, `goto`, `parseLR0`. A implementação do SLR modifica apenas a geração das entradas de redução. Enquanto LR(0) adiciona `Reduce` para todos os terminais, SLR restringe às entradas cujo terminal pertence a $\text{FOLLOW}(A)$:

```

-- LR(0): reduz em todos os terminais
reduceA = [ ((i, t), Reduce lhs alpha)
            | item <- Set.toList state, null (itemAfter item)
            , t <- terminals g ++ ["$"] ]

-- SLR: reduz apenas nos terminais em FOLLOW(lhs)
reduceA = [ ((i, t), Reduce lhs alpha)
            | item <- Set.toList state, null (itemAfter item)
            , t <- fromMaybe [] (Map.lookup lhs followSets) ]

```

Esta mudança pontual é suficiente para que a gramática de expressões

$$\begin{array}{lcl}
 E & \rightarrow & E + T \\
 & | & T \\
 T & \rightarrow & T * F \\
 & | & F \\
 F & \rightarrow & (E) \\
 & | & n
 \end{array}$$

deixe de ter conflitos: no estado que contém $E \rightarrow T$. e $T \rightarrow T * F$, LR(0) colocaria Reduce $E \rightarrow T$ em $*$, gerando um conflito com Shift. Com SLR, como $*$ $\notin \text{FOLLOW}(E) = \{+,), \$\}$, a entrada `action[estado, *]` recebe apenas Shift, não resultando em conflito.

9.5 Conclusão

Este capítulo explorou técnicas bottom-up para análise sintática, destacando como estas oferecem maior expressividade ao lidar com recursão à esquerda e prefixos mais longos da entrada. Investigamos o Algoritmo de Earley, uma solução robusta capaz de processar qualquer gramática livre de contexto através da simulação de múltiplas frentes de análise em paralelo, e introduzimos os fundamentos da análise LR(0) e SLR, que otimizam esse processo via autômatos de shift-reduce e pilhas de estados. Embora o SLR melhore a eficiência ao utilizar conjuntos FOLLOW para decidir reduções, ele ainda apresenta limitações em gramáticas mais complexas. Para superar essas restrições, o próximo capítulo abordará os algoritmos LR(1) e LALR.

Capítulo 10

Análise sintática LR(1) e LALR

10.1 Introdução

No capítulo anterior, vimos os algoritmos LR(0) e SLR. O LR(0) constrói um autômato cujos estados são conjuntos de itens da forma $A \rightarrow \beta.\gamma$ e preenche a tabela de ações sem considerar o próximo token de entrada: toda entrada de reduce é preenchida para **todos** os terminais.

Para motivar a necessidade de ir além do LR(0), considere a seguinte gramática de expressões com recursão à **esquerda**:

$$\begin{aligned} E &\rightarrow E + T \mid T \\ T &\rightarrow n \mid (E) \end{aligned}$$

Essa gramática é LR(0): nenhum estado do autômato possui simultaneamente um item completo e um item não completo com o mesmo símbolo à direita do ponto. Por exemplo, o estado obtido após reconhecer E no estado inicial contém:

$$\begin{aligned} S' &\rightarrow E. \\ E &\rightarrow E. + T \end{aligned}$$

O primeiro item é aceitar em \$ e o segundo é um shift em +, portanto não há conflito.

Agora modifiquemos a gramática para usar recursão à **direita**:

$$\begin{aligned} E &\rightarrow T + E \mid T \\ T &\rightarrow n \mid (E) \end{aligned}$$

Com essa mudança, o estado obtido após reconhecer T no estado inicial passa a conter:

$$\begin{aligned} E &\rightarrow T. + E \\ E &\rightarrow T. \end{aligned}$$

O item $E \rightarrow T.$ está completo, então a construção LR(0) insere uma ação de reduce $E \rightarrow T$ para **todos** os terminais, incluindo +. Mas $E \rightarrow T. + E$ exige uma ação de shift em +. Temos, portanto, um **conflito shift-reduce** em + que o LR(0) não consegue resolver.

A raiz do problema é clara: quando o próximo token é +, ainda é possível completar $E \rightarrow T + E$, portanto **não** devemos reduzir. Quando o próximo token é \$ ou), não há como continuar e **devemos** reduzir. Basta um único token de lookahead para tomar a decisão correta.

Isso nos leva ao algoritmo **LR(1)**, que inclui em cada item com um conjunto de lookaheads calculados localmente, não globalmente como o SLR, tornando as ações de reduce precisas o suficiente para resolver esses conflitos. O algoritmo **LALR(1)** é uma variante prática do LR(1) que mantém o mesmo número de estados do LR(0) ao fundir estados com o mesmo núcleo, reunindo seus lookaheads.

10.2 O Algoritmo LR(1)

O algoritmo LR(1) constrói um autômato cujos estados representam o progresso simultâneo de todas as derivações possíveis no contexto atual. A cada estado está associado um conjunto de **itens LR(1)**, cada um descrevendo o quanto de uma produção já foi analisado e qual token pode legitimamente aparecer a seguir caso essa produção seja completada. A partir desses estados, são extraídas as tabelas de ações (shift, reduce, accept) e goto que guiam o analisador.

10.2.1 Itens LR(1)

Um **item LR(1)** tem a forma $[A \rightarrow \beta.\gamma, a]$. A produção $A \rightarrow \beta\gamma$ indica qual regra está sendo reconhecida; o ponto separa a parte β já processada, que está no topo da pilha, da parte γ que ainda está por vir na entrada. O componente a é o **lookahead**: um terminal (ou \$) que pode aparecer imediatamente depois de toda a cadeia derivada de A no contexto em que este item foi gerado. Ao contrário do SLR, que calcula lookaheads globalmente usando conjuntos FOLLOW, o LR(1) os deriva localmente a partir do contexto de cada item.

Quando o item está completo; isto é, quando $\gamma = \lambda$; o lookahead determina exatamente em qual coluna da tabela de ações a redução $A \rightarrow \beta$ deve ser inserida, evitando conflitos com ações de shift que porventura também estejam presentes no mesmo estado.

10.2.2 Fechamento LR(1)

O fechamento de um conjunto de itens LR(1) propaga as previsões de forma análoga ao LR(0), mas agora carregando o lookahead correto para cada nova previsão. Dado um item $[A \rightarrow \alpha.B\beta, a]$ em que o próximo símbolo após o ponto é o não-terminal B , para cada produção $B \rightarrow \gamma$ da gramática adicionamos ao conjunto o item $[B \rightarrow \gamma, b]$, onde b percorre todos os elementos de $\text{FIRST}(\beta a)$. Esse conjunto capta exatamente os terminais que podem aparecer depois de B neste contexto: se β não for anulável, são os primeiros de β ; se β for anulável ou vazio, o próprio lookahead a também é incluído. O processo é repetido até que nenhum item novo seja adicionado.

10.2.3 Função goto LR(1)

A transição entre estados é calculada pela função $\text{goto}(I, X)$, que recebe um estado I e um símbolo X (terminal ou não-terminal) e devolve o estado alcançado após reconhecer X . Para construí-lo, tomamos todos os itens de I que têm X imediatamente após o ponto, isto é, itens da forma $[A \rightarrow \alpha.X\beta, a]$, avançamos o ponto para obter $[A \rightarrow \alpha.X.\beta, a]$ e calculamos o fechamento desse conjunto. O lookahead a é transferido sem alteração, pois ele depende do contexto em que o item foi criado, não do símbolo recém-reconhecido.

10.2.4 Construção da Tabela LR(1)

A construção começa com o estado inicial I_0 , obtido pelo fechamento do item-semente $[S' \rightarrow .S, \$]$, onde S' é o símbolo de partida aumentado. A partir de I_0 , novos estados são gerados aplicando-se goto para cada símbolo que aparece à direita de algum ponto no estado atual; esse processo é repetido para todos os estados descobertos até que não haja mais novidades.

Com os estados em mãos, a tabela de ações é preenchida da seguinte forma. Para cada item $[A \rightarrow \alpha.a\beta, b]$ em que a é um terminal, insere-se um shift para o estado $\text{goto}(I, a)$ na entrada $\text{action}[I, a]$. Para cada item completo $[A \rightarrow \alpha., a]$ com $A \neq S'$, insere-se uma redução $A \rightarrow \alpha$ na entrada $\text{action}[I, a]$: apenas na coluna do lookahead a , não em todas. Finalmente, quando o item $[S' \rightarrow S., \$]$ está presente, registra-se accept em $\text{action}[I, \$]$. A tabela goto é preenchida com $\text{goto}[I, B] = \text{goto}(I, B)$ para cada não-terminal B . Se nenhum par (I, a) receber duas ações distintas, a gramática é **LR(1)**.

10.3 Exemplo: Tabela LR(1) para Expressões Aritméticas

Considere a gramática de expressões com precedência:

$$\begin{aligned} E &\rightarrow E + T \mid T \\ T &\rightarrow T * F \mid F \\ F &\rightarrow (E) \mid n \end{aligned}$$

Aumentamos a gramática com $S' \rightarrow E$.

10.3.1 Construção dos estados

Estado 0: fechamento de $\{[S' \rightarrow .E, \$]\}$:

$$\begin{aligned} S' &\rightarrow .E && \$ \\ E &\rightarrow .E + T && \$, + \\ E &\rightarrow .T && \$, + \\ T &\rightarrow .T * F && \$, +, * \\ T &\rightarrow .F && \$, +, * \\ F &\rightarrow .(E) && \$, +, *, (\\ F &\rightarrow .n && \$, +, *,) \end{aligned}$$

Estado 1: goto(I_0, E):

$$\begin{aligned} S' &\rightarrow E. && \$ \\ E &\rightarrow E. + T && \$, + \end{aligned}$$

Estado 2: goto(I_0, T):

$$\begin{aligned} E &\rightarrow T. && \$, + \\ T &\rightarrow T. * F && \$, +, * \end{aligned}$$

Estado 3: goto(I_0, F):

$$T \rightarrow F. \quad \$, +, *$$

Estado 4: goto($I_0, ($):

$$\begin{aligned} F &\rightarrow (.E) && \$, +, *, (\\ E &\rightarrow .E + T &&), + \\ E &\rightarrow .T &&), + \\ T &\rightarrow .T * F &&), +, * \\ T &\rightarrow .F &&), +, * \\ F &\rightarrow .(E) &&), +, *, (\\ F &\rightarrow .n &&), +, *, (\end{aligned}$$

Estado 5: goto(I_0, n):

$$F \rightarrow n. \quad \$, +, *,)$$

Estado 6: goto($I_1, +$):

$$\begin{aligned} E &\rightarrow E + .T && \$, + \\ T &\rightarrow .T * F && \$, +, * \\ T &\rightarrow .F && \$, +, * \\ F &\rightarrow .(E) && \$, +, *, (\\ F &\rightarrow .n && \$, +, *,) \end{aligned}$$

Estado 7: $\text{goto}(I_2, \star)$:

$$\begin{array}{ll} T \rightarrow T \star .F & \$, +, \star \\ F \rightarrow .(E) & \$, +, \star,) \\ F \rightarrow .n & \$, +, \star,) \end{array}$$

Estado 8: $\text{goto}(I_4, E)$:

$$\begin{array}{ll} F \rightarrow (E.) & \$, +, \star,) \\ E \rightarrow E. + T &), + \end{array}$$

Estado 9: $\text{goto}(I_4, T)$:

$$\begin{array}{ll} E \rightarrow T. &), + \\ T \rightarrow T. \star F &), +, \star \end{array}$$

Estado 10: $\text{goto}(I_4, F) = \text{goto}(I_6, F) = \text{goto}(I_7, F)$:

$$T \rightarrow F. \quad \$, +, \star,)$$

(Estados 10–12 para os retornos dos subestados de 4 e 6 são análogos; por simplicidade, listamos apenas os mais relevantes.)

Estado 11: $\text{goto}(I_6, T)$:

$$\begin{array}{ll} E \rightarrow E + T. & \$, + \\ T \rightarrow T. \star F & \$, +, \star \end{array}$$

Estado 12: $\text{goto}(I_7, F)$:

$$T \rightarrow T \star F. \quad \$, +, \star,)$$

Estado 13: $\text{goto}(I_8,)$:

$$F \rightarrow (E). \quad \$, +, \star,)$$

Estado 14: $\text{goto}(I_8, +)$:

$$\begin{array}{ll} E \rightarrow E + .T &), + \\ T \rightarrow .T \star F &), +, \star \\ T \rightarrow .F &), +, \star \\ F \rightarrow .(E) &), +, \star, (\\ F \rightarrow .n &), +, \star, (\end{array}$$

Estado 15: $\text{goto}(I_9, \star)$:

$$\begin{array}{ll} T \rightarrow T \star .F &), +, \star \\ F \rightarrow .(E) &), +, \star, (\\ F \rightarrow .n &), +, \star, (\end{array}$$

10.3.2 Tabela LR(1)

Os estados com itens completos têm reduções restritas aos lookaheads do item:

Estado	n	+	*	(	)	\$	E	T	F
0	s5			s4			1	2	3
1	s6					acc			
2	r E→T		s7			r E→T			
3	r T→F		r T→F			r T→F			
4	s5			s4			8	9	3
5	r F→n		r F→n	r F→n		r F→n			
6	s5			s4				11	10
7	s5			s4					12
8	s14			s13					
9	r E→T		s15	r E→T					
10	r T→F		r T→F	r T→F		r T→F			
11	r E→E+T		s7			r E→E+T			
12	r T→T*F		r T→T*F			r T→T*F			
13	r F→(E)		r F→(E)	r F→(E)		r F→(E)			
14	s5			s4				16	10
15	s5			s4					17

(Estados 16 e 17 são análogos a 11 e 12, mas com lookaheads $\{), +\}$.)

Observe que cada ação de reduce aparece apenas nas colunas correspondentes ao lookahead do item, não em todas as colunas como no LR(0). Isso evita conflitos que o SLR não conseguiria resolver quando o conjunto FOLLOW é impreciso.

10.4 O Algoritmo LALR(1)

O LALR(1) (Look-Ahead LR) é uma simplificação prática do LR(1) que reduz o número de estados sem abrir mão dos lookaheads locais. A ideia central é observar que vários estados LR(1) distintos compartilham o mesmo **núcleo**; isto é, os mesmos itens quando ignoramos os lookaheads; e podem ser fundidos em um único estado sem comprometer a correção do analisador na maioria das gramáticas de interesse prático.

10.4.1 Motivação

O LR(1) resolve conflitos ao custo de multiplicar estados: dois contextos diferentes que levam ao mesmo conjunto de itens LR(0) geram estados LR(1) distintos porque seus lookaheads diferem. Na gramática de expressões acima, por exemplo, os estados 3 e 10 têm o mesmo núcleo $T \rightarrow F$, mas lookaheads diferentes: o estado 3, originado fora de parênteses, carrega $\{\$, +, *\}$, enquanto o estado 10, originado dentro de parênteses, carrega $\{\$, +, *,)\}$. Para gramáticas de linguagens de programação realistas, essa duplicação pode fazer o número de estados LR(1) ser uma ordem de grandeza maior do que o número de estados LR(0), tornando as tabelas proibitivamente grandes.

10.4.2 Fusão de Estados

O LALR(1) contorna esse problema fundindo todos os estados LR(1) que compartilham o mesmo núcleo. O estado resultante reúne os lookaheads de todos os estados fundidos, tomando a união dos conjuntos. Todas as transições que apontavam para qualquer um dos estados originais passam a apontar para o estado fundido. Como o núcleo (e portanto a estrutura de shift e goto) é idêntico em todos os estados mesclados, as transições permanecem consistentes após a fusão.

O único risco introduzido pela fusão é o surgimento de conflitos reduce-reduce: se dois itens completos com o mesmo núcleo tinham lookaheads disjuntos antes da fusão, sua união pode fazer com que ambos reivindicuem a mesma entrada na tabela de ações. Quando isso ocorre, a gramática

é LR(1) mas não LALR(1). Vale notar que conflitos shift-reduce nunca são introduzidos pela fusão: um shift é determinado pelo núcleo do item, não pelo lookahead, de modo que um conflito desse tipo no estado fundido já existiria nos estados LR(1) originais.

10.4.3 Construção da Tabela LALR(1)

Na prática, a tabela LALR(1) é obtida calculando-se primeiro todos os estados LR(1), agrupando os que têm o mesmo núcleo e unindo seus lookaheads. A tabela de ações e goto é então preenchida sobre os estados fundidos exatamente da mesma forma que no LR(1): shifts são determinados pelo núcleo do item, reduces são restritos ao lookahead do item completo e accept ocorre ao atingir o item $[S' \rightarrow S, \$]$. O resultado é um analisador com o mesmo número de estados do LR(0) e lookaheads mais precisos que o SLR.

10.5 Exemplo: Tabela LALR(1) para Expressões Aritméticas

Retomamos a mesma gramática de expressões do exemplo anterior.

10.5.1 Identificação de estados com o mesmo núcleo

Comparando os estados LR(1) construídos acima, identificamos os seguintes grupos com o mesmo núcleo:

Núcleo	Estados LR(1)	Lookaheads unidos
$T \rightarrow F.$	3 e 10	$\{\$, +, *,)\}$
$E \rightarrow E + .T, T \rightarrow .T * F, T \rightarrow .F, \dots$	6 e 14	$\{\$, +,)\}$
$T \rightarrow T * .F, F \rightarrow .(E), F \rightarrow .n$	7 e 15	$\{\$, +, *,)\}$
$E \rightarrow E + T., T \rightarrow T * F$	11 e 16	$\{\$, +,)\}$
$T \rightarrow T * F.$	12 e 17	$\{\$, +, *,)\}$

Os demais estados (0, 1, 2, 4, 5, 8, 9, 13) têm núcleos únicos e não são fundidos.

Após a fusão, os estados LALR são nomeados como: **0, 1, 2, 3+10, 4, 5, 6+14, 7+15, 8, 9, 11+16, 12+17, 13.**

10.5.2 Estados LALR resultantes

Estado 3+10 (fusão de 3 e 10):

$$T \rightarrow F. \quad \$, +, *,)$$

Estado 6+14 (fusão de 6 e 14):

$$\begin{aligned} E \rightarrow E + .T & \quad \$, +,) \\ T \rightarrow .T * F & \quad \$, +, *,) \\ T \rightarrow .F & \quad \$, +, *,) \\ F \rightarrow .(E) & \quad \$, +, *,), (\\ F \rightarrow .n & \quad \$, +, *,), (\end{aligned}$$

Estado 7+15 (fusão de 7 e 15):

$$\begin{aligned} T \rightarrow T * .F & \quad \$, +, *,) \\ F \rightarrow .(E) & \quad \$, +, *,), (\\ F \rightarrow .n & \quad \$, +, *,), (\end{aligned}$$

Estado 11+16 (fusão de 11 e 16):

$$\begin{aligned} E &\rightarrow E + T. \quad \$, +,) \\ T &\rightarrow T \star F \quad \$, +, \star,) \end{aligned}$$

Estado 12+17 (fusão de 12 e 17):

$$T \rightarrow T \star F. \quad \$, +, \star,)$$

10.5.3 Tabela LALR(1)

A tabela LALR é construída sobre os estados fundidos. Usamos nomes curtos para os estados fundidos: **A** = 3+10, **B** = 6+14, **C** = 7+15, **D** = 11+16, **E** = 12+17.

Estado	n	+	★	()	\$	E	T	F
0	s5			s4		1	2	A
1		sB			acc			
2		r E→T	sC		r E→T			
A		r T→F	r T→F	r T→F	r T→F			
4	s5			s4		8	9	A
5		r F→n	r F→n	r F→n	r F→n			
B	s5			s4			D	A
C	s5			s4				E
8		sB		s13				
9		r E→T	sC	r E→T				
D		r E→E+T	sC	r E→E+T	r E→E+T			
E		r T→T★F	r T→T★F	r T→T★F	r T→T★F			
13		r F→(E)	r F→(E)	r F→(E)	r F→(E)			

Comparando com a tabela LR(1), a tabela LALR tem menos estados (13 em vez de 17), mas as ações de reduce são ligeiramente mais conservadoras do que no SLR; por exemplo, no estado **A** ($T \rightarrow F.$), a ação de reduce agora inclui $)$ como lookahead, o que é correto porque este estado resulta da fusão de um contexto externo (estado 3) e de um contexto dentro de parênteses (estado 10). Como nenhum conflito foi introduzido pela fusão, portanto essa gramática é **LALR(1)**.

10.6 Implementação

Nesta seção descrevemos os principais tipos e funções das implementações Haskell dos algoritmos de construção de tabela LR(1) e LALR(1), presentes nos módulos

```
Parsing.LR.LR1.BuildTable e
Parsing.LR.LALR.BuildTable.
```

Ambos compartilham a mesma representação de estados e reutilizam o algoritmo de parsing genérico do módulo `Parsing.LR.LRParser`.

10.6.1 LR(1)

Itens e estados

Um item LR(1) $[A \rightarrow \beta.\gamma, a]$ é representado pelo tipo:

```
data LR1Item = LR1Item
  { lr1LHS :: String      -- não-terminal do lado esquerdo
  , lr1Before :: [Symbol] -- símbolos antes do ponto
  , lr1After  :: [Symbol] -- símbolos depois do ponto
  , lr1Lookahead :: String -- token de lookahead ou "$"
  }
```

O campo `lr1Lookahead` é o componente essencial que distingue os itens LR(1) dos itens LR(0): ele registra o terminal que pode aparecer imediatamente após uma redução por $A \rightarrow \beta$ neste contexto específico. Um estado LR(1) é simplesmente um conjunto de itens:

```
type LR1State = Set LR1Item
```

Fechamento

A função `closure` implementa o fechamento de um conjunto de itens LR(1). Para cada item $[A \rightarrow \alpha.B\beta, a]$, ela prediz todos os itens $[B \rightarrow \gamma, b]$ onde $b \in \text{FIRST}(\beta a)$. O cálculo de $\text{FIRST}(\beta a)$ é feito pela função auxiliar `lookaheadsFor`:

```
lookaheadsFor :: Grammar -> FirstSet -> [Symbol] -> String -> [String]
lookaheadsFor g fs beta a =
  filter (/= "\\") $ firstForSequence g fs (beta ++ [Terminal a])
```

Ela concatena β com o lookahead a tratado como terminal e chama `firstForSequence`, que já lida com anulabilidade: se β for anulável, a é incluído no resultado; caso contrário, apenas $\text{FIRST}(\beta)$ aparece. O fechamento em si aplica essa lógica repetidamente, usando o ponto fixo da função `fixedPoint`:

```
closure :: Grammar -> FirstSet -> LR1State -> LR1State
closure g fs = fixedPoint step
  where
    step current = Set.union current newItems
    where
      newItems = Set.fromList $ concatMap expand (Set.toList current)
      expand item = case lr1After item of
        (NonTerminal b : beta) ->
          [ LR1Item b [] prod la
            | prod <- prods b
            , la   <- lookaheadsFor g fs beta (lr1Lookahead item)
          ]
        _ -> []
```

Função goto e construção dos estados

A função `goto` $g \text{ fs state } X$ avança o ponto nos itens que têm X após ele e calcula o fechamento do resultado, preservando o lookahead de cada item:

```
goto :: Grammar -> FirstSet -> LR1State -> Symbol -> LR1State
goto g fs items sym = closure g fs $ Set.fromList shifted
  where
    shifted = mapMaybe shiftItem (Set.toList items)
    shiftItem item = case lr1After item of
      (s : rest) | s == sym ->
        Just $ LR1Item (lr1LHS item) (lr1Before item ++ [s])
                  rest (lr1Lookahead item)
      _ -> Nothing
```

O conjunto canônico de estados LR(1) é construído por uma BFS que parte do fechamento do item-semente $[S' \rightarrow .S, \$]$ e expande cada estado pelos símbolos que aparecem após algum ponto:

```
buildLR1States :: Grammar -> [LR1State]
buildLR1States g = buildFrom [initial] [initial]
  where
    seed      = LR1Item "S'" [] [NonTerminal startSym] "$"
    initial = closure g' fs (Set.singleton seed)

    buildFrom seen [] = seen
    buildFrom seen (current : queue) =
      let newStates = [goto g' fs current sym | sym <- nextSymbols current]
          unseen    = filter (`notElem` seen) newStates
      in buildFrom (seen ++ unseen) (queue ++ unseen)
```

Tabela de ações

A construção da tabela em `buildLR1ParsingTable` distingue-se do LR(0) apenas na etapa de reduce: em vez de inserir a redução em todos os terminais, ela usa exclusivamente o lookahead do item completo:

```
reduceA =
  [ ((i, lr1Lookahead item), Reduce (lr1LHS item) (lr1Before item))
  | item <- Set.toList st
  , null (lr1After item)
  , lr1LHS item /= "S'"
  ]
```

Esse detalhe é exatamente o que torna o LR(1) mais preciso que o LR(0) e o SLR: a decisão de reduzir $A \rightarrow \beta$ depende do token seguinte calculado localmente para aquele item, não de um conjunto FOLLOW global.

10.6.2 LALR(1)

Núcleo de um estado

A ideia central do LALR(1) é identificar estados LR(1) com o mesmo **núcleo**, o conjunto de itens sem os lookaheads, e fundi-los. O núcleo de um item é representado pelo tipo:

```
type ItemCore = (String, [Symbol], [Symbol])

itemCore :: LR1Item -> ItemCore
itemCore item = (lr1LHS item, lr1Before item, lr1After item)

stateCore :: LR1State -> Set ItemCore
stateCore = Set.map itemCore
```

Dois estados com `stateCore s1 == stateCore s2` possuem exatamente os mesmos itens LR(0) subjacentes e são candidatos à fusão.

Fusão de estados

A função `mergeGroup` recebe um grupo de estados LR(1) com o mesmo núcleo e produz um único estado cujos itens têm os lookaheads da **união** de todos os estados do grupo:

```
mergeGroup :: [LR1State] -> LR1State
mergeGroup group =
    let byCore = Map.fromListWith Set.union
        [(itemCore i, Set.singleton (lr1Lookahead i))
         | i <- concatMap Set.toList group]
    in Set.fromList
        [LR1Item lhs before after la
         | ((lhs, before, after), las) <- Map.toList byCore
         , la <- Set.toList las]
```

Construção dos estados LALR

A função `buildLALRStates` constrói o conjunto canônico de estados LALR em três passos: primeiro chama `buildLR1States` para obter todos os estados LR(1); depois agrupa-os por núcleo, preservando a ordem de primeira ocorrência de cada núcleo (o que faz os índices LALR coincidirem com os índices LR(0)/SLR); por fim, funde cada grupo com `mergeGroup`:

```
buildLALRStates :: Grammar -> [LR1State]
buildLALRStates g =
    let lr1States = buildLR1States g
        coreOrder = nub (map stateCore lr1States)
        grouped   = Map.fromListWith (++)
            [(stateCore s, [s]) | s <- lr1States]
    in [mergeGroup (grouped Map.! core) | core <- coreOrder]
```

Tabela e busca por núcleo

A tabela LALR é construída exatamente como a tabela LR(1), mas as transições são resolvidas por **correspondência de núcleo** em vez de igualdade exata. Isso é necessário porque `goto g' fs mergedState X` devolve um estado LR(1) intermediário que pode não estar diretamente na lista de estados LALR; o estado destino correto é aquele cujo núcleo coincide:

```
findLALRStateIndex :: [LR1State] -> LR1State -> Maybe Int
findLALRStateIndex sts target =
    findIndex (\s -> stateCore s == stateCore target) sts
```

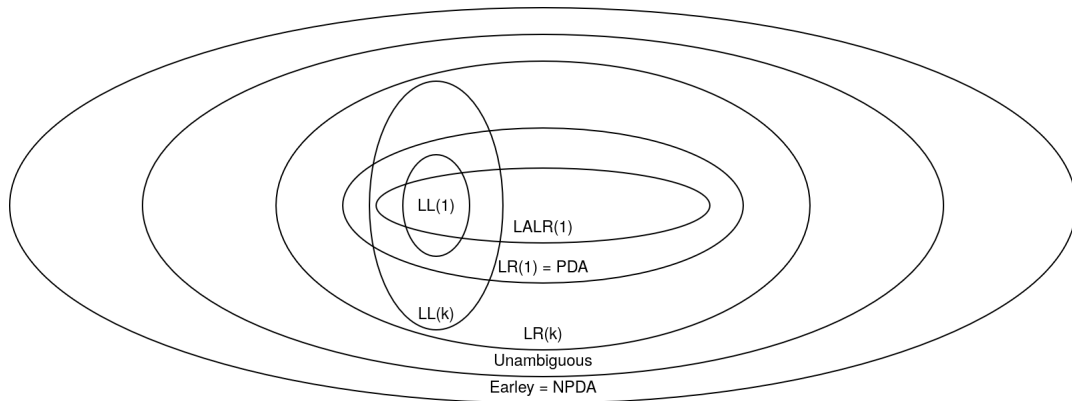
Com isso, as entradas de shift e goto da tabela LALR apontam sempre para estados fundidos, e as entradas de reduce usam os lookaheads ampliados pela fusão: mais precisos que o FOLLOW global do SLR, mas possivelmente mais amplos que os lookaheads originais do LR(1).

10.7 Comparando algoritmos de análise sintática

Os analisadores de Earley são os mais poderosos que já vimos: eles podem reconhecer qualquer gramática, incluindo gramáticas ambíguas. Eles podem até reconhecer línguas ambíguas, línguas para as quais não existe uma gramática não ambígua, como $\{a^n b^m c^p \mid n = m \vee m = p\}$. Os analisadores de Earley exploram todo o poder do autômato de pilha não determinístico (NPDA), que você pode imaginar como um analisador shift-reduce com um oráculo indicando o que fazer em caso de conflitos.

Parsing expression grammars permitem expressar gramáticas para línguas que não são livres de contexto, como $\{a^n b^n c^n \mid n \geq 0\}$. Porém, não se sabe se existem PEGs para línguas livres de contexto simples: como a linguagem de palíndromos. PEGs possuem a vantagem de não possuir problemas de ambiguidade mas introduzem dificuldades que não estão presentes em especificações baseadas em gramáticas.

Algoritmos baseados em tabelas (LL(1), LR(1) e LALR) podem ser caracterizados pelo seguinte diagrama, que demonstra a relação entre diferentes classes de linguagens livres de contexto.



10.8 Conclusão

Os algoritmos LR(1) e LALR(1) estendem o LR(0) e o SLR ao associar lookaheads **locais** a cada item, em vez de usar o conjunto FOLLOW global. O LR(1) é o mais preciso, mas gera o maior número de estados. O LALR(1) oferece um equilíbrio excelente: funde estados LR(1) com o mesmo núcleo, mantendo o número de estados igual ao LR(0)/SLR e os lookaheads mais precisos que o SLR. Por isso, o LALR(1) é o algoritmo de escolha para a maioria dos geradores de analisadores sintáticos práticos.

Capítulo 11

Geradores de analisadores sintáticos

11.1 Introdução

No capítulo anterior estudamos os algoritmos LR(1) e LALR(1) de forma teórica e implementamos suas tabelas em Haskell. Na prática, contudo, construir essas tabelas à mão para uma gramática real seria impraticável. Assim como o Alex automatiza a construção de analisadores léxicos a partir de expressões regulares, o **Happy** automatiza a construção de analisadores sintáticos a partir de especificações gramaticais no estilo BNF. O Happy gera analisadores LALR(1) e é a ferramenta padrão para este fim no ecossistema Haskell: o próprio compilador GHC usa um parser gerado pelo Happy.

Este capítulo apresenta o Happy através da especificação do parser de expressões aritméticas, disponível em

`Exp/Frontend/Parser/Happy/ExpParser.y`

Mostramos como cada parte do arquivo `.y` se relaciona com os conceitos vistos nos capítulos anteriores e como usar a opção `-i` para inspecionar a tabela LALR gerada. Por fim, ilustramos os dois tipos de conflito que podem surgir em gramáticas ambíguas: conflitos shift-reduce e conflitos reduce-reduce.

11.2 Estrutura de uma especificação Happy

Um arquivo de especificação Happy (extensão `.y`) é dividido em quatro partes, separadas pelo marcador `%%`:

```
{ cabeçalho Haskell }

diretivas e declaração de tokens

%%

regras gramaticais

{ rodapé Haskell }
```

11.2.1 Cabeçalho

A primeira seção, delimitada por chaves, contém código Haskell puro que é copiado literalmente para o início do arquivo gerado. É aqui que se declara o nome do módulo e os imports necessários:

```
{
module Exp.Frontend.Parser.Happy.ExpParser (expParser, parserTest) where

import Exp.Frontend.Lexer.Token
import Exp.Frontend.Lexer.Alex.ExpLexer hiding (lexer)
```

```
import Exp.Frontend.Syntax.ExpSyntax
}
```

11.2.2 Diretivas

As diretivas configuram o parser gerado. As mais comuns são:

Diretiva	Significado
<code>%name p N</code>	Gera a função <code>p</code> que inicia o parsing pelo não-terminal <code>N</code>
<code>%tokentype { T }</code>	Define o tipo Haskell dos tokens consumidos
<code>%monad { M }{ (>=>) }{ return }</code>	Executa o parser dentro da mônada <code>M</code>
<code>%lexer { f }{ eof }</code>	Integra um lexer incremental; <code>f</code> obtém o próximo token e <code>eof</code> é o token de fim de entrada
<code>%error { h }</code>	Chama a função <code>h</code> quando ocorre um erro de sintaxe
<code>%expect n</code>	Declara que a gramática possui exatamente <code>n</code> conflitos shift-reduce esperados; o Happy falha se o número for diferente

No parser de expressões, essas diretivas ficam assim:

```
%name parser Exp
%monad {Alex}{(>=>)}{return}
%tokentype { Token }
%error      { parseError }
%lexer {lexer}{Token _ TEOF}
```

A diretiva `%monad` integra o Happy com a mônada `Alex` do lexer gerado pelo Alex. Com `%lexer`, o Happy chama a função `lexer` cada vez que precisa de um token, em vez de receber a lista completa de antemão. Esse modo incremental é necessário quando o lexer é monádico.

11.2.3 Declaração de tokens

A seção `%token` mapeia nomes simbólicos usados nas regras gramaticais para padrões de correspondência (em Haskell) contra o tipo de token. Cada linha tem a forma:

```
nome_simbólico { padrão Haskell }
```

No parser de expressões:

```
%token
    num      {Token _ (TNumber $$)}
    '('      {Token _ TLParen}
    ')'      {Token _ TRParen}
    '+'      {Token _ TPlus}
    '*'      {Token _ TTimes}
```

O uso de `$$` no padrão `Token \_ (TNumber $$)` é especial: ele captura o valor inteiro contido no token e o torna disponível nas ações semânticas como `$1`, `$2`, etc. Para tokens sem valor associado, como `TLParen`, o padrão apenas verifica que o construtor bate.

11.2.4 Declarações de precedência

As declarações `%left`, `%right` e `%nonassoc` servem para resolver conflitos shift-reduce que surgem em gramáticas de expressões escritas de forma ambígua. Cada linha associa uma precedência (determinada pela ordem das declarações, de menor para maior) e uma associatividade a um ou mais tokens:

```
%left '+'
%left '*'
```

Neste exemplo, `*` tem precedência mais alta que `+` (pois aparece depois), e ambos são associativos à esquerda. Veremos na seção sobre conflitos como essas declarações eliminam a ambiguidade.

11.2.5 Regras gramaticais

Após o marcador `%`, definem-se as produções da gramática no estilo BNF aumentado com ações semânticas em Haskell:

```
Exp : num      { EInt $1 }
    | Exp '+' Exp { $1 :+: $3 }
    | Exp '*' Exp { $1 **: $3 }
    | '(' Exp ')' { $2 }
```

Cada alternativa é uma produção seguida de uma ação entre chaves. Os símbolos `$1`, `$2`, `$3` referem-se, respectivamente, ao primeiro, segundo e terceiro símbolo do lado direito da produção. A ação é executada quando o parser reduz por aquela produção, construindo o nó correspondente da AST.

O tipo de retorno de cada não-terminal é inferido pelo compilador Haskell a partir das ações: como `EInt $1` e `$1 :+: $3` têm tipo `Exp`, o não-terminal `Exp` produz valores do tipo `Exp`.

11.2.6 Rodapé

A última seção, também delimitada por chaves, contém funções auxiliares usadas nas ações ou nas diretivas:

```
{
parserTest :: String -> IO ()
parserTest s = do
  r <- expParser s
  print r

parseError (Token (line, col) lexeme)
  = alexError $ "Parse error while processing lexeme: " ++ show lexeme
               ++ "\n at line " ++ show line ++ ", column " ++ show col

lexer :: (Token -> Alex a) -> Alex a
lexer = (<< alexMonadScan)

expParser :: String -> IO (Either String Exp)
expParser content = do
  pure $ runAlex content parser
}
```

A função `parseError` recebe o token que causou o erro e usa `alexError` para propagar a mensagem de erro dentro da mônada Alex. A função `lexer` adapta `alexMonadScan` (que produz `Alex Token`) para a interface esperada pelo Happy (que recebe uma continuação `Token -> Alex a`).

11.3 Gerando o parser e inspecionando a tabela

Para gerar o parser a partir da especificação, basta invocar o Happy:

```
happy ExpParser.y
happy -o ExpParser.hs ExpParser.y
```

Quando a especificação faz parte de um projeto Cabal, o próprio Cabal detecta arquivos .y em `build-tool-depends` (com happy listado) e os processa automaticamente antes da compilação Haskell.

11.3.1 A opção -i

A opção `-i` instrui o Happy a gerar um **arquivo de informação** (por padrão `ExpParser.info`) que descreve em detalhe a tabela LALR construída:

```
happy -i ExpParser.y
```

O arquivo gerado contém:

1. **A gramática aumentada:** as produções numeradas, incluindo a produção inicial $S' \rightarrow Exp$.
2. **Conjuntos FIRST** de cada não-terminal.
3. **Os estados do autômato:** cada estado lista seus itens LR(1), com os lookaheads calculados pelo algoritmo LALR.
4. **A tabela de ações:** para cada par (estado, terminal), a ação: shift, reduce ou accept.
5. **A tabela goto:** para cada par (estado, não-terminal), o estado destino.
6. **Conflitos:** se houver, eles são listados com o estado e o token envolvidos, indicando qual ação o Happy escolheu (por padrão, shift vence em conflitos shift-reduce).

Este arquivo é extremamente útil para depurar gramáticas: ao ver em qual estado um conflito ocorre e quais itens ele contém, é possível identificar a ambiguidade na gramática e corrigi-la com declarações de precedência ou reestruturando as produções.

11.4 Conflitos shift-reduce

Um conflito shift-reduce ocorre quando, em um determinado estado, o parser pode tanto deslocar (shift) o próximo token quanto reduzir por alguma produção. O caso mais comum é o de gramáticas de expressões escritas de forma ambígua.

Considere a gramática de expressões **sem** declarações de precedência:

```
Exp : num
    | Exp '+' Exp
    | Exp '*' Exp
    | '(' Exp ')'
```

Ao analisar a entrada `1 + 2 * 3`, após ter empilhado `Exp '+' Exp` (com o segundo `Exp` valendo 2), o parser vê `*` na entrada. Ele pode:

- **Reduzir** `Exp '+' Exp` para `Exp` (interpretando `(1 + 2) * 3`), ou
- **Deslocar** `*` (interpretando `1 + (2 * 3)`).

Sem orientação adicional, o Happy reporta o conflito e, por padrão, escolhe sempre o shift. A mensagem no terminal seria:

```
ExpParser.y: shift/reduce conflicts: 4
```

E no arquivo .info, cada conflito é detalhado, por exemplo:

State 7

```
Exp -> Exp . '+' Exp      (rule 2)
Exp -> Exp . '*' Exp      (rule 3)
Exp -> Exp '+' Exp .      (rule 2)

'*'    shift, and enter state 5
        (reduce using rule 2)

'+'    shift, and enter state 4
        (reduce using rule 2)

')'    reduce using rule 2
'$'    reduce using rule 2
```

A resolução correta é declarar precedência e associatividade antes do `%%`. Ao escrevermos:

```
%left '+'
%left '*'
```

o Happy usa essas informações para resolver cada conflito: quando o token a deslocar tem precedência maior que a produção a reduzir, faz shift; quando tem menor, reduz; quando são iguais, a associatividade decide. Com as declarações acima, `*` tem precedência maior que `+`, portanto `1 + 2 * 3` é interpretado como `1 + (2 * 3)`, e `%expect 0` pode ser declarado para confirmar que nenhum conflito resta.

Outro exemplo clássico de conflito shift-reduce é o **do else pendente** (*dangling else*), presente em linguagens com construção `if-then-else` opcional:

```
Stmt : 'if' Exp 'then' Stmt
      | 'if' Exp 'then' Stmt 'else' Stmt
      | 'other'
```

Ao analisar `if e1 then if e2 then s1 else s2`, após empilhar o `then` interior, o parser vê `else` e tem dúvida: este `else` pertence ao `if` externo ou ao interno? Shift associa o `else` ao `if` mais próximo (interno), que é o comportamento convencional na maioria das linguagens. A declaração `%shift!` ou simplesmente aceitar o comportamento padrão do Happy resolve o conflito.

11.5 Conflitos reduce-reduce

Um conflito reduce-reduce ocorre quando, em um determinado estado e para um mesmo token de lookahead, dois itens completos diferentes indicam reduções por produções distintas. Isso geralmente revela uma ambiguidade estrutural na gramática que não pode ser resolvida por precedência.

Considere a gramática:

```
Expr : termo
      | termo
```

```

termo : 'x'
      | 'x'

```

De forma mais realista, o conflito aparece quando dois não-terminais distintos derivam a mesma sequência de terminais e são intercambiáveis no mesmo contexto:

```

Prog : Decl
      | Expr

Decl : 'id'
Expr : 'id'

```

Ao ver o token `id` e precisar reduzir, o parser não sabe se deve criar um nó `Decl` ou um nó `Expr`. O arquivo `.info` mostraria algo como:

```

State 1

Decl -> 'id' .          (rule 1)
Expr -> 'id' .          (rule 2)

'$'    reduce using rule 1
        (reduce using rule 2)

```

O Happy resolve o conflito escolhendo sempre a **primeira regra** na ordem em que aparece no arquivo, mas emite um aviso. Diferentemente dos conflitos `shift-reduce`, conflitos `reduce-reduce` raramente têm uma resolução correta por padrão: eles quase sempre indicam que a gramática precisa ser reformulada.

A solução mais comum é tornar os dois não-terminais sintáticos distintos por contexto, introduzindo palavras-chave discriminadoras:

```

Prog : 'let' 'id'        -- declaração
      | 'id'             -- expressão

```

Ou, alternativamente, desambiguar na fase semântica (análise de tipos ou tabela de símbolos), como fazem as gramáticas de linguagens como C, onde `typedef` torna certos identificadores indistinguíveis de tipos sem informação de contexto.

11.6 Conclusão

Este capítulo apresentou o Happy, o gerador de analisadores sintáticos padrão em Haskell. Vimos como um arquivo `.y` é estruturado em quatro partes (cabeçalho, diretivas, regras e rodapé) e como cada diretiva configura o parser gerado. Estudamos a especificação concreta do parser de expressões aritméticas, que integra o Happy com o lexer monádico gerado pelo Alex. Mostramos como a opção `-i` permite inspecionar os estados LALR e os conflitos detectados, tornando o processo de depuração de gramáticas transparente. Por fim, ilustramos os dois tipos de conflito (`shift-reduce`, geralmente resolúvel com declarações de precedência, e `reduce-reduce`, que normalmente exige a reformulação da gramática), conectando os conceitos teóricos dos capítulos anteriores à prática do desenvolvimento de compiladores em Haskell.

Parte IV

Semântica formal e interpretadores

Capítulo 12

Introdução à semântica operacional

12.1 Introdução

Até agora, estudamos como reconhecer se uma sequência de tokens pertence a uma linguagem: a análise léxica e sintática. O próximo passo é atribuir **significado** aos programas reconhecidos: dado um programa sintaticamente correto, o que ele *faz*? Essa pergunta é o objeto de estudo da **semântica de linguagens de programação**.

Existem várias maneiras de formalizar o significado de um programa. Neste capítulo, estudaremos a **semântica operacional**, que define o significado de um programa descrevendo como ele é *executado*, passo a passo, por uma máquina abstrata. A semântica operacional é particularmente útil para a construção de compiladores e interpretadores, pois a sua especificação se traduz diretamente em código.

Apresentamos dois estilos de semântica operacional, ambos para uma linguagem de expressões aritméticas com adição, multiplicação e números inteiros:

- **Small-step** (*pequenos passos*): define a execução como uma sequência de reduções elementares, cada uma simplificando a expressão um pouco.
- **Big-step** (*grandes passos*): define diretamente o valor final de uma expressão, abstraindo os passos intermediários.

Por fim, mostramos como a semântica big-step se traduz em um interpretador Haskell, disponível em

Exp/Interp/ExpInterp.hs

12.2 A Linguagem de Expressões

A linguagem que estudaremos é formada por três construções: números inteiros, adição e multiplicação. Sua sintaxe abstrata pode ser descrita pela gramática:

$$e ::= n \mid e_1 + e_2 \mid e_1 * e_2$$

onde n representa um número inteiro. Em Haskell, o tipo de dado correspondente é:

```
data Exp
= EInt Int
| Exp :+: Exp
| Exp **: Exp
```

Os construtores infixos `(:::)` e `(**)` refletem diretamente a estrutura binária das operações.

12.3 Semântica Small-Step

A semântica **small-step** (ou semântica de redução) define a execução de uma expressão como uma sequência de passos de simplificação. Cada passo é descrito por uma **relação de redução** $e \rightarrow e'$, que se lê “a expressão e reduz em um passo para e' ”.

12.3.1 Valores

Antes de apresentar as regras, precisamos distinguir as expressões que já chegaram ao seu resultado final, os **valores**, das que ainda podem ser reduzidas. Para a linguagem de expressões aritméticas, o único tipo de valor é um número inteiro:

$$v ::= n$$

12.3.2 Regras de Redução

As regras de redução são apresentadas na forma de um sistema de inferência. Cada regra tem a forma:

$$\frac{\text{premissas}}{\text{conclusão}} \{Nome\}$$

Se não houver premissas, a regra é um **axioma** e é escrita sem a linha horizontal. Para a linguagem de expressões, temos as seguintes regras:

Adição de valores:

$$\frac{}{n_1 + n_2 \rightarrow n_1 \hat{+} n_2} \{Add\}$$

onde $\hat{+}$ denota a adição aritmética de inteiros (para distinguir o operador da linguagem do operador matemático).

Redução no lado esquerdo da adição:

$$\frac{e_1 \rightarrow e'_1}{e_1 + e_2 \rightarrow e'_1 + e_2} \{AddL\}$$

Redução no lado direito da adição:

$$\frac{e_2 \rightarrow e'_2}{n_1 + e_2 \rightarrow n_1 + e'_2} \{AddR\}$$

A regra (AddR) só se aplica quando o lado esquerdo já é um valor n_1 , garantindo uma ordem de avaliação da esquerda para a direita.

As regras para multiplicação são análogas:

$$\frac{}{n_1 * n_2 \rightarrow n_1 \hat{*} n_2} \{Mul\}$$

$$\frac{e_1 \rightarrow e'_1}{e_1 * e_2 \rightarrow e'_1 * e_2} \{MulL\}$$

$$\frac{e_2 \rightarrow e'_2}{n_1 * e_2 \rightarrow n_1 * e'_2} \{MulR\}$$

12.3.3 Sequências de Redução

A execução completa de uma expressão é uma sequência de reduções que termina em um valor:

$$e \rightarrow e_1 \rightarrow e_2 \rightarrow \cdots \rightarrow v$$

Escrevemos $e \rightarrow^* v$ quando existe tal sequência (zero ou mais passos). Para uma expressão que já é um valor, temos $v \rightarrow^* v$ com zero passos.

12.3.4 Exemplo: Small-Step

Considere a expressão $(2 + 3) * 4$. A redução small-step procede assim:

$$(2 + 3) * 4 \xrightarrow{\text{MulL}} 5 * 4 \xrightarrow{\text{Mul}} 20$$

O passo 1 usa (MulL) com premissa $2 + 3 \rightarrow 5$ (pela regra Add). O passo 2 usa (Mul) diretamente, pois ambos os operandos já são valores.

Considere agora $1 + 2 * 3$. Aplicando (AddL) e depois (MulL):

$$1 + 2 * 3 \xrightarrow{\text{AddR}} 1 + 6 \xrightarrow{\text{Add}} 7$$

Note que a regra (AddR) se aplica porque o lado esquerdo 1 já é um valor, forçando a avaliação do subtermos $2 * 3$ antes da soma.

12.4 Semântica Big-Step

A semântica **big-step** (ou semântica natural) define diretamente a relação entre uma expressão e o seu valor final, abstraindo todos os passos intermediários. A relação é escrita $e \Downarrow v$, que se lê “a expressão e avalia para o valor v ”.

12.4.1 Regras de Avaliação

Um número avalia para si mesmo:

$$\frac{}{n \Downarrow n} \{Num\}$$

Adição:

$$\frac{e_1 \Downarrow n_1 \quad e_2 \Downarrow n_2}{e_1 + e_2 \Downarrow n_1 \hat{+} n_2} \{Add\}$$

Multiplicação:

$$\frac{e_1 \Downarrow n_1 \quad e_2 \Downarrow n_2}{e_1 * e_2 \Downarrow n_1 \hat{*} n_2} \{Mul\}$$

A estrutura é mais direta que a semântica small-step: cada regra corresponde exatamente a um construtor da árvore sintática, e as premissas avaliam recursivamente os sub-termos.

12.4.2 Exemplos: Big-Step

Exemplo 1: $(2 + 3) * 4 \Downarrow 20$

A derivação é uma árvore de inferência:

$$\frac{\frac{2 \Downarrow 2 \quad 3 \Downarrow 3}{2 + 3 \Downarrow 5} \quad 4 \Downarrow 4}{(2 + 3) * 4 \Downarrow 20}$$

Exemplo 2: $1 + 2 * 3 \Downarrow 7$

$$\frac{1 \Downarrow 1 \quad \frac{2 \Downarrow 2 \quad 3 \Downarrow 3}{2 * 3 \Downarrow 6}}{1 + 2 * 3 \Downarrow 7}$$

Note que a árvore reflete diretamente a estrutura da árvore sintática abstrata da expressão, é exatamente essa correspondência que torna a semântica big-step tão conveniente para a implementação de interpretadores.

12.5 Implementação em Haskell

A semântica big-step se traduz em Haskell de forma quase literal. Cada regra de inferência corresponde a uma equação da função `eval`:

```
module Exp.Interp.ExpInterp (eval) where

import Exp.Frontend.Syntax.ExpSyntax

eval :: Exp -> Int
eval (EInt n)      = n
eval (e1 :+: e2)   = eval e1 + eval e2
eval (e1 :*: e2)   = eval e1 * eval e2
```

A correspondência com as regras semânticas é direta:

- A equação `eval (EInt n) = n` implementa a regra (Num): um número n avalia para si mesmo.
- A equação `eval (e1 :+: e2) = eval e1 + eval e2` implementa a regra (Add): avalia recursivamente e_1 e e_2 e soma os resultados.
- A equação `eval (e1 :*: e2) = eval e1 * eval e2` implementa a regra (Mul) da mesma forma.

O casamento de padrões (pattern matching) de Haskell corresponde às premissas de cada regra, e a chamada recursiva a `eval` corresponde a aplicar a regra à sub-expressão.

12.5.1 Executando o Interpretador

O interpretador é parte do compilador `exp`. Para usá-lo em modo de interpretação, basta invocar:

```
cabal run exp -- --interp --file input.exp
```

onde `input.exp` contém uma expressão na linguagem. Por exemplo, para a entrada

$(2 + 3) * 4$

o interpretador produz 20, confirmando o resultado que calculamos pelas derivações semânticas acima.

12.5.2 Totalidade e Terminação

A função `eval` é **total**: está definida para todas as expressões do tipo `Exp` e sempre termina. Isso pode ser verificado observando que cada chamada recursiva é feita sobre sub-expressões estritamente menores, a estrutura da árvore sintática abstrata diminui a cada nível. Essa propriedade é garantida pela indução estrutural e corresponde, na semântica, ao fato de que toda expressão da linguagem possui um valor (não há expressões que ficam presas sem produzir resultado).

12.6 Conclusão

Neste capítulo, introduzimos a semântica operacional como ferramenta para definir o significado de programas. Estudamos dois estilos complementares:

- A **semântica small-step** expõe cada passo de redução individualmente, permitindo raciocinar sobre o processo de execução.
- A **semântica big-step** relaciona expressões diretamente aos seus valores finais e se traduz quase literalmente em uma função recursiva Haskell.

A implementação do interpretador `eval` demonstra que uma especificação semântica formal não é apenas um documento teórico: ela é um projeto de implementação preciso. Nos próximos capítulos, estenderemos esta linguagem com variáveis, comandos e funções, exigindo que a semântica modele também **ambientes** e **efeitos**.

Capítulo 13

Semântica operacional para estado

13.1 Introdução

No capítulo anterior, estudamos a semântica operacional de uma linguagem de expressões aritméticas puras. Naquela linguagem, o valor de uma expressão depende apenas da sua estrutura sintática: não há variáveis, não há memória, não há efeitos colaterais. A execução é, portanto, uma função pura: dada a expressão, o resultado é unicamente determinado.

Programas reais, contudo, manipulam **estado**: valores são armazenados em variáveis, lidos da entrada e escritos na saída. Para modelar formalmente esse comportamento, a semântica operacional precisa ser estendida com um componente que representa o estado da memória em cada momento da execução.

Este capítulo estende a linguagem de expressões com **variáveis** e apresenta uma linguagem de **comandos** simples, a linguagem *Line*, que inclui atribuição, leitura da entrada padrão e impressão. Definimos a semântica small-step e big-step para essa linguagem e mostramos como a especificação semântica se traduz diretamente no interpretador disponível em

Line/Interp/LineInterp.hs.

13.2 A Linguagem Line

A linguagem *Line* é uma linguagem imperativa mínima. Um programa é uma sequência de comandos (instruções) que são executados em ordem. A sua sintaxe abstrata é:

$$\begin{array}{ll} prog & ::= s^* \\ s & ::= x := e \\ & \quad | \text{read } x \\ & \quad | \text{print } e \\ e & ::= n \mid x \mid e_1 + e_2 \mid e_1 * e_2 \end{array}$$

onde n é um inteiro, x é um nome de variável e s^* denota uma sequência (possivelmente vazia) de comandos.

Em Haskell, esses tipos são definidos como:

```
data Line = Line [Stmt]

data Stmt
  = SAssign Var Exp
  | SRead   Var
  | SPrint  Exp

data Exp
  = EInt Int
  | EVar Var
```

```

| Exp :+: Exp
| Exp **: Exp

type Var = String

```

13.3 Ambientes

Para dar significado a um programa que usa variáveis, precisamos de uma noção de **ambiente** (uma representação da memória): uma função parcial $\sigma : Var \rightarrow \mathbb{Z}$ que mapeia nomes de variáveis para os seus valores inteiros atuais.

Escrevemos:

- $\sigma(x)$ para o valor de x em σ (supondo $x \in \text{dom}(\sigma)$)
- $\sigma[x \mapsto n]$ para o ambiente obtido de σ atualizando (ou inserindo) a variável x com o valor n
- $\emptyset$ para o ambiente vazio (sem variáveis definidas)

Em Haskell, o ambiente é representado como `Map String Int` do módulo `Data.Map`.

13.4 Semântica Small-Step

13.4.1 Expressões

A avaliação de expressões não modifica o ambiente: expressões são **puras** no sentido de que apenas consultam σ , sem alterá-lo. Uma configuração de expressão é um par (σ, e) e a relação de redução é $(\sigma, e) \rightarrow (\sigma, e')$.

Variável:

$$\frac{x \in \text{dom}(\sigma)}{(\sigma, x) \rightarrow (\sigma, \sigma(x))} \{Var\}$$

Uma variável é substituída pelo seu valor corrente no ambiente. Se x não estiver definida em σ , a avaliação trava: erro de variável não inicializada.

Redução de subexpressões:

$$\frac{(\sigma, e_1) \rightarrow (\sigma, e'_1)}{(\sigma, e_1 + e_2) \rightarrow (\sigma, e'_1 + e_2)} \{AddL\} \quad \frac{(\sigma, e_2) \rightarrow (\sigma, e'_2)}{(\sigma, n_1 + e_2) \rightarrow (\sigma, n_1 + e'_2)} \{AddR\}$$

$$\frac{}{(\sigma, n_1 + n_2) \rightarrow (\sigma, n_1 + n_2)} \{Add\}$$

As regras para multiplicação são análogas: (MulL), (MulR) e (Mul).

13.4.2 Comandos

A configuração de um comando inclui o ambiente, pois os comandos podem modificá-lo. Usamos a notação $(\sigma, s) \rightarrow (\sigma', s')$, onde s' é o comando residual após a redução. Quando um comando termina sua execução, dizemos que ele reduz para **skip**, um comando que não faz nada.

Atribuição: avalia a expressão primeiro:

$$\frac{(\sigma, e) \rightarrow (\sigma, e')}{(\sigma, x := e) \rightarrow (\sigma, x := e')} \{AssignStep\}$$

Atribuição: expressão já é um valor:

$$\frac{}{(\sigma, x := n) \rightarrow (\sigma[x \mapsto n], \mathbf{skip})} \{Assign\}$$

Impressão: avalia a expressão primeiro:

$$\frac{(\sigma, e) \rightarrow (\sigma, e')}{(\sigma, \mathbf{print} \ e) \rightarrow (\sigma, \mathbf{print} \ e')} \{PrintStep\}$$

Impressão: expressão já é um valor:

$$\frac{}{(\sigma, \mathbf{print} \ n) \rightarrow (\sigma, \mathbf{skip})} \{Print\} \quad [\text{emite } n \text{ na saída}]$$

Sequência:

$$\frac{}{(\sigma, \mathbf{skip}; s^*) \rightarrow (\sigma, s^*)} \{SeqSkip\}$$

$$\frac{(\sigma, s) \rightarrow (\sigma', s')}{(\sigma, s; s^*) \rightarrow (\sigma', s'; s^*)} \{Seq\}$$

13.4.3 Exemplo: Small-Step

Considere o programa:

```
x := 2 + 3
print x * 4
```

Iniciamos com o ambiente vazio $\emptyset$. A execução procede assim:

$$\begin{aligned} & (\emptyset, x := 2 + 3; \mathbf{print} \ x * 4) \\ & \xrightarrow{\text{Seq, Add}} ([x \mapsto 5], \mathbf{skip}; \mathbf{print} \ x * 4) \\ & \xrightarrow{\text{SeqSkip}} ([x \mapsto 5], \mathbf{print} \ x * 4) \\ & \xrightarrow{\text{PrintStep, Var}} ([x \mapsto 5], \mathbf{print} \ 5 * 4) \\ & \xrightarrow{\text{PrintStep, Mul}} ([x \mapsto 5], \mathbf{print} \ 20) \\ & \xrightarrow{\text{Print}} ([x \mapsto 5], \mathbf{skip}) \quad [\text{emite } 20] \end{aligned}$$

Cada passo é justificado por uma regra; a sequência termina quando restam apenas **skips**.

13.5 Semântica Big-Step

Na semântica big-step, abstraímos os passos intermediários e definimos diretamente a relação entre configuração inicial e resultado final.

13.5.1 Expressões

A relação $\sigma \vdash e \Downarrow n$ significa “no ambiente σ , a expressão e avalia para o inteiro n ”.

Número:

$$\frac{}{\sigma \vdash n \Downarrow n} \{Num\}$$

Variável:

$$\frac{x \in \text{dom}(\sigma)}{\sigma \vdash x \Downarrow \sigma(x)} \{Var\}$$

Adição:

$$\frac{\sigma \vdash e_1 \Downarrow n_1 \quad \sigma \vdash e_2 \Downarrow n_2}{\sigma \vdash e_1 + e_2 \Downarrow n_1 \hat{+} n_2} \{Add\}$$

Multiplicação:

$$\frac{\sigma \vdash e_1 \Downarrow n_1 \quad \sigma \vdash e_2 \Downarrow n_2}{\sigma \vdash e_1 * e_2 \Downarrow n_1 \hat{*} n_2} \{Mul\}$$

13.5.2 Comandos

A relação $\sigma, s \Downarrow \sigma'$ significa “executar o comando s no ambiente σ produz o ambiente σ' ”. O ambiente é o único “resultado” de um comando; os efeitos de I/O (impressão, leitura) são considerados efeitos colaterais implícitos.

Atribuição:

$$\frac{\sigma \vdash e \Downarrow n}{\sigma, x := e \Downarrow \sigma[x \mapsto n]} \{Assign\}$$

Impressão:

$$\frac{\sigma \vdash e \Downarrow n}{\sigma, \text{print } e \Downarrow \sigma} \{Print\}$$

O ambiente não muda com **print**; apenas o valor de e é emitido como efeito colateral.

Leitura:

$$\frac{}{\sigma, \text{read } x \Downarrow \sigma[x \mapsto n]} \{Read\} \quad n \text{ lido da entrada}$$

A regra (Read) é um axioma cuja premissa implícita é a leitura de um inteiro n da entrada padrão.

Sequência vazia:

$$\frac{}{\sigma, \epsilon \Downarrow \sigma} \{SeqNil\}$$

Sequência não-vazia:

$$\frac{\sigma, s \Downarrow \sigma' \quad \sigma', s^* \Downarrow \sigma''}{\sigma, s; s^* \Downarrow \sigma''} \{SeqCons\}$$

13.5.3 Exemplos: Big-Step

Exemplo 1: Derivação para $x := 2 + 3$ partindo de $\emptyset$.

$$\frac{\frac{\frac{\emptyset \vdash 2 \Downarrow 2}{\emptyset \vdash 2 + 3 \Downarrow 5}}{\emptyset, x := 2 + 3 \Downarrow [x \mapsto 5]}}$$

Exemplo 2: Derivação para o programa completo ($x := 2 + 3$; **print** $x * 4$) partindo de $\emptyset$.

$$\frac{\frac{\underbrace{\emptyset, x := 2 + 3 \Downarrow [x \mapsto 5]}_{\text{Exemplo 1}} \quad \frac{\frac{[x \mapsto 5] \vdash x \Downarrow 5 \quad [x \mapsto 5] \vdash 4 \Downarrow 4}{[x \mapsto 5] \vdash x * 4 \Downarrow 20}}{[x \mapsto 5], \text{print } x * 4 \Downarrow [x \mapsto 5]}}{\emptyset, x := 2 + 3; \text{print } x * 4 \Downarrow [x \mapsto 5]}}$$

A derivação confirma que, ao final, $x = 5$ e o valor 20 foi emitido.

13.6 Implementação em Haskell

13.6.1 O Ambiente como Mônada de Estado

A semântica big-step dita que cada comando recebe um ambiente e produz um (possivelmente diferente) ambiente. Em Haskell, isso é capturado pela **mônada de estado** (**StateT**):

```
type Env = Map String Int
type Interp a = ExceptT String (StateT Env IO) a
```

A pilha de mônadas combina três capacidades:

- **StateT Env**: o ambiente σ implícito, que pode ser lido com **get** e atualizado com **modify**;
- **ExceptT String**: tratamento de erros (como variável não definida) via **throwError**
- **IO**: efeitos de I/O para **read** e **print**.

O ponto de entrada executa a pilha a partir do ambiente vazio:

```
interp :: Line -> IO ()
interp p = do
  (r, _) <- runStateT (runExceptT (interpProgram p)) Map.empty
  case r of
    Left err -> putStrLn err
    Right _ -> pure ()
```

13.6.2 Interpretador de Expressões

A função **interpExp** implementa as regras (Num), (Var), (Add) e (Mul):

```
interpExp :: Exp -> Interp Int
interpExp (EInt n) = pure n
interpExp (EVar v) = askVar v
interpExp (e1 :+: e2) = (+) <$> interpExp e1 <*> interpExp e2
interpExp (e1 **: e2) = (*) <$> interpExp e1 <*> interpExp e2
```

A função auxiliar `askVar` implementa a regra (Var): consulta o ambiente corrente e lança um erro se a variável não estiver definida (caso não coberto pelas regras semânticas):

```
askVar :: String -> Interp Int
askVar s = do
  env <- get
  case Map.lookup s env of
    Nothing -> throwError $ unwords ["Undefined variable:", s]
    Just val -> pure val
```

13.6.3 Interpretador de Comandos

A função `interpStmt` implementa as regras (Assign), (Print) e (Read):

```
interpStmt :: Stmt -> Interp ()
interpStmt (SAssign v e) = do
  val <- interpExp e
  updateVar v val
interpStmt (SRead v) = do
  str <- liftIO getLine
  let val = read str
  updateVar v val
interpStmt (SPrint e) = do
  val <- interpExp e
  liftIO $ print val
```

A correspondência com as regras semânticas é direta:

- **SAssign** v e : avalia e (regra Assign, premissa) e atualiza o ambiente com `updateVar` (conclusão: $\sigma[x \mapsto n]$).
- **SPrint** e : avalia e e imprime o resultado; o ambiente não muda (regra Print).
- **SRead** v : lê um inteiro da entrada padrão e armazena em v (regra Read).

A função `updateVar` encapsula a operação $\sigma[x \mapsto n]$:

```
updateVar :: String -> Int -> Interp ()
updateVar s val = modify (Map.insert s val)
```

A interpretação de um programa aplica `interpStmt` a cada comando em sequência, correspondendo às regras (SeqNil) e (SeqCons):

```
interpProgram :: Line -> Interp ()
interpProgram (Line ss) = mapM_ interpStmt ss
```

`mapM_` sequencia os comandos na ordem e descarta os resultados unitários, o que equivale exatamente a encadear as regras (SeqCons) até atingir (SeqNil).

13.6.4 Executando o Interpretador

```
cabal run line -- --interp --file programa.line
```

Por exemplo, dado o arquivo `programa.line`:

```
x := 10
y := x * 2
print y
```

O interpretador produziria 20.

13.7 Conclusão

Neste capítulo, estendemos a semântica operacional para lidar com **estado**, introduzindo ambientes que mapeiam variáveis a valores. As contribuições principais foram:

- A **semântica small-step** para comandos usa configurações (σ, s) que transportam o ambiente ao longo dos passos de redução. A regra (Assign) modifica o ambiente na transição; as outras regras o propagam sem alteração.
- A **semântica big-step** relaciona um comando e um ambiente inicial ao ambiente final, abstraíndo os passos intermediários. A sua estrutura de “entrada $\rightarrow$ saída de ambiente” traduz-se diretamente na mônada de estado de Haskell.
- A **implementação** em LineInterp.hs usa a pilha `ExceptT String (StateT Env IO)`, que integra estado (ambiente), erros e I/O. Cada equação do interpretador corresponde a uma regra semântica.

O padrão estado-como-mônada que vimos aqui reaparecerá nos próximos capítulos quando estendermos a linguagem com estruturas de controlo como condicionais e laços.

Capítulo 14

Semântica operacional para uma linguagem imperativa simples

14.1 Introdução

No capítulo anterior, definimos a semântica operacional da linguagem *Line*, que inclui atribuição, leitura e impressão. Com isso, já conseguimos modelar programas que executam uma sequência fixa de comandos. No entanto, programas reais precisam de **controle de fluxo**: executar blocos diferentes dependendo de uma condição, e repetir um bloco enquanto uma condição permanecer verdadeira.

Este capítulo estende a linguagem com **condicionais** (if/else) e **repetição** (while), formando a linguagem *While*. Para suportar condições, a linguagem de expressões também é estendida com **valores booleanos** e operações de comparação. Definimos a semântica small-step e big-step para todas as novas construções e mostramos como essa especificação se traduz no interpretador disponível em

While/Interp/WhileInterp.hs

14.2 A Linguagem While

A linguagem *While* estende *Line* com dois novos comandos e uma classe de expressões booleanas. A sua sintaxe abstrata é:

$$\begin{aligned} prog &::= s^* \\ s &::= x := e \mid \text{read } x \mid \text{print } e \\ &\quad \mid \text{if } e \text{ then } \{s^*\} \text{ else } \{s^*\} \\ &\quad \mid \text{while } e \text{ do } \{s^*\} \\ e &::= v \mid x \mid e_1 + e_2 \mid e_1 * e_2 \\ &\quad \mid e_1 < e_2 \mid e_1 = e_2 \mid e_1 \mid e_2 \mid \neg e \\ v &::= n \mid \text{true} \mid \text{false} \end{aligned}$$

Os valores agora incluem inteiros e booleanos. Em Haskell:

```
data Value
= VInt Int
| VBool Bool

data Exp
= EValue Value
| EVar Var
| Exp :+: Exp -- adição
| Exp *: Exp -- multiplicação
| Exp <: Exp -- menor que
| Exp ==: Exp -- igualdade
```

```

| Exp :: Exp  -- disjunção
| ENot Exp    -- negação

data Stmt
= SAssign Var Exp
| SRead  Var
| SPrint Exp
| SIf    Exp Block Block
| SWhile Exp Block

type Block = [Stmt]

```

14.3 Ambientes

O ambiente $\sigma : Var \rightarrow Val$ agora associa variáveis a **valores** (inteiros ou booleanos). A notação $\sigma(x)$, $\sigma[x \mapsto v]$ e $\emptyset$ é a mesma do capítulo anterior.

14.4 Semântica Small-Step

14.4.1 Expressões

Os valores v são as formas terminais; qualquer expressão que não seja um valor pode ainda ser reduzida. As regras para adição e multiplicação são idênticas ao capítulo anterior. As novas regras tratam das operações booleanas.

Variável:

$$\frac{x \in \text{dom}(\sigma)}{(\sigma, x) \rightarrow (\sigma, \sigma(x))} \{Var\}$$

Redução de subexpressões:

Para cada operador binário $\oplus$, as regras (OpL) e (OpR) reduzem o operando esquerdo primeiro e depois o direito, preservando o ambiente:

$$\frac{(\sigma, e_1) \rightarrow (\sigma, e'_1)}{(\sigma, e_1 \oplus e_2) \rightarrow (\sigma, e'_1 \oplus e_2)} \{OpL\} \quad \frac{(\sigma, e_2) \rightarrow (\sigma, e'_2)}{(\sigma, v_1 \oplus e_2) \rightarrow (\sigma, v_1 \oplus e'_2)} \{OpR\}$$

Operações sobre valores:

$$\frac{}{(\sigma, n_1 + n_2) \rightarrow (\sigma, n_1 \hat{+} n_2)} \{Add\} \quad \frac{}{(\sigma, n_1 \star n_2) \rightarrow (\sigma, n_1 \hat{\star} n_2)} \{Mul\}$$

$$\frac{}{(\sigma, n_1 < n_2) \rightarrow (\sigma, n_1 \hat{<} n_2)} \{Lt\} \quad \frac{}{(\sigma, v_1 = v_2) \rightarrow (\sigma, v_1 \hat{=} v_2)} \{Eq\}$$

$$\frac{}{(\sigma, b_1 \mid b_2) \rightarrow (\sigma, b_1 \hat{\vee} b_2)} \{Or\} \quad \frac{}{(\sigma, \neg b) \rightarrow (\sigma, \hat{\neg} b)} \{Not\}$$

$$\frac{(\sigma, e) \rightarrow (\sigma, e')}{(\sigma, \neg e) \rightarrow (\sigma, \neg e')} \{NotStep\}$$

Os operadores com acento ($\hat{+}$, $\hat{<}$, $\hat{\vee}$, etc.) denotam as operações matemáticas correspondentes, para distingui-las dos operadores da linguagem.

14.4.2 Comandos

As regras para **SAssign**, **SRead**, **SPrint** e para sequências são as mesmas do capítulo anterior. As novas regras tratam de **if** e **while**.

Condicional: avaliação da condição

$$\frac{(\sigma, e) \rightarrow (\sigma, e')}{(\sigma, \text{if } e \text{ then } B_1 \text{ else } B_2) \rightarrow (\sigma, \text{if } e' \text{ then } B_1 \text{ else } B_2)} \{IfStep\}$$

Condicional: ramo verdadeiro

$$\frac{}{(\sigma, \text{if true then } B_1 \text{ else } B_2) \rightarrow (\sigma, B_1)} \{IfTrue\}$$

Condicional: ramo falso

$$\frac{}{(\sigma, \text{if false then } B_1 \text{ else } B_2) \rightarrow (\sigma, B_2)} \{IfFalse\}$$

While: iteração

A regra chave para while é a de **iteração condicional**: o laço se transforma em um condicional que, no ramo verdadeiro, executa o corpo e depois o laço novamente.

$$\frac{}{(\sigma, \text{while } e \text{ do } B) \rightarrow (\sigma, \text{if } e \text{ then } (B; \text{while } e \text{ do } B) \text{ else } \epsilon)} \{While\}$$

onde ϵ denota a sequência vazia. Esta regra captura a semântica de repetição de forma simples: o while se reduz a um condicional cujo ramo verdadeiro contém o corpo seguido do próprio while.

14.4.3 Exemplo: Small-Step

Considere o programa:

```
i := 0
while i < 3 do { i := i + 1 }
print i
```

Partindo de $\emptyset$, após a atribuição inicial temos $\sigma_0 = [i \mapsto 0]$. Mostramos os passos de uma iteração do laço:

$$\begin{aligned} & (\sigma_0, \text{while } i < 3 \text{ do } \{i := i + 1\}) \\ & \xrightarrow{\text{While}} (\sigma_0, \text{if } i < 3 \text{ then } \{i := i + 1; \text{while } \dots\} \text{ else } \epsilon) \\ & \xrightarrow{\text{IfStep, Var, Lt}} (\sigma_0, \text{if true then } \{i := i + 1; \text{while } \dots\} \text{ else } \epsilon) \\ & \xrightarrow{\text{IfTrue}} (\sigma_0, i := i + 1; \text{while } i < 3 \text{ do } \{i := i + 1\}) \\ & \xrightarrow{\text{Assign}} ([i \mapsto 1], \text{while } i < 3 \text{ do } \{i := i + 1\}) \end{aligned}$$

Após três iterações, $\sigma = [i \mapsto 3]$. Na quarta, a condição $3 < 3$ reduz para **false** e a regra (IfFalse) seleciona ϵ , terminando o laço.

14.4.4 Extensão TImp: Semântica Small-Step

A linguagem *TImp* estende *While* com construções de registro e funções. As regras small-step para registros seguem o mesmo padrão das construções já apresentadas; a semântica completa de chamadas de função é especificada na forma big-step na Seção 14.5.4, pois o comando **return** exigiria um modelo de pilha de chamadas para ser tratado em small-step.

As configurações passam a incluir o ambiente de funções Φ : (σ, Φ, e) para expressões e (σ, Φ, s) para comandos.

Construção de registro. Os campos são avaliados da esquerda para a direita. Enquanto houver um campo não-valor, ele é reduzido (*NewStep*); quando todos os campos são valores, o registro é construído (*New*):

$$\frac{(\sigma, \Phi, e_i) \rightarrow (\sigma', \Phi, e'_i)}{(\sigma, \Phi, \text{new } R \{f_1 = v_1, \dots, f_i = e_i, \dots\}) \rightarrow (\sigma', \Phi, \text{new } R \{f_1 = v_1, \dots, f_i = e'_i, \dots\})} \{NewStep\}$$

$$\frac{}{(\sigma, \Phi, \text{new } R \{f_1 = v_1, \dots, f_n = v_n\}) \rightarrow (\sigma, \Phi, \text{rec}(R, \{f_1 \mapsto v_1, \dots, f_n \mapsto v_n\}))} \{New\}$$

Acesso a campo.

$$\frac{(\sigma, \Phi, e) \rightarrow (\sigma', \Phi, e')}{(\sigma, \Phi, e.f) \rightarrow (\sigma', \Phi, e'.f)} \{FieldStep\}$$

$$\frac{f \in \text{dom}(flds)}{(\sigma, \Phi, \text{rec}(R, flds).f) \rightarrow (\sigma, \Phi, flds(f))} \{Field\}$$

Declaração de variável.

$$\frac{(\sigma, \Phi, e) \rightarrow (\sigma', \Phi, e')}{(\sigma, \Phi, \text{var } x : \tau = e) \rightarrow (\sigma', \Phi, \text{var } x : \tau = e')} \{DeclStep\}$$

$$\frac{}{(\sigma, \Phi, \text{var } x : \tau = v) \rightarrow (\sigma[x \mapsto v], \Phi, \epsilon)} \{Decl\}$$

Atribuição de campo.

$$\frac{(\sigma, \Phi, e) \rightarrow (\sigma', \Phi, e')}{(\sigma, \Phi, x.f := e) \rightarrow (\sigma', \Phi, x.f := e')} \{FieldAssignStep\}$$

$$\frac{\sigma(x) = \text{rec}(R, flds)}{(\sigma, \Phi, x.f := v) \rightarrow (\sigma[x \mapsto \text{rec}(R, flds[f \mapsto v])], \Phi, \epsilon)} \{FieldAssign\}$$

Avaliação de argumentos. Os argumentos de uma chamada de função são avaliados da esquerda para a direita antes da invocação. A regra de invocação — que cria um ambiente de ativação fresco e executa o corpo — é especificada na forma big-step:

$$\frac{(\sigma, \Phi, e_i) \rightarrow (\sigma', \Phi, e'_i)}{(\sigma, \Phi, f(v_1, \dots, v_{i-1}, e_i, \dots, e_n)) \rightarrow (\sigma', \Phi, f(v_1, \dots, v_{i-1}, e'_i, \dots, e_n))} \{CallStep\}$$

14.5 Semântica Big-Step

14.5.1 Expressões

A relação $\sigma \vdash e \Downarrow v$ agora produz um **valor** (inteiro ou booleano).

$$\begin{array}{c}
 \frac{}{\sigma \vdash v \Downarrow v} \{Val\} \quad \frac{x \in \text{dom}(\sigma)}{\sigma \vdash x \Downarrow \sigma(x)} \{Var\} \\
 \\
 \frac{\sigma \vdash e_1 \Downarrow n_1 \quad \sigma \vdash e_2 \Downarrow n_2}{\sigma \vdash e_1 + e_2 \Downarrow n_1 \hat{+} n_2} \{Add\} \quad \frac{\sigma \vdash e_1 \Downarrow n_1 \quad \sigma \vdash e_2 \Downarrow n_2}{\sigma \vdash e_1 \star e_2 \Downarrow n_1 \hat{\star} n_2} \{Mul\} \\
 \\
 \frac{\sigma \vdash e_1 \Downarrow n_1 \quad \sigma \vdash e_2 \Downarrow n_2}{\sigma \vdash e_1 < e_2 \Downarrow n_1 \hat{<} n_2} \{Lt\} \quad \frac{\sigma \vdash e_1 \Downarrow v_1 \quad \sigma \vdash e_2 \Downarrow v_2}{\sigma \vdash e_1 = e_2 \Downarrow v_1 \hat{=} v_2} \{Eq\} \\
 \\
 \frac{\sigma \vdash e_1 \Downarrow b_1 \quad \sigma \vdash e_2 \Downarrow b_2}{\sigma \vdash e_1 \mid e_2 \Downarrow b_1 \hat{\vee} b_2} \{Or\} \quad \frac{\sigma \vdash e \Downarrow b}{\sigma \vdash \neg e \Downarrow \hat{\neg} b} \{Not\}
 \end{array}$$

14.5.2 Comandos

As regras para **SAssign**, **SRead**, **SPrint**, (SeqNil) e (SeqCons) permanecem iguais ao capítulo anterior. As novas regras são:

Condicional: ramo verdadeiro

$$\frac{\sigma \vdash e \Downarrow \mathbf{true} \quad \sigma, B_1 \Downarrow \sigma'}{\sigma, \text{if } e \text{ then } B_1 \text{ else } B_2 \Downarrow \sigma'} \{IfTrue\}$$

Condicional: ramo falso

$$\frac{\sigma \vdash e \Downarrow \mathbf{false} \quad \sigma, B_2 \Downarrow \sigma'}{\sigma, \text{if } e \text{ then } B_1 \text{ else } B_2 \Downarrow \sigma'} \{IfFalse\}$$

While: condição falsa (terminação)

$$\frac{\sigma \vdash e \Downarrow \mathbf{false}}{\sigma, \text{while } e \text{ do } B \Downarrow \sigma} \{WhileFalse\}$$

While: condição verdadeira (iteração)

$$\frac{\sigma \vdash e \Downarrow \mathbf{true} \quad \sigma, B \Downarrow \sigma' \quad \sigma', \text{while } e \text{ do } B \Downarrow \sigma''}{\sigma, \text{while } e \text{ do } B \Downarrow \sigma''} \{WhileTrue\}$$

A regra (WhileTrue) é recursiva: após executar o corpo B e obter σ' , o while é aplicado novamente a partir de σ' . A recursão termina quando a condição se torna **false**, coberta por (WhileFalse). Se a condição nunca se tornar falsa, não existe derivação big-step finita: o programa diverge.

14.5.3 Exemplos: Big-Step

Exemplo 1: Considere o seguinte programa:

```

if 2 < 3 then {
  x := 10;
} else {
  x := 20;
}

```

$$\frac{\frac{\frac{\emptyset \vdash 2 \Downarrow 2}{\emptyset \vdash 2 < 3 \Downarrow \mathbf{true}} \quad \frac{\emptyset \vdash 3 \Downarrow 3}{\emptyset \vdash 10 \Downarrow 10}}{\emptyset, x := 10 \Downarrow [x \mapsto 10]} \quad \frac{\emptyset \vdash 10 \Downarrow 10}{\emptyset, x := 10 \Downarrow [x \mapsto 10]} \quad \frac{\emptyset \vdash 2 < 3 \Downarrow \mathbf{true} \quad \emptyset, x := 10 \Downarrow [x \mapsto 10]}{\emptyset, \text{if } 2 < 3 \text{ then } \{x := 10\} \text{ else } \{x := 20\} \Downarrow [x \mapsto 10]}$$

Exemplo 2: Considere o programa:

```
while i < 2 do {
  i := i + 1;
}
```

e o estado inicial $\sigma_0 = [i \mapsto 0]$.

Abreviemos $\sigma_k = [i \mapsto k]$ e o programa anterior por W .

$$\frac{\sigma_0 \vdash i < 2 \Downarrow \mathbf{true} \quad \sigma_0, i := i + 1 \Downarrow \sigma_1 \quad \frac{\sigma_1 \vdash i < 2 \Downarrow \mathbf{true} \quad \sigma_1, i := i + 1 \Downarrow \sigma_2 \quad \frac{\sigma_2 \vdash i < 2 \Downarrow \mathbf{false}}{\sigma_2, W \Downarrow \sigma_2}}{\sigma_1, W \Downarrow \sigma_2}}{\sigma_0, W \Downarrow \sigma_2}$$

A derivação é uma árvore de profundidade proporcional ao número de iterações. O laço termina porque i cresce a cada iteração e eventualmente atinge 2.

14.5.4 Extensão TImp: Semântica Big-Step

A linguagem *TImp* estende *While* com declarações de funções e tipos registro. Nesta seção apresentamos os domínios semânticos e as regras big-step para as novas construções.

Sintaxe Estendida

Um programa TImp consiste em uma sequência de declarações seguida de um bloco de comandos de nível superior:

```
prog ::= decl* s*
decl ::= record R {  $\overline{f_i : \tau_i}$  }
      | fn  $f(\overline{x_i : \tau_i}) : \rho \{ s^* \}$ 
 $\rho$     ::= void |  $\tau$ 
 $\tau$     ::= int | bool | string | R
s      ::= ... | var  $x : \tau = e$  |  $x.f := e$  | return e | return |  $f(e_1, \dots, e_n)$ 
e      ::= ... | e.f | new R {  $\overline{f_i = e_i}$  } |  $f(e_1, \dots, e_n)$ 
```

Os reticências $\dots$ denotam as construções já presentes em *While*.

Domínios Semânticos Estendidos

Valores. O domínio de valores é estendido com valores registro e com o valor unitário (resultado de funções `void`):

$$v ::= n \mid b \mid s \mid \mathbf{rec}(R, \{f_1 \mapsto v_1, \dots, f_n \mapsto v_n\}) \mid \mathbf{unit}$$

Um valor registro $\mathbf{rec}(R, flds)$ carrega o nome do tipo R e uma função parcial $flds$ que mapeia nomes de campo a valores. A semântica adota **semântica de valor** para registros: toda atribuição e passagem de parâmetro copia o registro inteiro.

Ambientes de funções. Além do ambiente de variáveis σ , a semântica introduz um **ambiente de funções** global

$$\Phi : \text{Name} \rightarrow (\overline{x_i}, B)$$

que mapeia cada nome de função à sua lista de parâmetros formais $\overline{x_i}$ e ao seu corpo B . O ambiente Φ é construído uma única vez a partir das declarações do programa e permanece imutável durante a execução.

Resultado de bloco. As construções **return** interrompem a execução normal de um bloco e devolvem um valor ao chamador. Para modelar esse comportamento, a relação de avaliação de comandos e blocos passa a produzir um **resultado**:

$$r ::= \text{normal} \mid \uparrow v$$

O resultado **normal** indica terminação sem retorno; o resultado $\uparrow v$ indica que um **return** foi executado e o valor v está sendo propagado para o ponto de chamada. A relação de avaliação de comandos torna-se $\sigma, \Phi \vdash s \Downarrow (\sigma', r)$ e a de blocos, $\sigma, \Phi \vdash B \Downarrow (\sigma', r)$.

Semântica Big-Step: Registros

Expressões Construção de registro:

$$\frac{\sigma, \Phi \vdash e_1 \Downarrow v_1 \quad \dots \quad \sigma, \Phi \vdash e_n \Downarrow v_n}{\sigma, \Phi \vdash \text{new } R \{f_1 = e_1, \dots, f_n = e_n\} \Downarrow \text{rec}(R, \{f_1 \mapsto v_1, \dots, f_n \mapsto v_n\})} \{New\}$$

Cada campo é avaliado na ordem em que aparece na expressão.

Acesso a campo:

$$\frac{\sigma, \Phi \vdash e \Downarrow \text{rec}(R, flds) \quad f \in \text{dom}(flds)}{\sigma, \Phi \vdash e.f \Downarrow flds(f)} \{Field\}$$

O acesso avalia e para um valor registro e extrai o campo f . Se f não existir, a execução trava (erro de tipo prevenido pela verificação de tipos).

Comandos Declaração de variável:

$$\frac{\sigma, \Phi \vdash e \Downarrow v}{\sigma, \Phi \vdash \text{var } x : \tau = e \Downarrow (\sigma[x \mapsto v], \text{normal})} \{Decl\}$$

Atribuição de campo:

$$\frac{\sigma(x) = \text{rec}(R, flds) \quad \sigma, \Phi \vdash e \Downarrow v}{\sigma, \Phi \vdash x.f := e \Downarrow (\sigma[x \mapsto \text{rec}(R, flds[f \mapsto v])], \text{normal})} \{FieldAssign\}$$

A atribuição não aloca um novo registro: ela atualiza o registro em x substituindo o campo f pelo novo valor v . Como a semântica é de valor, x recebe uma cópia modificada.

Semântica Big-Step: Funções

Propagação de return Para que **return** interrompa a execução de um bloco, as regras de sequência precisam propagar o resultado.

Bloco vazio:

$$\frac{}{\sigma, \Phi \vdash \epsilon \Downarrow (\sigma, \mathbf{normal})} \{BlkNil\}$$

Bloco com retorno antecipado:

$$\frac{\sigma, \Phi \vdash s \Downarrow (\sigma', \uparrow v)}{\sigma, \Phi \vdash s; B \Downarrow (\sigma', \uparrow v)} \{BlkRet\}$$

Se o primeiro comando produz um retorno, o resto do bloco B não é executado e o resultado é propagado imediatamente.

Bloco com continuação normal:

$$\frac{\sigma, \Phi \vdash s \Downarrow (\sigma', \mathbf{normal}) \quad \sigma', \Phi \vdash B \Downarrow r}{\sigma, \Phi \vdash s; B \Downarrow r} \{BlkCons\}$$

Comandos de retorno Retorno com valor:

$$\frac{\sigma, \Phi \vdash e \Downarrow v}{\sigma, \Phi \vdash \mathbf{return} e \Downarrow (\sigma, \uparrow v)} \{RetVal\}$$

Retorno sem valor (void):

$$\frac{}{\sigma, \Phi \vdash \mathbf{return} \Downarrow (\sigma, \uparrow \mathbf{unit})} \{RetVoid\}$$

Chamada de função Chamada como expressão (função não-void):

$$\frac{\Phi(f) = (\overline{x_i}, B) \quad \sigma, \Phi \vdash e_i \Downarrow v_i \ (\forall i) \quad [x_1 \mapsto v_1, \dots, x_n \mapsto v_n], \Phi \vdash B \Downarrow (_, \uparrow v)}{\sigma, \Phi \vdash f(e_1, \dots, e_n) \Downarrow v} \{CallExpr\}$$

O corpo de f é executado em um **ambiente fresco** $[x_1 \mapsto v_1, \dots, x_n \mapsto v_n]$: apenas os parâmetros formais estão em escopo. O resultado $\uparrow v$ fornece o valor de retorno; o ambiente da chamada σ não é modificado.

Chamada como comando (função void ou retorno descartado):

$$\frac{\Phi(f) = (\overline{x_i}, B) \quad \sigma, \Phi \vdash e_i \Downarrow v_i \ (\forall i) \quad [x_1 \mapsto v_1, \dots, x_n \mapsto v_n], \Phi \vdash B \Downarrow (_, _)}{\sigma, \Phi \vdash f(e_1, \dots, e_n) \Downarrow (\sigma, \mathbf{normal})} \{CallStmt\}$$

O ambiente de chamada é devolvido inalterado: a função não pode alterar o estado externo através de variáveis globais mutáveis nem por passagem por referência.

14.6 Implementação em Haskell

14.6.1 Expressões

A função `interpExp` retorna agora um **Value** (inteiro ou booleano). As novas equações implementam diretamente as regras semânticas:

```
interpExp :: Exp -> Interp Value
interpExp (EValue v) = pure v
interpExp (EVar v) = askVar v
```



```

interpExp (e1 :+: e2) = do { v1 <- interpExp e1; v2 <- interpExp e2; v1 :+: v2 }
interpExp (e1 **: e2) = do { v1 <- interpExp e1; v2 <- interpExp e2; v1 **: v2 }
interpExp (e1 <: e2) = do { v1 <- interpExp e1; v2 <- interpExp e2; v1 <. v2 }
interpExp (e1 :=: e2) = do { v1 <- interpExp e1; v2 <- interpExp e2; v1 == v2 }
interpExp (e1 |: e2) = do { v1 <- interpExp e1; v2 <- interpExp e2; v1 |: v2 }
interpExp (ENot e) = do { v <- interpExp e; vnot v }

```

As funções `(.+:)`, `(.<.)`, `(.=.)`, `(.|.)` e `vnot`, definidas em `Utils.Value`, verificam que os operandos têm os tipos corretos e lançam um erro via `throwError` caso contrário.

14.6.2 Condicional

A equação para `SIf` implementa as regras (IfTrue) e (IfFalse):

```

interpStmt (SIf e blk1 blk2) = do
  v <- interpExp e
  case v of
    VBool True  -> mapM_ interpStmt blk1
    VBool False -> mapM_ interpStmt blk2
    _           -> throwError $ unlines
      ["Expecting a boolean value, but found:", pretty e]

```

O casamento de padrões em `VBool True` e `VBool False` corresponde diretamente às premissas das regras (IfTrue) e (IfFalse). O caso `_` trata erros de tipo que as regras semânticas simplesmente não cobrem.

14.6.3 Repetição

A equação para `SWhile` implementa as regras (WhileFalse) e (WhileTrue):

```

interpStmt (SWhile e blk1) = do
  v <- interpExp e
  case v of
    VBool False -> pure ()
    VBool True  -> do
      mapM_ interpStmt blk1
      interpStmt (SWhile e blk1)
    _ -> throwError $ unlines
      ["Expecting a boolean value, but found:", pretty e]

```

A correspondência com as regras semânticas é precisa:

- `VBool False -> pure ()`, correspondendo à regra (WhileFalse): o ambiente não muda, a execução termina sem produzir nenhum efeito adicional.
- `VBool True -> mapM_ interpStmt blk1 >> interpStmt (SWhile e blk1)`, correspondendo à regra (WhileTrue): executa o bloco $(\sigma, B \Downarrow \sigma')$ e depois invoca `interpStmt (SWhile e blk1)` recursivamente $(\sigma', \text{while} \dots \Downarrow \sigma'')$.

A recursão na implementação espelha a recursão na regra semântica: se o laço não terminar, a função Haskell também não terminará.

14.6.4 Executando o Interpretador

```
cabal run while -- --interp --file programa.while
```

Por exemplo, o seguinte programa calcula o fatorial de 5:

```
n := 5;
acc := 1;
while 0 < n do {
  acc := acc * n;
  n := n + (-1);
}
print acc;
```

O interpretador imprime 120.

14.6.5 Implementação da Extensão TImp

A implementação está em `TImp/Interp/TImpInterp.hs`. As regras semânticas apresentadas na Seção 14.5.4 têm correspondência direta com o código Haskell.

Domínio de valores e estado.

```
data Value
  = VInt    Int
  | VBool   Bool
  | VString String
  | VRecord Name (Map Field Value)  -- rec(R, flds)
  | VUnit
  -- unit

data InterpState = InterpState
  { varEnv  :: Map Var Value      -- sigma
  , funcEnv :: Map Name FuncDecl  -- Phi
  }
```

```
type InterpM a = StateT InterpState (ExceptT String IO) a
```

O campo `funcEnv` corresponde ao ambiente global Φ , construído uma vez no ponto de entrada e nunca modificado.

Resultado de bloco. A função `execBlock` retorna `Maybe Value`, onde `Nothing` corresponde a `normal` e `Just v` corresponde a $\uparrow v$:

```
execBlock :: [Stmt] -> InterpM (Maybe Value)
execBlock []      = pure Nothing          -- BlkNil
execBlock (s:ss) = do
  r <- execStmt s
  case r of
    Just _  -> pure r                    -- BlkRet: propagate
    Nothing -> execBlock ss              -- BlkCons: continue
```

Registros. A avaliação de `ENew` implementa a regra (New):

```
evalExp (ENew rname initFields) = do
  vals <- mapM (\(f, e) -> (f,) <$> evalExp e) initFields
  pure (VRecord rname (Map.fromList vals))
```

O acesso a campo implementa a regra (Field):

```
evalExp (EField e f) = do
  v <- evalExp e
  case v of
    VRecord _ fields ->
      case Map.lookup f fields of
        Just val -> pure val
        Nothing  -> throwError ("Record has no field: " ++ f)
    _ -> throwError "Field access on non-record value"
```

A atribuição de campo implementa (FieldAssign):

```
execStmt (SFieldAssign v f e) = do
  rval <- lookupVar v
  newVal <- evalExp e
  case rval of
    VRecord name fields ->
      let fields' = Map.insert f newVal fields
      in modify (\st -> st
        { varEnv = Map.insert v (VRecord name fields') (varEnv st) })
    _ -> throwError (v ++ " is not a record")
  pure Nothing
```

Funções. O comando `return` implementa as regras (RetVal) e (RetValVoid):

```
execStmt (SReturn Nothing) = pure (Just VUnit)
execStmt (SReturn (Just e)) = Just <$> evalExp e
```

A chamada de função implementa as regras (CallExpr) e (CallStmt). A função auxiliar `callFunc` cria um novo ambiente e executa o corpo:

```
callFunc :: Name -> [Value] -> InterpM Value
callFunc fname argVals = do
  fenv <- gets funcEnv
  case Map.lookup fname fenv of
    Nothing -> throwError ("Undefined function: " ++ fname)
    Just (FuncDecl _ params _ body) -> do
      let bindings = zip [v | Param v _ <- params] argVals
      withFreshEnv bindings $ do
        r <- execBlock body
        pure (fromMaybe VUnit r)
```

-- ambiente novo: sigma_0
-- extrai v de (uparrow v)

A função `withFreshEnv` salva o ambiente corrente, inicializa o novo ambiente com apenas os parâmetros formais e restaura o ambiente original ao terminar, implementando a semântica de escopo local:

```
withFreshEnv :: [(Var, Value)] -> InterpM a -> InterpM a
withFreshEnv bindings m = do
  saved <- gets varEnv
  modify (\st -> st { varEnv = Map.fromList bindings })
  r <- m
  modify (\st -> st { varEnv = saved })
  pure r
```

14.7 Conclusão

Neste capítulo, completámos a semântica operacional de uma linguagem imperativa com as construções de controle de fluxo mais fundamentais:

- O **condicional** é modelado por duas regras simétricas, (IfTrue) e (IfFalse), que selecionam o ramo a executar com base no valor booleano da condição.
- O **laço while** é modelado por (WhileFalse), que termina quando a condição é falsa, e (WhileTrue), que executa o corpo e recomeça: uma regra inerentemente recursiva que espelha a definição operacional de repetição.
- Na semântica small-step, o while se **desenrola** em um condicional, o que torna explícita a estrutura de cada iteração.
- Na semântica big-step, a não-terminação corresponde à ausência de uma derivação finita: a semântica é parcial por construção.
- **Registros** são modelados como valores estruturados $\text{rec}(R, fids)$ com semântica de cópia; as regras (New), (Field) e (FieldAssign) cobrem criação, leitura e atualização.
- **Funções** introduzem o conceito de *resultado de bloco* $r \in \{\text{normal}, \uparrow v\}$ para modelar **return**: as regras (BlkRet) e (BlkCons) propagam ou descartam esse resultado; (CallExpr) executa a função em um ambiente fresco e extrai o valor retornado.

A implementação em `TImp/Interp/TImpInterp.hs` segue a especificação semântica com precisão, confirmando que uma boa especificação formal é também um guia de implementação direta.

Parte V

Sistemas e verificação de tipos

Capítulo 15

Introdução aos sistemas de tipos

15.1 Introdução

Nos capítulos anteriores, definimos semânticas operacionais para linguagens de expressões e para linguagens imperativas. Vimos que a semântica de uma linguagem pode ser dada por regras de inferência que descrevem como os programas são executados. No entanto, a semântica por si só não impede que expressões sem sentido sejam escritas: como somar um booleano a um inteiro, ou executar `pred true`. Essas expressões são **sintaticamente válidas**, mas **semanticamente sem significado**.

É exatamente para isso que servem os **sistemas de tipos**: impor, de forma estática, que apenas expressões semanticamente bem-definidas possam ser executadas. Este capítulo apresenta as diferentes estratégias de tipagem utilizadas por linguagens de programação e discute os sistemas de tipos, utilizados para descrever formalmente regras para validação de tipos em linguagens de programação.

15.2 Tipos em Linguagens de Programação

Um **tipo** é uma classificação de valores que define:

1. Quais operações são aplicáveis a eles;
2. Como eles são representados na memória;
3. Que conjunto de valores o tipo compreende.

Por exemplo, o tipo **Int** classifica valores inteiros e admite operações aritméticas; o tipo **Bool** classifica os valores `true` e `false` e admite operações lógicas. O **sistema de tipos** de uma linguagem é o conjunto de regras que associa tipos a expressões e verifica se essas associações são coerentes.

15.2.1 Tipagem Estática e Dinâmica

A distinção mais fundamental em sistemas de tipos é *quando* a verificação de tipos ocorre:

Tipagem estática (*static typing*): a verificação é feita em **tempo de compilação**, antes da execução do programa. Se o programa contiver um erro de tipo, o compilador o rejeita. As linguagens Haskell, Java, C e Rust são exemplos que usam tipagem estática.

Tipagem dinâmica (*dynamic typing*): a verificação é feita em **tempo de execução**, à medida que o programa é executado. Erros de tipo são detectados apenas quando a operação inválida é executada. Exemplos de linguagens que usam essa estratégia são Python, JavaScript, Ruby e Lisp.

A tipagem estática oferece garantias mais fortes: o sistema de tipos pode certificar que certos tipos de erros *nunca* ocorrerão em nenhuma execução do programa. A tipagem dinâmica oferece mais flexibilidade, mas adia a detecção de erros para o momento da execução.

15.2.2 Tipagem Forte e Fraca

Uma distinção ortogonal é a de **tipagem forte** vs **tipagem fraca**:

Tipagem forte (*strong typing*): a linguagem não permite conversões implícitas entre tipos incompatíveis. Operações entre tipos diferentes resultam em erro (em tempo de compilação ou execução). Exemplos: Haskell, Python.

Tipagem fraca (*weak typing*): a linguagem realiza conversões implícitas entre tipos, frequentemente de maneiras surpreendentes. Exemplos: C (converte implicitamente entre inteiros e ponteiros), JavaScript ($1 + \text{"1"}$ resulta em "11").

Note que estas classificações são independentes: Python é dinamicamente tipado e fortemente tipado; C é estaticamente tipado e fracamente tipado.

15.3 A Linguagem de Expressões Tipadas

Para discutirmos sobre sistemas de tipos, vamos utilizar um linguagem de expressões simples. A sua **sintaxe abstrata** é representada pela seguinte gramática:

$t ::=$	true	(verdadeiro)
	false	(falso)
	if t_1 then t_2 else t_3	(condicional)
	0	(zero)
	succ t	(sucessor)
	pred t	(predecessor)
	iszero t	(teste de zero)

Os **tipos** são:

$T ::=$	Bool	(booleano)
	Nat	(número natural)

Os **valores** da linguagem são os termos que já não podem ser reduzidos pela semântica operacional. Distinguimos dois tipos de valores:

$v ::=$	true false nv
$nv ::=$	0 succ nv

onde nv denota os **valores numéricos**, isto é, **0** e qualquer número de aplicações de **succ** a **0**.

15.4 O Sistema de Tipos

Um **sistema de tipos** é definido por uma **relação de tipagem** entre termos e tipos. Escrevemos $\vdash t : T$ para dizer que “o termo t tem tipo T ”. A relação é definida por um conjunto de regras de inferência.

15.4.1 Regras de Tipagem

Constantes booleanas:

$\frac{}{\vdash \text{true} : \text{Bool}}$	(T-True)	$\frac{}{\vdash \text{false} : \text{Bool}}$	(T-False)
---	----------	--	-----------

Condicional:

$\frac{\vdash t_1 : \text{Bool} \quad \vdash t_2 : T \quad \vdash t_3 : T}{\vdash \text{if } t_1 \text{ then } t_2 \text{ else } t_3 : T}$	(T-If)
--	--------

A regra (T-If) exige que a condição t_1 seja booleana e que os dois ramos t_2 e t_3 tenham o **mesmo tipo** T . O resultado do condicional tem esse tipo.

Naturais:

$$\begin{array}{c}
 \frac{}{\vdash 0 : \mathbf{Nat}} \quad (\text{T-Zero}) \\
 \\
 \frac{\vdash t_1 : \mathbf{Nat}}{\vdash \text{succ } t_1 : \mathbf{Nat}} \quad (\text{T-Succ}) \qquad \frac{\vdash t_1 : \mathbf{Nat}}{\vdash \text{pred } t_1 : \mathbf{Nat}} \quad (\text{T-Pred}) \\
 \\
 \frac{\vdash t_1 : \mathbf{Nat}}{\vdash \text{iszero } t_1 : \mathbf{Bool}} \quad (\text{T-IsZero})
 \end{array}$$

15.4.2 Exemplos de Derivações de Tipo

Exemplo 1: O termo `iszero (succ 0)` tem tipo **Bool**:

$$\frac{\frac{\frac{}{\vdash 0 : \mathbf{Nat}}}{\vdash \text{succ } 0 : \mathbf{Nat}}}{\vdash \text{iszero (succ 0) : Bool}}$$

Exemplo 2: O condicional `if (iszero 0) then 0 else succ 0` tem tipo **Nat**:

$$\frac{\frac{\frac{}{\vdash 0 : \mathbf{Nat}}}{\vdash \text{iszero } 0 : \mathbf{Bool}} \quad \frac{}{\vdash 0 : \mathbf{Nat}} \quad \frac{}{\vdash \text{succ } 0 : \mathbf{Nat}}}{\vdash \text{if (iszero 0) then 0 else succ 0 : Nat}}$$

Exemplo 3 (erro de tipo): O termo `succ true` não é tipável: a regra (T-Succ) exige que o argumento tenha tipo **Nat**, mas `true` tem tipo **Bool**. Não existe derivação válida para este termo.

15.4.3 Unicidade de Tipos

Uma propriedade importante desta linguagem é a **unicidade de tipos**: cada termo bem tipado tem **exatamente um tipo**. Formalmente:

Lema (Unicidade): Se $\vdash t : T$ e $\vdash t : T'$, então $T = T'$.

Prova: Por indução estrutural no termo t .

- *Caso* $t = \text{true}$: a única regra aplicável é (T-True), que conclui $T = \mathbf{Bool}$. Logo $T = T' = \mathbf{Bool}$.
- *Caso* $t = \text{false}$: análogo.
- *Caso* $t = 0$: a única regra aplicável é (T-Zero), que conclui $T = \mathbf{Nat}$. Logo $T = T' = \mathbf{Nat}$.
- *Caso* $t = \text{succ } t_1$: a única regra aplicável é (T-Succ). Ambas as derivações de T e de T' devem usar (T-Succ), portanto $T = T' = \mathbf{Nat}$.
- *Caso* $t = \text{pred } t_1$: análogo; a única regra é (T-Pred), logo $T = T' = \mathbf{Nat}$.
- *Caso* $t = \text{iszero } t_1$: a única regra é (T-IsZero), logo $T = T' = \mathbf{Bool}$.
- *Caso* $t = \text{if } t_1 \text{ then } t_2 \text{ else } t_3$: a única regra aplicável é (T-If). Pela hipótese de indução aplicada a t_2 , o tipo de t_2 é único; como ambas as derivações concluem T e T' com base no tipo de t_2 , temos $T = T'$.

Como a prova cobre todos os casos da gramática, o lema está demonstrado. A consequência prática é que a tipagem é **determinística**: inspecionando um termo, é possível determinar seu tipo de forma única, sem ambiguidade.

15.5 Semântica Small-Step

A semântica small-step define a execução como uma sequência de reduções $t \rightarrow t'$. Os termos que não podem ser reduzidos são os **valores** v e as **formas presas** (*stuck terms*), termos sem mais reduções disponíveis que não são valores, como **succ true**.

15.5.1 Regras de Redução para Booleanos

Condicional (redução da condição):

$$\frac{t_1 \rightarrow t'_1}{\text{if } t_1 \text{ then } t_2 \text{ else } t_3 \rightarrow \text{if } t'_1 \text{ then } t_2 \text{ else } t_3} \quad (\text{E-If})$$

Condicional (ramos):

$$\frac{}{\text{if true then } t_2 \text{ else } t_3 \rightarrow t_2} \quad (\text{E-IfTrue}) \qquad \frac{}{\text{if false then } t_2 \text{ else } t_3 \rightarrow t_3} \quad (\text{E-IfFalse})$$

15.5.2 Regras de Redução para Naturais

Sucessor:

$$\frac{t_1 \rightarrow t'_1}{\text{succ } t_1 \rightarrow \text{succ } t'_1} \quad (\text{E-Succ})$$

Predecessor:

$$\frac{t_1 \rightarrow t'_1}{\text{pred } t_1 \rightarrow \text{pred } t'_1} \quad (\text{E-Pred}) \qquad \frac{}{\text{pred } 0 \rightarrow 0} \quad (\text{E-PredZero}) \qquad \frac{}{\text{pred } (\text{succ } nv_1) \rightarrow nv_1} \quad (\text{E-PredSucc})$$

Teste de zero:

$$\frac{t_1 \rightarrow t'_1}{\text{iszero } t_1 \rightarrow \text{iszero } t'_1} \quad (\text{E-IsZero}) \qquad \frac{}{\text{iszero } 0 \rightarrow \text{true}} \quad (\text{E-IsZeroZero}) \qquad \frac{}{\text{iszero } (\text{succ } nv_1) \rightarrow \text{false}} \quad (\text{E-IsZeroSucc})$$

15.5.3 Formas Presas

Uma **forma presa** (*stuck term*) é um termo que não é um valor e para o qual nenhuma regra de redução se aplica. Por exemplo:

- **succ true**: não é um valor (pois **true** não é um nv) e nenhuma regra se aplica;
- **pred false**: idem;
- **if 0 then t_2 else t_3 : 0** é um valor numérico, não booleano.

Formas presas representam **erros em tempo de execução**. O papel do sistema de tipos é garantir que termos bem tipados nunca se tornem formas presas.

15.5.4 Exemplo: Small-Step

Considere o termo $t = \text{pred}(\text{succ}(\text{succ } 0))$:

$$\text{pred}(\text{succ}(\text{succ } 0)) \xrightarrow{\text{E-PredSucc}} \text{succ } 0$$

O resultado **succ 0** é um valor numérico (nv).

Considere agora:

$$\text{if}(\text{iszero } 0) \text{ then } (\text{succ } 0) \text{ else } 0$$

$$\xrightarrow{\text{E-If, E-IsZeroZero}} \text{if true then } (\text{succ } 0) \text{ else } 0$$

$$\xrightarrow{\text{E-IfTrue}} \text{succ } 0$$

15.6 Semântica Big-Step

A semântica big-step define a relação $t \Downarrow v$, lida como “o termo t avalia para o valor v ”.

15.6.1 Regras para Booleanos

$$\begin{array}{c} \frac{}{\text{true} \Downarrow \text{true}} \quad (\text{B-True}) \qquad \frac{}{\text{false} \Downarrow \text{false}} \quad (\text{B-False}) \\[10pt] \frac{t_1 \Downarrow \text{true} \quad t_2 \Downarrow v}{\text{if } t_1 \text{ then } t_2 \text{ else } t_3 \Downarrow v} \quad (\text{B-IfTrue}) \qquad \frac{t_1 \Downarrow \text{false} \quad t_3 \Downarrow v}{\text{if } t_1 \text{ then } t_2 \text{ else } t_3 \Downarrow v} \quad (\text{B-IfFalse}) \end{array}$$

15.6.2 Regras para Naturais

$$\begin{array}{c} \frac{}{0 \Downarrow 0} \quad (\text{B-Zero}) \qquad \frac{t_1 \Downarrow nv_1}{\text{succ } t_1 \Downarrow \text{succ } nv_1} \quad (\text{B-Succ}) \\[10pt] \frac{t_1 \Downarrow 0}{\text{pred } t_1 \Downarrow 0} \quad (\text{B-PredZero}) \qquad \frac{t_1 \Downarrow \text{succ } nv_1}{\text{pred } t_1 \Downarrow nv_1} \quad (\text{B-PredSucc}) \\[10pt] \frac{t_1 \Downarrow 0}{\text{iszero } t_1 \Downarrow \text{true}} \quad (\text{B-IsZeroZero}) \qquad \frac{t_1 \Downarrow \text{succ } nv_1}{\text{iszero } t_1 \Downarrow \text{false}} \quad (\text{B-IsZeroSucc}) \end{array}$$

15.6.3 Exemplo: Big-Step

Exemplo: $\text{if}(\text{iszero } 0) \text{ then } \text{succ } 0 \text{ else } 0 \Downarrow \text{succ } 0$

$$\frac{\frac{0 \Downarrow 0}{\text{iszero } 0 \Downarrow \text{true}} \quad \frac{0 \Downarrow 0}{\text{succ } 0 \Downarrow \text{succ } 0}}{\text{if}(\text{iszero } 0) \text{ then } \text{succ } 0 \text{ else } 0 \Downarrow \text{succ } 0}$$

15.7 Corretude do Sistema de Tipos

A propriedade de **corretude** (*type soundness*) estabelece que o sistema de tipos é uma garantia válida: termos bem tipados nunca se tornam formas presas. Em outras palavras, um programa que passa na verificação de tipos **sempre progride** até um valor, sem travar em uma operação sem sentido.

Formalmente, a corretude é enunciada como:

Teorema (Type Soundness): Se $\vdash t : T$ e $t \rightarrow^* t'$, então t' é um valor ou existe t'' tal que $t' \rightarrow t''$.

Em português: *programas bem tipados não ficam presos*. A prova deste teorema se apoia em dois lemas: o de preservação e o de progresso.

15.7.1 Lema de Preservação

Lema de Preservação (Preservation): Se $\vdash t : T$ e $t \rightarrow t'$, então $\vdash t' : T$.

Este lema afirma que **a tipagem é invariante sob redução**: se um termo tem tipo T e dá um passo de execução, o termo resultante ainda tem o mesmo tipo T . O tipo é *preservado* ao longo de toda a computação.

Por que é importante? Sem a preservação, um termo poderia começar com tipo correto e, após algumas reduções, tornar-se um termo de tipo diferente, o que quebraria qualquer garantia que o verificador de tipos tentasse fornecer. A preservação garante que a “etiqueta de tipo” permanece coerente em cada passo.

Prova: Por indução na derivação de $t \rightarrow t'$, analisando cada regra de redução.

Caso (E-IfTrue): $t = \text{if true then } t_2 \text{ else } t_3$ e $t' = t_2$. Como $\vdash t : T$ foi derivado por (T-If), temos $\vdash \text{true} : \text{Bool}$, $\vdash t_2 : T$ e $\vdash t_3 : T$. Logo $\vdash t' = t_2 : T$.

Caso (E-IfFalse): $t = \text{if false then } t_2 \text{ else } t_3$ e $t' = t_3$. Analogamente, de (T-If) obtemos $\vdash t_3 : T$, portanto $\vdash t' : T$.

Caso (E-If): $t = \text{if } t_1 \text{ then } t_2 \text{ else } t_3$ com $t_1 \rightarrow t'_1$ e $t' = \text{if } t'_1 \text{ then } t_2 \text{ else } t_3$. De (T-If) temos $\vdash t_1 : \text{Bool}$, $\vdash t_2 : T$ e $\vdash t_3 : T$. Pela hipótese de indução aplicada a $t_1 \rightarrow t'_1$, obtemos $\vdash t'_1 : \text{Bool}$. Aplicando (T-If), concluímos $\vdash t' : T$.

Caso (E-Succ): $t = \text{succ } t_1$ com $t_1 \rightarrow t'_1$ e $t' = \text{succ } t'_1$. De (T-Succ) temos $\vdash t_1 : \text{Nat}$. Pela hipótese de indução, $\vdash t'_1 : \text{Nat}$. Aplicando (T-Succ), concluímos $\vdash \text{succ } t'_1 : \text{Nat}$.

Caso (E-Pred): $t = \text{pred } t_1$ com $t_1 \rightarrow t'_1$ e $t' = \text{pred } t'_1$. De (T-Pred) temos $\vdash t_1 : \text{Nat}$. Pela hipótese de indução, $\vdash t'_1 : \text{Nat}$. Aplicando (T-Pred), concluímos $\vdash \text{pred } t'_1 : \text{Nat}$.

Caso (E-PredZero): $t = \text{pred } 0$ e $t' = 0$. De (T-Pred) e (T-Zero) temos $\vdash 0 : \text{Nat}$, logo $\vdash t' : \text{Nat}$.

Caso (E-PredSucc): $t = \text{pred } (\text{succ } nv_1)$ e $t' = nv_1$. De (T-Pred) temos $\vdash \text{succ } nv_1 : \text{Nat}$; de (T-Succ) obtemos $\vdash nv_1 : \text{Nat}$. Logo $\vdash t' : \text{Nat}$.

Caso (E-IsZero): $t = \text{iszero } t_1$ com $t_1 \rightarrow t'_1$ e $t' = \text{iszero } t'_1$. De (T-IsZero) temos $\vdash t_1 : \text{Nat}$. Pela hipótese de indução, $\vdash t'_1 : \text{Nat}$. Aplicando (T-IsZero), concluímos $\vdash \text{iszero } t'_1 : \text{Bool}$.

Caso (E-IsZeroZero): $t = \text{iszero } 0$ e $t' = \text{true}$. Por (T-True), $\vdash \text{true} : \text{Bool}$; e de (T-IsZero) o tipo de t é Bool . Logo $\vdash t' : \text{Bool}$.

Caso (E-IsZeroSucc): $t = \text{iszero } (\text{succ } nv_1)$ e $t' = \text{false}$. Por (T-False), $\vdash \text{false} : \text{Bool}$; e de (T-IsZero) o tipo de t é Bool . Logo $\vdash t' : \text{Bool}$.

Como todos os casos da relação de redução estão cobertos, o lema está demonstrado.

15.7.2 Lema de Progresso

Lema de Progresso (*Progress*): Se $\vdash t : T$, então t é um valor ou existe t' tal que $t \rightarrow t'$.

Este lema afirma que **termos bem tipados nunca ficam presos**: a cada passo de execução, ou o termo já é um valor (a computação terminou), ou existe pelo menos uma redução possível (a computação pode continuar).

Por que é importante? Um termo preso é um programa em um estado de erro irrecuperável, como tentar aplicar **pred** a um booleano. O lema de progresso garante que isso nunca acontece para termos bem tipados: o sistema de tipos elimina exatamente os termos que poderiam ficar presos.

Prova: Por indução estrutural na derivação de $\vdash t : T$.

Caso (T-True): $t = \mathbf{true}$ é um valor.

Caso (T-False): $t = \mathbf{false}$ é um valor.

Caso (T-Zero): $t = \mathbf{0}$ é um valor numérico, portanto um valor.

Caso (T-If): $t = \mathbf{if } t_1 \mathbf{ then } t_2 \mathbf{ else } t_3$ com $\vdash t_1 : \mathbf{Bool}$. Pela hipótese de indução aplicada a t_1 : ou t_1 é valor, ou existe t'_1 tal que $t_1 \rightarrow t'_1$.

- Se $t_1 \rightarrow t'_1$: aplica-se (E-If), obtendo $t \rightarrow \mathbf{if } t'_1 \mathbf{ then } t_2 \mathbf{ else } t_3$.
- Se t_1 é valor de tipo **Bool**: pelo lema de unicidade de tipos, t_1 não pode ser um valor numérico nv (que tem tipo **Nat**). Portanto $t_1 \in \{\mathbf{true}, \mathbf{false}\}$.
 - Se $t_1 = \mathbf{true}$: aplica-se (E-IfTrue), $t \rightarrow t_2$.
 - Se $t_1 = \mathbf{false}$: aplica-se (E-IfFalse), $t \rightarrow t_3$.

Caso (T-Succ): $t = \mathbf{succ } t_1$ com $\vdash t_1 : \mathbf{Nat}$. Pela hipótese de indução aplicada a t_1 : ou t_1 é valor, ou reduz.

- Se $t_1 \rightarrow t'_1$: aplica-se (E-Succ), $t \rightarrow \mathbf{succ } t'_1$.
- Se t_1 é valor de tipo **Nat**: pelo lema de unicidade, t_1 não pode ser **true** nem **false** (que têm tipo **Bool**), portanto t_1 é um valor numérico nv . Logo $t = \mathbf{succ } nv$ é também um valor numérico.

Caso (T-Pred): $t = \mathbf{pred } t_1$ com $\vdash t_1 : \mathbf{Nat}$. Pela hipótese de indução aplicada a t_1 : ou t_1 é valor, ou reduz.

- Se $t_1 \rightarrow t'_1$: aplica-se (E-Pred), $t \rightarrow \mathbf{pred } t'_1$.
- Se t_1 é valor de tipo **Nat**: pelo lema de unicidade, t_1 é um valor numérico nv . Há dois subcasos:
 - $nv = \mathbf{0}$: aplica-se (E-PredZero), $t \rightarrow \mathbf{0}$.
 - $nv = \mathbf{succ } nv_1$: aplica-se (E-PredSucc), $t \rightarrow nv_1$.

Caso (T-IsZero): $t = \mathbf{iszero } t_1$ com $\vdash t_1 : \mathbf{Nat}$. Pela hipótese de indução aplicada a t_1 : ou t_1 é valor, ou reduz.

- Se $t_1 \rightarrow t'_1$: aplica-se (E-IsZero), $t \rightarrow \mathbf{iszero } t'_1$.
- Se t_1 é valor de tipo **Nat**: pelo lema de unicidade, t_1 é um valor numérico nv . Há dois subcasos:
 - $nv = \mathbf{0}$: aplica-se (E-IsZeroZero), $t \rightarrow \mathbf{true}$.
 - $nv = \mathbf{succ } nv_1$: aplica-se (E-IsZeroSucc), $t \rightarrow \mathbf{false}$.

Como todos os casos da derivação de tipagem estão cobertos, o lema está demonstrado.

15.7.3 Da Preservação e do Progresso à Corretude

Com os dois lemas, a prova de corretude é imediata:

Prova do Teorema (Type Soundness): Seja $\vdash t : T$ e $t \rightarrow^* t'$. Queremos mostrar que t' é valor ou pode reduzir.

Por **preservação**, como $\vdash t : T$ e $t \rightarrow^* t'$, temos $\vdash t' : T$ (aplicando o lema de preservação em cada passo da sequência de reduções, por indução no comprimento da sequência).

Por **progresso**, como $\vdash t' : T$, t' é valor ou existe t'' tal que $t' \rightarrow t''$.

A contribuição de cada lema é distinta e indispensável:

Lema	Garante	Sem ele, poderia acontecer
Preservação	O tipo se mantém durante a execução	Um passo de redução muda o tipo e o progresso não se aplica ao resultado
Progresso	Termos bem tipados não ficam presos	O tipo se mantém, mas a execução trava mesmo assim

A vantagem desta abordagem é que ela **decompõe a corretude** em dois lemas independentes e mais simples, cada um demonstrável por indução. A prova de progresso usa *apenas* as regras de tipagem; a prova de preservação usa *apenas* as regras de tipagem e as de redução.

15.8 Sistema de Tipos para TLine

TLine é uma linguagem imperativa simples com declaração de variáveis tipadas, atribuição, leitura e impressão. Os seus tipos são **Int**, **Bool** e **String**. A principal novidade em relação a TExp é a presença de **variáveis**, o que torna necessário um **contexto de tipagem**:

$$\Gamma : Var \rightarrow Ty$$

15.8.1 Regras de Tipagem para Expressões

As regras têm a forma $\Gamma \vdash e : T$:

$$\begin{array}{c}
\frac{}{\Gamma \vdash n : \mathbf{Int}} \quad (\text{T-Int}) \qquad \frac{}{\Gamma \vdash b : \mathbf{Bool}} \quad (\text{T-Bool}) \qquad \frac{}{\Gamma \vdash s : \mathbf{String}} \quad (\text{T-String}) \\
\\
\frac{x \in \text{dom}(\Gamma)}{\Gamma \vdash x : \Gamma(x)} \quad (\text{T-Var}) \\
\\
\frac{\Gamma \vdash e_1 : \mathbf{Int} \quad \Gamma \vdash e_2 : \mathbf{Int}}{\Gamma \vdash e_1 \oplus e_2 : \mathbf{Int}} \quad (\text{T-ArithOp}) \qquad \frac{\Gamma \vdash e_1 : \mathbf{Int} \quad \Gamma \vdash e_2 : \mathbf{Int}}{\Gamma \vdash e_1 \otimes e_2 : \mathbf{Bool}} \quad (\text{T-CmpOp}) \\
\\
\frac{\Gamma \vdash e : \mathbf{Bool}}{\Gamma \vdash \neg e : \mathbf{Bool}} \quad (\text{T-Not})
\end{array}$$

onde $\oplus \in \{+, -, *, /\}$ e $\otimes \in \{<, =\}$.

15.8.2 Regras de Tipagem para Comandos

As regras têm a forma $\Gamma \vdash s \dashv \Gamma'$, onde Γ' é o contexto produzido após verificar o comando s :

$$\frac{\Gamma \vdash e : T}{\Gamma \vdash \mathbf{var} \ x : T = e \dashv \Gamma[x \mapsto T]} \quad (\text{T-Decl})$$

$$\frac{x \in \text{dom}(\Gamma) \quad \Gamma \vdash e : \Gamma(x)}{\Gamma \vdash x := e \dashv \Gamma} \quad (\text{T-Assign})$$

$$\frac{\Gamma \vdash e_p : \mathbf{String} \quad x \in \text{dom}(\Gamma) \quad \Gamma(x) = \mathbf{Int}}{\Gamma \vdash \mathbf{read} \ e_p \ x \dashv \Gamma} \quad (\text{T-Read})$$

$$\frac{\Gamma \vdash e : T}{\Gamma \vdash \mathbf{print} \ e \dashv \Gamma} \quad (\text{T-Print})$$

A regra (T-Decl) é a única que **estende** o contexto: declara x com tipo T , produzindo $\Gamma' = \Gamma[x \mapsto T]$. As demais **preservam** o contexto: $\Gamma' = \Gamma$.

15.9 Sistema de Tipos para TWhile

TWhile estende TLine com condicionais e repetição:

- **if** e **then** B_1 **else** B_2 : executa o bloco B_1 se e for verdadeiro, B_2 caso contrário.
- **while** e **do** B : repete o bloco B enquanto e for verdadeiro.

A novidade semântica fundamental é o **escopo léxico**: variáveis declaradas dentro de um bloco **if** ou **while** não são visíveis fora dele. Isso é formalizado pelas regras de tipagem: o contexto de **saída** é sempre o contexto de **entrada** Γ , independentemente das declarações internas aos blocos.

15.9.1 Regras de Tipagem para os Novos Comandos

$$\frac{\Gamma \vdash e : \mathbf{Bool} \quad \Gamma \vdash B_1 \dashv \Gamma_1 \quad \Gamma \vdash B_2 \dashv \Gamma_2}{\Gamma \vdash \mathbf{if} \ e \ \mathbf{then} \ B_1 \ \mathbf{else} \ B_2 \dashv \Gamma} \quad (\text{T-If})$$

$$\frac{\Gamma \vdash e : \mathbf{Bool} \quad \Gamma \vdash B \dashv \Gamma'}{\Gamma \vdash \mathbf{while} \ e \ \mathbf{do} \ B \dashv \Gamma} \quad (\text{T-While})$$

O contexto de saída em (T-If) é Γ , e **não** Γ_1 nem Γ_2 . Os contextos Γ_1 e Γ_2 são descartados ao sair dos blocos, de modo que variáveis locais a cada ramo não escapam para o escopo externo. O mesmo vale para (T-While): Γ' é descartado e o contexto retorna a Γ .

15.10 Sistema de Tipos para Funções e Registros

A linguagem TImp estende TWhile com declarações de funções e de tipos registro. Essas duas extensões exigem enriquecer os ambientes de tipagem e adicionar novas regras ao sistema. Nesta seção apresentamos o sistema de tipos de TImp de forma completa.

15.10.1 Tipos, Contextos e Ambientes

Tipos.

$$\tau ::= \mathbf{Int} \mid \mathbf{Bool} \mid \mathbf{String} \mid R \qquad \rho ::= \mathbf{void} \mid \tau$$

O tipo R é um **tipo registro** nomeado; ρ é o tipo de retorno de funções: ou o tipo unitário **void** (sem valor de retorno) ou um tipo τ .

Contexto de variáveis.

$$\Gamma : Var \rightarrow \tau$$

Ambiente de registros.

$$\Delta : Name \rightarrow \overline{(f_i : \tau_i)}$$

$\Delta(R)$ lista os campos e seus tipos para o tipo registro R . Escreve-se $\Delta(R)(f)$ para o tipo do campo f em R .

Ambiente de funções.

$$\Phi : Name \rightarrow ([\tau_1, \dots, \tau_n], \rho)$$

$\Phi(f)$ fornece a lista de tipos dos parâmetros e o tipo de retorno de f . Ambos Δ e Φ são construídos das declarações do programa e permanecem *imutáveis* durante a verificação.

A relação de tipagem de expressões torna-se $\Gamma; \Delta; \Phi \vdash e : \tau$. A de comandos torna-se $\Gamma; \Delta; \Phi; \rho_{ret} \vdash s \dashv \Gamma'$, onde ρ_{ret} é o tipo de retorno esperado da função corrente (ou **void** no nível superior). Por brevidade, omitimos Δ , Φ e ρ_{ret} quando não são necessários numa regra.

15.10.2 Regras de Tipagem para Registros

Expressões

Acesso a campo:

$$\frac{\Gamma; \Delta; \Phi \vdash e : R \quad R \in \text{dom}(\Delta) \quad \Delta(R)(f) = \tau}{\Gamma; \Delta; \Phi \vdash e.f : \tau} \text{ (T-Field)}$$

Construção de registro:

$$\frac{R \in \text{dom}(\Delta) \quad \Delta(R) = \overline{(f_i : \tau_i)} \quad \Gamma; \Delta; \Phi \vdash e_i : \tau_i \quad (\forall i)}{\Gamma; \Delta; \Phi \vdash \mathbf{new} \ R \{f_1 = e_1, \dots, f_n = e_n\} : R} \text{ (T-New)}$$

A regra (T-New) exige que o conjunto de campos fornecidos coincida exatamente com o conjunto de campos declarados em $\Delta(R)$ e que cada campo tenha o tipo correto.

Comandos

Atribuição de campo:

$$\frac{\Gamma(x) = R \quad \Delta(R)(f) = \tau \quad \Gamma; \Delta; \Phi \vdash e : \tau}{\Gamma; \Delta; \Phi \vdash x.f := e \dashv \Gamma} \text{ (T-FieldAssign)}$$

15.10.3 Regras de Tipagem para Funções

Chamadas de função

Chamada como expressão (função não-void):

$$\frac{\Phi(f) = ([\tau_1, \dots, \tau_n], \tau) \quad \Gamma; \Delta; \Phi \vdash e_i : \tau_i \quad (\forall i)}{\Gamma; \Delta; \Phi \vdash f(e_1, \dots, e_n) : \tau} \text{ (T-CallExpr)}$$

Chamada como comando (função void):

$$\frac{\Phi(f) = ([\tau_1, \dots, \tau_n], \mathbf{void}) \quad \Gamma; \Delta; \Phi \vdash e_i : \tau_i \quad (\forall i)}{\Gamma; \Delta; \Phi \vdash f(e_1, \dots, e_n) \dashv \Gamma} \text{ (T-CallStmt)}$$

A distinção entre (T-CallExpr) e (T-CallStmt) é essencial: só funções **void** podem ser usadas como comandos; usar uma função não-**void** como comando desperdiçaria o valor de retorno e é considerado um erro de tipo.

Retorno

As regras de tipagem para **return** dependem do tipo de retorno ρ_{ret} da função corrente, propagado pelo ambiente de tipagem:

$$\frac{\rho_{ret} = \tau \quad \Gamma; \Delta; \Phi \vdash e : \tau}{\Gamma; \dots; \rho_{ret} \vdash \mathbf{return} \ e \dashv \Gamma} \text{ (T-RetVal)}$$

$$\frac{\rho_{ret} = \mathbf{void}}{\Gamma; \dots; \rho_{ret} \vdash \mathbf{return} \dashv \Gamma} \text{ (T-RetVoid)}$$

Declaração de função

A tipagem de uma declaração de função verifica o corpo sob um contexto inicial formado pelos parâmetros formais, com ρ_{ret} fixado ao tipo de retorno declarado:

$$\frac{[x_1 \mapsto \tau_1, \dots, x_n \mapsto \tau_n]; \Delta; \Phi; \rho \vdash B \dashv _}{\Delta; \Phi \vdash \mathbf{fn} \ f(\bar{x}_i : \bar{\tau}_i) : \rho \ \{B\} \checkmark} \text{ (T-FuncDecl)}$$

O contexto de saída do bloco é descartado: parâmetros e variáveis locais à função não são visíveis no nível superior.

15.10.4 Exemplo de Derivação de Tipo

Considere a expressão **new Point** $\{x=3, y=4\}.x + 1$ com $\Delta(\mathbf{Point}) = [(x : \mathbf{Int}), (y : \mathbf{Int})]$ e $\Gamma = \emptyset$:

$$\frac{\frac{\frac{\overline{\vdash 3 : \mathbf{Int}} \quad \overline{\vdash 4 : \mathbf{Int}}}{\vdash \mathbf{new Point}\{x=3, y=4\} : \mathbf{Point}} \quad \Delta(\mathbf{Point})(x) = \mathbf{Int}}{\vdash (\mathbf{new Point}\{x=3, y=4\}).x : \mathbf{Int}} \quad \overline{\vdash 1 : \mathbf{Int}}}{\vdash (\mathbf{new Point}\{x=3, y=4\}).x + 1 : \mathbf{Int}}$$

15.10.5 Extensão da Corretude

As propriedades de preservação e de progresso estendem-se naturalmente para TImp. Dois aspectos merecem nota:

- **Preservação** para (T-Field) requer que o valor $\mathbf{rec}(R, fids)$ tenha os campos esperados por $\Delta(R)$, invariante mantido por (T-New) e (T-FieldAssign).
- **Progresso** para chamadas de função requer que o corpo de f termine com um **return** e quando $\rho \neq \mathbf{void}$. A regra (T-FuncDecl) tipifica o corpo, mas a garantia de que toda execução efetivamente atinge um **return** exige **análise de fluxo de controle** (verificação de retorno garantido), que está além do escopo deste capítulo.

15.11 Conclusão

Neste capítulo, estudamos os fundamentos dos sistemas de tipos em linguagens de programação. Vimos que tipos classificam os valores de uma linguagem e restringem as operações admissíveis, e que a verificação de tipos pode ocorrer em tempo de compilação (tipagem estática) ou em tempo de execução (tipagem dinâmica). Ortogonalmente, uma linguagem pode ser fortemente tipada, proibindo conversões implícitas entre tipos incompatíveis, ou fracamente tipada, admitindo tais coerções de forma automática.

Para dar uma base formal a esses conceitos, utilizamos a linguagem de expressões tipadas de Pierce, que combina booleanos e números naturais. O seu sistema de tipos é definido por regras de inferência da forma $\vdash t : T$, que atribuem um tipo a cada termo de maneira única, propriedade expressa pelo lema de unicidade de tipos. A semântica small-step descreve a execução como uma sequência de reduções $t \rightarrow t'$ e torna explícito o conceito de **formas presas** (*stuck terms*): termos que não são valores e para os quais nenhuma regra de redução se aplica, representando erros irrecuperáveis em tempo de execução. A semântica big-step, por sua vez, relaciona cada termo diretamente ao seu valor final $t \Downarrow v$, abstraindo os passos intermediários.

A propriedade central que justifica a utilidade da tipagem estática é a **corretude do sistema de tipos** (*type soundness*): programas bem tipados nunca ficam presos. Demonstramos essa propriedade pela composição de dois lemas independentes. O lema de **preservação** garante que cada passo de redução mantém o tipo do termo: a informação de tipo não se corrompe durante a execução. O lema de **progresso** garante que um termo bem tipado que ainda não é valor sempre pode dar um passo de redução, de modo que a execução nunca trava. Juntos, os dois lemas asseguram que toda computação a partir de um termo bem tipado avança indefinidamente ou termina em um valor, nunca numa forma presa.

Estendemos o formalismo para linguagens imperativas apresentando os sistemas de tipos de TLine, TWhile e TImp. Para TLine, a presença de variáveis exige um contexto Γ e as regras distinguem declaração (que estende Γ) de atribuição (que o preserva). Para TWhile, a adição de condicionais e repetição com escopo léxico é formalizada pelas regras (T-If) e (T-While), que mantêm o contexto de saída igual ao de entrada, descartando variáveis declaradas internamente. Para TImp, funções e registros introduzem dois ambientes globais imutáveis (Δ para registros e Φ para funções), além do tipo de retorno esperado ρ_{ret} por função. A implementação desses verificadores de tipos é tratada no próximo capítulo.

Capítulo 16

Verificação de tipos

16.1 Introdução

No capítulo anterior, definimos formalmente o que é um sistema de tipos e demonstramos as propriedades de corretude que ele oferece. Vimos que as regras de tipagem, escritas como sistemas de inferência, constituem uma especificação matemática precisa do que significa um programa estar bem tipado. Este capítulo trata de transformar essa especificação em código: o **verificador de tipos** (*type checker*).

A ideia central é que existe uma correspondência direta entre as regras de inferência e as equações de um programa funcional. Cada regra se torna um caso em uma função recursiva; as premissas da regra se tornam chamadas recursivas; e a conclusão é o resultado retornado. Essa tradução é tão sistemática que, em muitos casos, o verificador de tipos pode ser lido como uma transcrição quase literal das regras.

Apresentamos verificadores de tipos para quatro linguagens em progressão crescente de complexidade: *TExp*, *TLine*, *TWhile* e *TImp*. Os sistemas de tipos formais dessas linguagens foram apresentados no capítulo anterior; aqui focamos exclusivamente na tradução dessas regras para código Haskell.

16.2 Verificação de Tipos para TExp

16.2.1 A Linguagem e o seu Sistema de Tipos

A linguagem TExp tem dois tipos, **Bool** e **Nat**, e sete construções: **true**, **false**, **if**, **0**, **succ**, **pred** e **iszero**. Em Haskell:

```
data Ty    = TBool | TNat

data Term
  = TTrue  | TFalse
  | TIf Term Term Term
  | TZero
  | TSucc Term | TPred Term | TIsZero Term
```

O sistema de tipos é definido pela relação $\vdash t : T$ através de sete regras de inferência. A verificação de tipos é o problema de, dado um termo t , encontrar T tal que $\vdash t : T$, ou reportar que nenhum tal T existe.

16.2.2 Da Regra à Equação

A tradução de cada regra de tipagem para uma equação Haskell segue um padrão uniforme. Consideremos a regra (T-If):

$$\frac{\vdash t_1 : \mathbf{Bool} \quad \vdash t_2 : T \quad \vdash t_3 : T}{\vdash \mathbf{if } t_1 \mathbf{ then } t_2 \mathbf{ else } t_3 : T} \quad (\text{T-If})$$

A conclusão nos diz que o padrão a combinar é **TIf** t_1 t_2 t_3 . As premissas indicam três verificações recursivas a realizar:

1. Verificar que t_1 tem tipo **Bool**: a condição deve ser booleana;
2. Verificar que t_2 tem algum tipo T ;
3. Verificar que t_3 tem o **mesmo** tipo T : os ramos devem coincidir;
4. Retornar T como resultado.

Em Haskell:

```
tcTerm (TIf t1 t2 t3) = do
  ty1 <- tcTerm t1
  unless (ty1 == TBool) $
    throwError "Type error in if: condition must have Bool"
  ty2 <- tcTerm t2
  ty3 <- tcTerm t3
  unless (ty2 == ty3) $
    throwError "Type error in if: both branches must have the same type"
  pure ty2
```

A mônada **TcM = ExceptT String Identity** carrega a possibilidade de falha sem precisar de nenhum contexto externo, pois TExp não tem variáveis.

16.2.3 O Verificador Completo

As demais equações seguem o mesmo padrão: cada uma implementa diretamente a sua regra correspondente:

```
type TcM a = ExceptT String Identity a

tcTerm :: Term -> TcM Ty
tcTerm TTrue  = pure TBool           -- (T-True)
tcTerm TFalse = pure TBool           -- (T-False)
tcTerm TZero  = pure TNat            -- (T-Zero)

tcTerm (TIf t1 t2 t3) = do           -- (T-If)
  ty1 <- tcTerm t1
  unless (ty1 == TBool) $
    throwError "Type error in if: condition must have type Bool"
  ty2 <- tcTerm t2
  ty3 <- tcTerm t3
  unless (ty2 == ty3) $
    throwError "Type error in if: both branches must have the same type"
  pure ty2

tcTerm (TSucc t1) = do               -- (T-Succ)
  ty <- tcTerm t1
  unless (ty == TNat) $
    throwError "Type error in succ: argument must have type Nat"
  pure TNat
```

```

tcTerm (TPred t1) = do                                -- (T-Pred)
  ty <- tcTerm t1
  unless (ty == TNat) $
    throwError "Type error in pred: argument must have type Nat"
  pure TNat

tcTerm (TIsZero t1) = do                              -- (T-IsZero)
  ty <- tcTerm t1
  unless (ty == TNat) $
    throwError "Type error in iszero: argument must have type Nat"
  pure TBool

```

A estrutura recursiva de `tcTerm` espelha a estrutura indutiva do sistema de tipos. O verificador de tipos é, em essência, a prova construtiva do lema de unicidade de tipos, executada como um programa.

16.3 Verificação de Tipos para TLine

16.3.1 A Linguagem TLine

TLine é uma linguagem imperativa simples com declaração de variáveis tipadas, atribuição, leitura e impressão. A sua sintaxe abstrata é:

```

data Ty    = TInt  | TBool | TString

data Stmt
  = SDecl   Var Ty Exp    -- var x : T = e ;
  | SAssign Var Exp       -- x := e ;
  | SRead   Exp Var       -- read prompt x ;
  | SPrint  Exp           -- print e ;

data Exp
  = EInt Int | EBool Bool | EString String
  | EVar Var
  | Exp :+: Exp | Exp *: Exp | Exp :-: Exp | Exp :/: Exp
  | Exp <: Exp | Exp :=: Exp
  | Not Exp

```

16.3.2 O Contexto de Tipos

A principal novidade em relação a TExp é a presença de **variáveis**. Para verificar o tipo de `EVar x`, é preciso saber o tipo declarado para `x`. Essa informação é mantida em um **contexto de tipos**:

$$\Gamma : \text{Var} \rightarrow \text{Ty}$$

Em Haskell, o contexto é representado por um `Map`:

```
type Ctx = Map Var Ty
```

O verificador precisa tanto **consultar** o contexto (para variáveis) quanto **estendê-lo** (quando uma declaração é processada). A mônada combina estado e tratamento de erros:

```
type TcM a = StateT Ctx (ExceptT String Identity) a
```

O `StateT Ctx` transporta o contexto ao longo da verificação; o `ExceptT String` interrompe a verificação com uma mensagem de erro.

16.3.3 O Algoritmo de Verificação

O verificador implementa diretamente as regras. Para expressões:

```

tcExp :: Exp -> TcM Ty
tcExp (EInt _)      = pure TInt      -- (T-Int)
tcExp (EBool _)     = pure TBool     -- (T-Bool)
tcExp (EString _)   = pure TString   -- (T-String)
tcExp (EVar v)      = lookupVar v    -- (T-Var)
tcExp (e1 :+: e2) = do              -- (T-ArithOp)
  ty1 <- tcExp e1
  ty2 <- tcExp e2
  unless (ty1 == TInt) $ throwError "Type error"
  unless (ty2 == TInt) $ throwError "Type error"
  pure TInt
tcExp (e1 :<: e2) = do              -- (T-CmpOp)
  ty1 <- tcExp e1
  ty2 <- tcExp e2
  unless (ty1 == TInt) $ throwError "Type error"
  unless (ty2 == TInt) $ throwError "Type error"
  pure TBool
tcExp (Not e1) = do                -- (T-Not)
  ty <- tcExp e1
  unless (ty == TBool) $ throwError "Type error"
  pure TBool

```

As equações para `*:`, `-:` e `/:` são idênticas à de `:+:`; a de `:=:` é idêntica à de `:<:`.

Para comandos, o contexto é atualizado com `modify` quando uma variável é declarada:

```

tcStmt :: Stmt -> TcM ()
tcStmt (SDecl v t e) = do          -- (T-Decl)
  t' <- tcExp e
  unless (t == t') $ throwError "Type error"
  addNewVar v t
tcStmt (SAssign v e) = do          -- (T-Assign)
  t <- lookupVar v
  t' <- tcExp e
  unless (t == t') $ throwError "Type error"
tcStmt (SRead e v) = do            -- (T-Read)
  t <- tcExp e
  unless (t == TString) $ throwError "Type error"
  t' <- lookupVar v
  unless (t' == TInt) $ throwError "Type error"
tcStmt (SPrint e) = tcExp e >> pure () -- (T-Print)

```

O ponto de entrada processa os comandos sequencialmente e retorna o contexto final:

```

typeCheck :: TLine -> Either String [(Var, Ty)]
typeCheck p = either Left (Right . Map.toList) (runTcM (tcTLine p))

tcTLine :: TLine -> TcM ()
tcTLine (TLine ss) = mapM_ tcStmt ss

```

```
runTcM :: TcM a -> Either String Ctx
runTcM m = runIdentity (runExceptT (execStateT m Map.empty))
```

A função `addNewVar` estende o contexto com o par $x \mapsto T$, correspondendo à notação $\Gamma[x \mapsto T]$ nas regras. `lookupVar` consulta o contexto e lança um erro se a variável não estiver definida, implementando a premissa $x \in \text{dom}(\Gamma)$.

16.4 Verificação de Tipos para TWhile

16.4.1 Extensões à Linguagem

TWhile estende TLine com dois novos comandos:

```
data Stmt
  = SDecl    Var Ty Exp
  | SAssign  Var Exp
  | SRead    Exp Var
  | SPrint   Exp
  | SIf      Exp Block Block
  | SWhile   Exp Block

type Block = [Stmt]
```

16.4.2 Escopo de Variáveis

A novidade mais importante de TWhile é que variáveis declaradas **dentro** de um bloco `if` ou `while` não devem ser visíveis fora dele. Este é o conceito de **escopo léxico**: cada bloco tem o seu próprio espaço de nomes local.

A regra se traduz na seguinte exigência de implementação: ao entrar em um bloco, o contexto pode ser estendido pelas declarações internas; ao sair do bloco, o contexto deve ser restaurado ao estado anterior. Essa operação é implementada por `withLocalCtx`:

```
withLocalCtx :: TcM a -> TcM a
withLocalCtx m = do
  ctx <- get      -- salva o contexto atual
  r   <- m        -- verifica o bloco
  put ctx        -- restaura o contexto original
  pure r
```

E `tcBlock` aplica essa operação a cada bloco de comandos:

```
tcBlock :: [Stmt] -> TcM ()
tcBlock ss = withLocalCtx (mapM_ tcStmt ss)
```

16.4.3 As Novas Equações do Verificador

```
tcStmt :: Stmt -> TcM ()
-- ... equações idênticas as de TLine ...
tcStmt (SIf e b1 b2) = do -- (T-If)
  t <- tcExp e
  unless (t == TBool) $ throwError "Type error"
  tcBlock b1
  tcBlock b2
```

```

tcStmt (SWhile e b) = do    -- (T-While)
  t <- tcExp e
  unless (t == TBool) $ throwError "Type error"
  tcBlock b

```

A correspondência com as regras é imediata: `tcExp` verifica a premissa $\Gamma \vdash e : \mathbf{Bool}$; as chamadas a `tcBlock` verificam os blocos; e como `withLocalCtx` restaura o contexto após cada bloco, o escopo léxico é automaticamente imposto.

16.5 Verificação de Tipos para TImp

TImp estende TWhile com declarações de funções e tipos registro. A sua verificação de tipos introduz dois novos ambientes globais (Δ e Φ) e um novo componente local (o tipo de retorno esperado da função corrente ρ_{ret}). A implementação encontra-se em `TImp/Frontend/TypeCheck/TImpTypeCheck.hs`.

16.5.1 A Mônada de Verificação

TWhile usa `StateT Ctx (ExceptT String Identity)` para transportar o contexto de variáveis mutável. TImp precisa, adicionalmente, de ambientes *imutáveis*: Δ , Φ e ρ_{ret} nunca mudam dentro de um bloco. A estrutura natural para informação somente-leitura é `ReaderT`:

```

data TcEnv = TcEnv
{ recordEnv   :: Map Name [(Field, Ty)]    -- Delta
, funcEnv     :: Map Name ([Ty], RetTy)    -- Phi
, returnType  :: Maybe RetTy              -- rho_ret
}

type VarCtx = Map Var Ty

type TcM a = ReaderT TcEnv
              (StateT VarCtx
               (ExceptT String Identity)) a

```

A pilha combina três camadas:

- `ReaderT TcEnv`: ambientes globais imutáveis (Δ , Φ) e o tipo de retorno esperado; acessados com `asks`.
- `StateT VarCtx`: contexto de variáveis Γ , que cresce à medida que variáveis são declaradas; manipulado com `get/modify`.
- `ExceptT String`: erros de tipo, interrompidos com `throwError`.

O ponto de entrada constrói Δ e Φ a partir das declarações antes de iniciar a verificação:

```

typeCheck :: TImp -> Either String TcResult
typeCheck (TImp decls stmts) = do
  recEnv <- buildRecordEnv decls    -- constroi Delta
  funcEnv <- buildFuncEnv recEnv decls -- constroi Phi
  let env = TcEnv recEnv funcEnv Nothing
  mapM_ (tcFuncDecl env) [fd | DFunc fd <- decls]
  finalCtx <- runTcM env (tcBlock stmts)
  ...

```

A construção separada de Δ e Φ garante que declarações de funções e registros sejam visíveis mutuamente: qualquer função pode referenciar qualquer tipo registro e qualquer outra função, independentemente da ordem de declaração.

16.5.2 Regras e Implementação: Registros

Acesso a campo: regra (T-Field)

$$\frac{\Gamma; \Delta; \Phi \vdash e : R \quad \Delta(R)(f) = \tau}{\Gamma; \Delta; \Phi \vdash e.f : \tau} \text{ (T-Field)}$$

```
tcExp (EField e f) = do
  t <- tcExp e
  case t of
    TRecord rname -> lookupField rname f    -- Delta(R)(f)
    _ -> throwError ("Field access on non-record type " ++ showTy t)
```

`lookupField` consulta Δ via `asks recordEnv` e lança um erro se o campo não existe.

Construção de registro: regra (T-New)

$$\frac{R \in \text{dom}(\Delta) \quad \Delta(R) = \overline{(f_i : \tau_i)} \quad \Gamma; \Delta; \Phi \vdash e_i : \tau_i}{\Gamma; \Delta; \Phi \vdash \text{new } R \{f_i = e_i\} : R} \text{ (T-New)}$$

```
tcExp (ENew rname fields) = do
  renv <- asks recordEnv
  case Map.lookup rname renv of
    Nothing -> throwError ("Undefined record type: " ++ rname)
    Just declFlds -> do
      mapM_ (checkField declFlds) fields -- verifica tipos dos campos
      -- verifica que nenhum campo extra foi fornecido
      let provided = map fst fields
          declared = map fst declFlds
      unless (all (`elem` declared) provided) $
        throwError ("Unknown field in new " ++ rname)
      unless (length provided == length declared) $
        throwError ("Wrong number of fields in new " ++ rname)
      pure (TRecord rname)
```

Atribuição de campo: regra (T-FieldAssign)

$$\frac{\Gamma(x) = R \quad \Delta(R)(f) = \tau \quad \Gamma; \Delta; \Phi \vdash e : \tau}{\Gamma; \Delta; \Phi \vdash x.f := e \dashv \Gamma} \text{ (T-FieldAssign)}$$

```
tcStmt (SFieldAssign v f e) = do
  vTy <- lookupVar v
  case vTy of
    TRecord rname -> do
      fieldTy <- lookupField rname f    -- Delta(R)(f)
      t' <- tcExp e
      unless (fieldTy == t') $
        throwError ("Type error in field assignment " ++ v ++ "." ++ f)
    _ -> throwError (v ++ " is not a record")
```


16.5.3 Regras e Implementação: Funções

Chamada como expressão: regra (T-CallExpr)

$$\frac{\Phi(f) = ([\tau_1, \dots, \tau_n], \tau) \quad \Gamma; \Delta; \Phi \vdash e_i : \tau_i}{\Gamma; \Delta; \Phi \vdash f(e_1, \dots, e_n) : \tau} \text{ (T-CallExpr)}$$

```
tcExp (ECall fname args) = do
  fenv <- asks funcEnv
  case Map.lookup fname fenv of
    Nothing -> throwError ("Undefined function: " ++ fname)
    Just (pts, rt) -> do
      argTys <- mapM tcExp args
      unless (argTys == pts) $
        throwError ("Argument type mismatch in call to " ++ fname)
      case rt of
        RTVoid -> throwError ("Void function " ++ fname ++ " used as expression")
        RTTy ty -> pure ty
```

A verificação de que $rt \neq \mathbf{void}$ implementa a distinção entre (T-CallExpr) e (T-CallStmt): uma função void não pode aparecer onde um valor é esperado.

Chamada como comando: regra (T-CallStmt)

$$\frac{\Phi(f) = ([\tau_1, \dots, \tau_n], \mathbf{void}) \quad \Gamma; \Delta; \Phi \vdash e_i : \tau_i}{\Gamma; \Delta; \Phi \vdash f(e_1, \dots, e_n) \dashv \Gamma} \text{ (T-CallStmt)}$$

```
tcStmt (SCall fname args) = do
  fenv <- asks funcEnv
  case Map.lookup fname fenv of
    Nothing -> throwError ("Undefined function: " ++ fname)
    Just (pts, rt) -> do
      unless (rt == RTVoid) $
        throwError ("Non-void call used as statement: " ++ fname)
      argTys <- mapM tcExp args
      unless (argTys == pts) $
        throwError ("Argument type mismatch in call to " ++ fname)
```

Retorno: regras (T-RetVal) e (T-RetVoid)

$$\frac{\rho_{ret} = \tau \quad \Gamma; \Delta; \Phi \vdash e : \tau}{\dots; \rho_{ret} \vdash \mathbf{return} e \dashv \Gamma} \text{ (T-RetVal)} \quad \frac{}{\dots; \mathbf{void} \vdash \mathbf{return} \dashv \Gamma} \text{ (T-RetVoid)}$$

O tipo de retorno esperado é lido do ambiente via `asks returnType`. O valor `Nothing` indica que estamos fora de qualquer função; `return` fora de função é um erro:

```
tcStmt (SReturn Nothing) = do
  mrt <- asks returnType
  case mrt of
    Nothing -> throwError "return outside a function"
    Just RTVoid -> pure ()
    Just (RTTy t) -> throwError ("return with no value in function returning "
      ++ showTy t)

tcStmt (SReturn (Just e)) = do
```

```

mrt <- asks returnType
t' <- tcExp e
case mrt of
  Nothing      -> throwError "return outside a function"
  Just RTVoid  -> throwError "return with value in void function"
  Just (RTTy t) ->
    unless (t == t') $
      throwError ("return type mismatch: expected " ++ showTy t)

```

Declaração de função: regra (T-FuncDecl)

$$\frac{[x_1 \mapsto \tau_1, \dots, x_n \mapsto \tau_n]; \Delta; \Phi; \rho \vdash B \dashv _}{\Delta; \Phi \vdash \mathbf{fn} f(x_i : \tau_i) : \rho \{B\} \checkmark} \text{ (T-FuncDecl)}$$

A função `tcFuncDecl` instala o contexto inicial com os parâmetros e ajusta ρ_{ret} no ambiente via `local`:

```

tcFuncDecl :: TcEnv -> FuncDecl -> Either String ()
tcFuncDecl baseEnv (FuncDecl name params retTy body) = do
  let paramPairs = [(v, t) | Param v t <- params]
      initCtx     = Map.fromList paramPairs
      env         = baseEnv { returnType = Just retTy }
  case runIdentity (runExceptT
    (execStateT (runReaderT (tcBlock body) env) initCtx)) of
    Left err -> Left ("In function " ++ name ++ ": " ++ err)
    Right _  -> Right ()

```

O contexto de variáveis é inicializado com os parâmetros formais ($\Gamma_0 = [x_1 \mapsto \tau_1, \dots, x_n \mapsto \tau_n]$) e o `returnType` é fixado a `Just retTy`. Como o ambiente é imutável e os parâmetros são carregados diretamente no estado inicial, variáveis externas à função são invisíveis dentro dela, implementando o escopo de função isolado exigido pela regra (T-FuncDecl).

16.6 Conclusão

Neste capítulo, mostramos que o verificador de tipos é uma transcrição quase literal do sistema de tipos, e que a correspondência entre regras e código é precisa e sistemática.

Para TExp, a ausência de variáveis permite que o verificador seja uma função pura sem contexto externo. A mônada `ExceptT String Identity` é suficiente.

Para TLine, a introdução de variáveis exige um contexto de tipos Γ mantido como estado da mônada. O verificador deve consultar o contexto para variáveis e estendê-lo para declarações, o que é naturalmente capturado por `StateT Ctx (ExceptT String Identity)`.

Para TWhile, a adição de blocos com escopo local exige que o verificador salve e restaure o contexto ao redor de cada bloco. As regras (T-If) e (T-While) formalizam essa exigência ao manter o contexto de saída igual ao de entrada.

Para TImp, funções e registros introduzem dois novos ambientes globais imutáveis (Δ e Φ) e um contexto de retorno por função (ρ_{ret}), todos naturalmente capturados por `ReaderT TcEnv`. O contexto de variáveis Γ permanece no `StateT`, crescendo com declarações e sendo restaurado ao sair de cada bloco ou função.

O padrão que emerge é geral: a estrutura da mônada de verificação espelha a estrutura dos ambientes semânticos: informação imutável vai em `ReaderT`, informação mutável vai em `StateT`, e falhas vão em `ExceptT`. O sistema de tipos é a especificação; o verificador de tipos é a sua implementação direta.

Parte VI

Geração de código intermediário

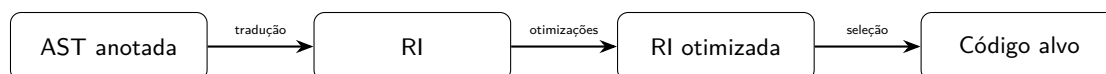
Capítulo 17

Geração de Código Intermediário

17.1 Introdução

A cadeia de transformações de um compilador raramente salta diretamente da árvore sintática abstrata (AST) para o código de máquina. Tal salto seria possível, mas produziria código de baixa qualidade e tornaria o compilador fortemente acoplado à arquitetura-alvo. A estratégia mais usada é introduzir uma **representação intermediária** (RI), uma linguagem interna de grau de abstração situado entre a linguagem-fonte de alto nível e o código de máquina.

A figura a seguir resume o posicionamento da RI no *pipeline*:



Este capítulo define formalmente a RI utilizada na disciplina, apresenta sua semântica operacional *big-step* e descreve as regras de **tradução dirigida por sintaxe** que convertem a AST das linguagens TWhile e TImp para essa RI.

17.2 Propriedades Desejáveis

Uma boa RI deve satisfazer as seguintes propriedades:

Simplicidade. Poucos construtores facilitam a análise e a transformação do código. Um IR com apenas ≈ 15 formas sintáticas é preferível a um que espelhe todas as construções da linguagem-fonte.

Independência de máquina. A RI não deve codificar detalhes do hardware-alvo (convenção de chamada, número de registradores, tamanho de ponteiro). Isso permite que um *front-end* gere RI para múltiplas arquiteturas.

Independência de linguagem. A RI não deve pressupor construções específicas da linguagem-fonte, de modo que diferentes *front-ends* possam compartilhar o mesmo *back-end*. LLVM IR e JVM bytecode são exemplos reais desse princípio.

Suporte a transformações. A RI deve facilitar análises de fluxo e otimizações como propagação de constantes, eliminação de subexpressões comuns e movimentação de código invariante a laços.

Proximidade do código-alvo. A RI deve expor operações de memória e controle de fluxo explícitos, de modo que a seleção de instruções seja direta.

17.3 Abordagens Principais

Existem quatro famílias de RI amplamente usadas:

1. **RI em árvore de baixo nível.** A AST é rebaixada (*lowered*) para uma árvore com operações explícitas de memória e saltos incondicionais/condicionais. É a abordagem adotada neste capítulo e no compilador **Tiger** de Appel.
2. **Código de máquina de pilha (*bytecode*).** Instruções operam em uma pilha de operandos virtual. Exemplos: *p-code* (Pascal), JVM, CPython.
3. **Código de três endereços / quádruplas.** Cada instrução tem a forma $t \leftarrow a \oplus b$, onde t , a e b são temporários ou constantes. Organizado em blocos básicos e grafo de fluxo de controle. Exemplo: LLVM IR.
4. **Representação em estilo de passagem de continuações (CPS).** Comum em compiladores de linguagens funcionais. Todo salto torna-se uma chamada de função; não há implicitamente um “próximo passo”.

Neste capítulo adotamos a abordagem (1): uma **representação intermediária em árvore** (doravante IRT).

17.4 A Representação Intermediária em Árvore (IRT)

A IRT distingue dois tipos de nós: **expressões** (que produzem um valor de tamanho de palavra) e **comandos** (que produzem efeitos colaterais).

17.4.1 Sintaxe das Expressões

Definição 17.1 (Expressões da IRT). *O conjunto das expressões da IRT é dado pela gramática:*

$$\begin{array}{ll}
 e ::= \mathbf{CONST}(n) & n \in \mathbb{Z} \\
 & | \mathbf{TEMP}(t) & t \in \mathit{Temp} \\
 & | \mathbf{NAME}(\ell) & \ell \in \mathit{Label} \\
 & | \mathbf{BINOP}(op, e_1, e_2) \\
 & | \mathbf{MEM}(e) \\
 & | \mathbf{CALL}(e_f, e_1, \dots, e_n) \\
 & | \mathbf{ESEQ}(s, e)
 \end{array}$$

onde $op \in \{\mathbf{ADD}, \mathbf{SUB}, \mathbf{MUL}, \mathbf{DIV}, \mathbf{MOD}, \mathbf{AND}, \mathbf{OR}, \mathbf{XOR}, \mathbf{LSHIFT}, \mathbf{RSHIFT}, \mathbf{ARSHIFT}, \mathbf{EQ}, \mathbf{NEQ}, \mathbf{LT}, \mathbf{LEQ}, \mathbf{GT}, \mathbf{GEQ}\}$.

A semântica informal de cada construtor é:

Construtor	Significado
CONST (n)	Constante inteira n . Booleanos são codificados como 0 (falso) e 1 (verdadeiro).
TEMP (t)	Valor armazenado no temporário t . Temporários são variáveis sem endereço; o alocador de registradores os mapeará para registradores ou posições na pilha.
NAME (ℓ)	Endereço de memória associado ao rótulo ℓ . Permite passar o endereço de uma função ou de um rótulo de salto como dado.
BINOP (op, e_1, e_2)	Aplica a operação binária op aos valores de e_1 e e_2 . Operações de comparação produzem 1 (verdadeiro) ou 0 (falso).
MEM (e)	Lê a palavra de memória no endereço e . No lado esquerdo de MOVE , denota escrita.
CALL ($e_f, \bar{e}$)	Chama a função cujo endereço é e_f , com argumentos $\bar{e}$. O resultado fica em registradores de retorno convencionais.
ESEQ (s, e)	Executa o comando s pelos seus efeitos e então avalia e como valor da expressão inteira. Permite embutir efeitos dentro de uma expressão.

17.4.2 Sintaxe dos Comandos

Definição 17.2 (Comandos da IRT). *O conjunto dos comandos da IRT é dado por:*

$$\begin{aligned}
s ::= & \mathbf{MOVE}(e_{dst}, e) & e_{dst} \in \{\mathbf{TEMP}(t), \mathbf{MEM}(e')\} \\
& | \mathbf{EXP}(e) \\
& | \mathbf{SEQ}(s_1, s_2) \\
& | \mathbf{JUMP}(e) \\
& | \mathbf{CJUMP}(e, \ell_t, \ell_f) \\
& | \mathbf{LABEL}(\ell) \\
& | \mathbf{RETURN}(e_1, \dots, e_n)
\end{aligned}$$

Construtor	Significado
MOVE (TEMP (t), e)	Atribui o valor de e ao temporário t .
MOVE (MEM (e_a), e)	Escreve o valor de e no endereço de memória e_a .
EXP (e)	Avalia e e descarta o resultado; usado para efeitos colaterais (e.g., chamadas sem valor de retorno).
SEQ (s_1, s_2)	Executa s_1 e depois s_2 sequencialmente.
JUMP (e)	Salta incondicionalmente para o endereço e .
CJUMP (e, ℓ_t, ℓ_f)	Se $e \neq 0$ salta para ℓ_t ; caso contrário, para ℓ_f .
LABEL (ℓ)	Define o rótulo ℓ neste ponto do código. Não produz instrução de máquina; apenas marca uma posição endereçável.
RETURN ($\bar{e}$)	Encerra a função corrente devolvendo os valores $\bar{e}$ ao chamador.

Observação. A restrição sobre o destino de **MOVE** (apenas **TEMP** ou **MEM**) é intencional: ela evita destinos arbitrários e facilita a geração de código, pois toda escrita em memória ou em registrador é sintaticamente distinguível.

17.5 Semântica Operacional Big-Step da IRT

17.5.1 Estado e Valores

Definição 17.3 (Estado). Um *estado* é um par $\sigma = \langle \tau, \mu \rangle$ onde:

- $\tau : Temp \rightarrow \mathbb{Z}$ é o armazenamento de temporários, uma função parcial de temporários em inteiros;
- $\mu : \mathbb{Z} \rightarrow \mathbb{Z}$ é a memória, que mapeia endereços inteiros em palavras.

Escrevemos $\sigma.\tau$ e $\sigma.\mu$ para acessar os componentes. As notações $\tau[t \mapsto v]$ e $\mu[a \mapsto v]$ denotam, respectivamente, a atualização do temporário t e da posição de memória a com o valor v .

Todos os valores da IRT são inteiros de precisão de máquina ($\mathbb{Z}$). Ponteiros, booleanos e outros tipos são codificados como inteiros.

Um **programa IRT** é uma função parcial $P : Label \rightarrow Stmt$ que mapeia cada rótulo a um comando (ou sequência de comandos).

17.5.2 Semântica das Expressões

A relação $\sigma \vdash e \Downarrow v$ significa “no estado σ , a expressão e avalia para o inteiro v ”.

Constantes e Temporários

$$\frac{}{\sigma \vdash \mathbf{CONST}(n) \Downarrow n} \text{CONST} \quad \frac{\sigma.\tau(t) = v}{\sigma \vdash \mathbf{TEMP}(t) \Downarrow v} \text{TEMP}$$

$$\frac{}{\sigma \vdash \mathbf{NAME}(\ell) \Downarrow \text{addr}(\ell)} \text{NAME}$$

A função $\text{addr}(\ell)$ devolve o endereço de código associado ao rótulo ℓ ; assume-se que este mapeamento é fixado em tempo de link.

Operações Binárias

$$\frac{\sigma \vdash e_1 \Downarrow v_1 \quad \sigma \vdash e_2 \Downarrow v_2}{\sigma \vdash \mathbf{BINOP}(op, e_1, e_2) \Downarrow op(v_1, v_2)} \text{BINOP}$$

A interpretação de $op(v_1, v_2)$ é a habitual. Para operadores de comparação (EQ, LT, etc.), o resultado é 1 se a relação se mantém e 0 caso contrário:

$$\text{EQ}(v_1, v_2) = \begin{cases} 1 & \text{se } v_1 = v_2 \\ 0 & \text{caso contrário.} \end{cases}$$

Acesso à Memória

$$\frac{\sigma \vdash e \Downarrow a \quad \sigma.\mu(a) = v}{\sigma \vdash \mathbf{MEM}(e) \Downarrow v} \text{MEM}$$

Chamada de Função

$$\frac{\sigma \vdash e_f \Downarrow a \quad \sigma \vdash e_i \Downarrow v_i \quad \text{invoke}(a, v_1, \dots, v_n) = v}{\sigma \vdash \mathbf{CALL}(e_f, e_1, \dots, e_n) \Downarrow v} \text{ CALL}$$

A função auxiliar *invoke* modela a invocação da função residente no endereço a com os argumentos fornecidos. Seu resultado é o valor de retorno. A implementação concreta de *invoke* depende da convenção de chamada e está fora do escopo desta apresentação formal.

Expressão Sequenciada

$$\frac{P, \sigma \vdash s \Downarrow \langle \sigma', \blacksquare \rangle \quad \sigma' \vdash e \Downarrow v}{\sigma \vdash \mathbf{ESEQ}(s, e) \Downarrow v} \text{ ESEQ}$$

O **ESEQ** executa o comando s (modificando o estado) e depois avalia e no estado resultante. A notação $P, \sigma \vdash s \Downarrow \langle \sigma', \blacksquare \rangle$ é definida na seção seguinte.

17.5.3 Semântica dos Comandos

A relação $P, \sigma \vdash s \Downarrow r$ significa “dado o programa P e o estado inicial σ , o comando s executa e produz o resultado r ”.

Definição 17.4 (Resultado de Execução). *O resultado r de um comando é:*

$$r ::= \langle \sigma', \blacksquare \rangle \mid \langle \sigma', \uparrow(v_1, \dots, v_k) \rangle$$

O símbolo $\blacksquare$ denota terminação normal e $\uparrow(v_1, \dots, v_k)$ denota retorno de função com valores $v_1, \dots, v_k$.

Atribuição a Temporário

$$\frac{\sigma \vdash e \Downarrow v \quad \sigma' = \langle \sigma.\tau[t \mapsto v], \sigma.\mu \rangle}{P, \sigma \vdash \mathbf{MOVE}(\mathbf{TEMP}(t), e) \Downarrow \langle \sigma', \blacksquare \rangle} \text{ MOVETEMP}$$

Escrita em Memória

$$\frac{\sigma \vdash e_a \Downarrow a \quad \sigma \vdash e \Downarrow v \quad \sigma' = \langle \sigma.\tau, \sigma.\mu[a \mapsto v] \rangle}{P, \sigma \vdash \mathbf{MOVE}(\mathbf{MEM}(e_a), e) \Downarrow \langle \sigma', \blacksquare \rangle} \text{ MOVEMEM}$$

Expressão como Comando

$$\frac{\sigma \vdash e \Downarrow _}{P, \sigma \vdash \mathbf{EXP}(e) \Downarrow \langle \sigma, \blacksquare \rangle} \text{ EXP}$$

O valor produzido por e é descartado; apenas os efeitos colaterais (como uma chamada de função com efeitos) são relevantes.

Sequência

$$\frac{P, \sigma \vdash s_1 \Downarrow \langle \sigma', \blacksquare \rangle \quad P, \sigma' \vdash s_2 \Downarrow \langle \sigma'', r \rangle}{P, \sigma \vdash \mathbf{SEQ}(s_1, s_2) \Downarrow \langle \sigma'', r \rangle} \text{ SEQNORM}$$

$$\frac{P, \sigma \vdash s_1 \Downarrow \langle \sigma', \uparrow \bar{v} \rangle}{P, \sigma \vdash \mathbf{SEQ}(s_1, s_2) \Downarrow \langle \sigma', \uparrow \bar{v} \rangle} \text{ SEQRET}$$

Se s_1 retorna, s_2 não é executado: o resultado de retorno propaga-se diretamente.

Rótulo

$$\frac{}{P, \sigma \vdash \mathbf{LABEL}(\ell) \Downarrow \langle \sigma, \blacksquare \rangle} \text{ LABEL}$$

LABEL não produz efeito em tempo de execução; é uma anotação estática para o vinculador e para os saltos.

Salto Incondicional

$$\frac{P(\ell) = s \quad P, \sigma \vdash s \Downarrow r}{P, \sigma \vdash \mathbf{JUMP}(\mathbf{NAME}(\ell)) \Downarrow r} \text{ JUMP}$$

O salto consulta P para obter a continuação do rótulo ℓ e a executa.

Salto Condicional

$$\frac{\sigma \vdash e \Downarrow v \quad v \neq 0 \quad P(\ell_t) = s \quad P, \sigma \vdash s \Downarrow r}{P, \sigma \vdash \mathbf{CJUMP}(e, \ell_t, \ell_f) \Downarrow r} \text{ CJUMPTTRUE}$$

$$\frac{\sigma \vdash e \Downarrow 0 \quad P(\ell_f) = s \quad P, \sigma \vdash s \Downarrow r}{P, \sigma \vdash \mathbf{CJUMP}(e, \ell_t, \ell_f) \Downarrow r} \text{ CJUMPFALSE}$$

Retorno

$$\frac{\sigma \vdash e_i \Downarrow v_i \quad (1 \leq i \leq k)}{P, \sigma \vdash \mathbf{RETURN}(e_1, \dots, e_k) \Downarrow \langle \sigma, \uparrow(v_1, \dots, v_k) \rangle} \text{ RETURN}$$

Observação. A semântica big-step apresentada não termina para programas que executam laços infinitos, o que é esperado: a semântica big-step é definida apenas para computações que terminam. Para raciocinar sobre não-terminação, utiliza-se semântica small-step ou a semântica de rastreamentos (*trace semantics*).

17.6 Tradução Dirigida por Sintaxe

A tradução de TWhile e TImp para a IRT é definida por duas funções de tradução indutivas sobre a AST:

- $\mathcal{E}[[e]]$: traduz uma expressão de entrada em uma expressão IRT;
- $\mathcal{S}[[s]]$: traduz um comando de entrada em um comando IRT.

Assumimos que a AST foi previamente verificada pelo analisador semântico, de modo que todos os tipos são conhecidos e corretos.

17.6.1 Tradução de Expressões

Literais

$$\begin{aligned} \mathcal{E}[[\mathbf{false}]] &= \mathbf{CONST}(0) \\ \mathcal{E}[[\mathbf{true}]] &= \mathbf{CONST}(1) \\ \mathcal{E}[[n]] &= \mathbf{CONST}(n) \quad n \in \mathbb{Z} \end{aligned}$$

Strings são tratadas como ponteiros para dados estáticos e são aqui omitidas por brevidade.

Variáveis

$$\mathcal{E}[\![x]\!] = \mathbf{TEMP}(x)$$

Cada variável local é mapeada para um temporário de mesmo nome. O alocador de registradores decide posteriormente se o temporário vive em um registrador ou em uma posição na pilha.

Operações Aritméticas e Relacionais

$$\mathcal{E}[\![e_1 \oplus e_2]\!] = \mathbf{BINOP}(\hat{\oplus}, \mathcal{E}[\![e_1]\!], \mathcal{E}[\![e_2]\!])$$

onde $\hat{\oplus}$ é o operador IRT correspondente:

TWhile	IRT	TWhile	IRT	TWhile	IRT
+	ADD	==	EQ	<	LT
-	SUB	!=	NEQ	<=	LEQ
*	MUL	>	GT	>=	GEQ
/	DIV	&&	AND		OR
%	MOD				

Negação Unária e not

$$\begin{aligned}\mathcal{E}[\![- e]\!] &= \mathbf{BINOP}(\mathbf{SUB}, \mathbf{CONST}(0), \mathcal{E}[\![e]\!]) \\ \mathcal{E}[\![\text{not } e]\!] &= \mathbf{BINOP}(\mathbf{EQ}, \mathcal{E}[\![e]\!], \mathbf{CONST}(0))\end{aligned}$$

Chamada de Função

$$\mathcal{E}[\![f(e_1, \dots, e_n)]\!] = \mathbf{CALL}(\mathbf{NAME}(f), \mathcal{E}[\![e_1]\!], \dots, \mathcal{E}[\![e_n]\!])$$

17.6.2 Tradução de Comandos**Declaração de Variável Local**

$$\mathcal{S}[\![\text{let } x : T := e;]\!] = \mathbf{MOVE}(\mathbf{TEMP}(x), \mathcal{E}[\![e]\!])$$

Atribuição Simples

$$\mathcal{S}[\![x := e;]\!] = \mathbf{MOVE}(\mathbf{TEMP}(x), \mathcal{E}[\![e]\!])$$

Retorno

$$\mathcal{S}[\![\text{return } e;]\!] = \mathbf{RETURN}(\mathcal{E}[\![e]\!])$$

$$\mathcal{S}[\![\text{return};]\!] = \mathbf{RETURN}()$$

Sequência de Comandos

$$\begin{aligned}\mathcal{S}[\![s_1 \ s_2 \ \dots \ s_n]\!] &= \mathbf{SEQ}(\mathcal{S}[\![s_1]\!], \\ &\quad \mathbf{SEQ}(\mathcal{S}[\![s_2]\!], \\ &\quad \dots \ \mathcal{S}[\![s_n]\!]\dots))\end{aligned}$$

17.6.3 Tradução de Estruturas de Controle

Condicional

Para traduzir `if e {bthen}`, geramos dois novos rótulos ℓ_t e ℓ_f :

$$\begin{aligned} S[\![\text{if } e \{b_t\}]\!] = & \text{SEQ}(\\ & \text{CJUMP}(\mathcal{E}[e], \ell_t, \ell_f), \\ & \text{SEQ}(\\ & \quad \text{LABEL}(\ell_t), \\ & \quad \text{SEQ}(\\ & \quad \quad S[b_t], \\ & \quad \quad \text{LABEL}(\ell_f))) \end{aligned}$$

Para `if e {bt} else {bf}`, introduzimos um rótulo adicional ℓ_{end} :

$$\begin{aligned} S[\![\text{if } e \{b_t\} \text{ else } \{b_f\}]\!] = & \text{SEQ}(\\ & \text{CJUMP}(\mathcal{E}[e], \ell_t, \ell_f), \\ & \text{SEQ}(\\ & \quad \text{LABEL}(\ell_t), \\ & \quad \text{SEQ}(\\ & \quad \quad S[b_t], \\ & \quad \quad \text{SEQ}(\\ & \quad \quad \quad \text{JUMP}(\text{NAME}(\ell_{end})), \\ & \quad \quad \quad \text{SEQ}(\\ & \quad \quad \quad \quad \text{LABEL}(\ell_f), \\ & \quad \quad \quad \quad \text{SEQ}(\\ & \quad \quad \quad \quad \quad S[b_f], \\ & \quad \quad \quad \quad \quad \text{LABEL}(\ell_{end})))))) \end{aligned}$$

O **JUMP** ao final do ramo *then* evita que a execução “caia” no corpo do ramo *else*.

Laço while

Para `while e {b}`, introduzimos três novos rótulos: ℓ_h (cabeça do laço), ℓ_b (corpo) e ℓ_f (após o laço):

$$\begin{aligned} S[\![\text{while } e \{b\}]\!] = & \text{SEQ}(\\ & \text{LABEL}(\ell_h), \\ & \text{SEQ}(\\ & \quad \text{CJUMP}(\mathcal{E}[e], \ell_b, \ell_f), \\ & \quad \text{SEQ}(\\ & \quad \quad \text{LABEL}(\ell_b), \\ & \quad \quad \text{SEQ}(\\ & \quad \quad \quad S[b], \\ & \quad \quad \quad \text{SEQ}(\\ & \quad \quad \quad \quad \text{JUMP}(\text{NAME}(\ell_h)), \\ & \quad \quad \quad \quad \text{LABEL}(\ell_f)))) \end{aligned}$$

O **JUMP**(NAME(ℓ_h)) ao final do corpo fecha o laço, retornando ao teste da condição.

17.6.4 Tradução de Funções

Uma declaração de função de nível superior

$$\text{fn } f (p_1 : T_1, \dots, p_n : T_n) : T_r \{b\}$$

é traduzida em um **fragmento de função** (f, s_{body}) onde:

$$s_{body} = \text{SEQ}(\begin{array}{l} \text{MOVE}(\text{TEMP}(p_1), \text{TEMP}(a_1)), \\ \dots \\ \text{SEQ}(\begin{array}{l} \text{MOVE}(\text{TEMP}(p_n), \text{TEMP}(a_n)), \\ \mathcal{S}[[b]] \end{array}) \end{array})$$

Os temporários $a_1, \dots, a_n$ representam os registradores (ou posições de pilha) onde a convenção de chamada deposita os argumentos. O prólogo da função copia cada argumento para o temporário do parâmetro formal correspondente.

17.7 Exemplo Completo

Consideremos a função fatorial:

```
fn fat(n : int) : int {
  if n <= 1 {
    return 1;
  } else {
    return n * fat(n - 1);
  }
}
```

Aplicando as regras de tradução, obtemos o fragmento IRT (omitindo os rótulos numéricos por brevidade):

```
fat:
  MOVE(TEMP(n), TEMP(a0))          -- copia arg da conv. de chamada
  CJUMP(BINOP(LEQ, TEMP(n), CONST(1)), L_then, L_else)
  LABEL(L_then)
    RETURN(CONST(1))
  LABEL(L_else)
    RETURN(BINOP(MUL,
                  TEMP(n),
                  CALL(NAME(fat),
                       BINOP(SUB, TEMP(n), CONST(1))))))
```

Observe que cada ramo do if-else termina com **RETURN**, de modo que não é necessário um rótulo de confluência ao final.

17.8 Formas Canônicas

A IRT recém-gerada contém dois tipos de “ruído” que complicam as fases posteriores:

1. **ESEQ em posições arbitrárias.** **ESEQ** pode reordenar efeitos colaterais de maneira não-óbvia quando aparece dentro de operandos de **BINOP** ou **CALL**.
2. **CALL dentro de expressões.** O valor de retorno de uma chamada precisa ser capturado em um temporário antes de ser usado como operando.

O processo de **canonicalização** elimina esses problemas em duas passagens:

Linearização. **ESEQ** é “içado” (*hoisted*) para o nível de comando via reescrita:

$$\text{BINOP}(op, \text{ESEQ}(s, e_1), e_2) \longrightarrow \text{ESEQ}(s, \text{BINOP}(op, e_1, e_2))$$

desde que s não contenha efeitos que interfiram com e_2 .

Extração de chamadas. Toda **CALL** dentro de uma expressão é substituída por

$$\begin{aligned} &\text{ESEQ}(\\ &\quad \text{MOVE}(\text{TEMP}(t), \text{CALL}(\dots)), \\ &\quad \text{TEMP}(t)) \end{aligned}$$

para um novo temporário t .

Após a canonicalização, a IRT está em **forma linear**: uma sequência plana de comandos sem **ESEQ**, que pode ser facilmente particionada em **blocos básicos** para análise de fluxo de controle.

17.9 Implementação em Haskell: Geração de Código para TWhile

Esta seção descreve a implementação concreta do *backend* de geração de código intermediário para a linguagem TWhile, desenvolvida em Haskell no módulo `TWhile.Backend.IR.TWhileCodegen`. A implementação segue de perto as regras de tradução apresentadas na Seção 17.6 e serve como referência direta entre a teoria e o código executável.

17.9.1 Mônada de Geração de Código

A geração de código requer dois efeitos distintos:

1. **Geração de rótulos:** os construtores **SWhile** e **SIf** necessitam de rótulos únicos para os pontos de entrada e saída de cada bloco de controle. Um contador global garante a unicidade.
2. **Sinalização de erros:** situações inesperadas durante a compilação são propagadas como mensagens de erro sem abortar o programa.

Esses dois efeitos são combinados via transformadores de mônada:

```
data CgState = CgState
  { nextLabel :: Int
  , stmtAcc   :: [IR.Stmt]
  }

type CgM a = StateT CgState (ExceptT String Identity) a
```

O estado `CgState` mantém:

- `nextLabel`: o próximo inteiro livre para formar um rótulo único;
- `stmtAcc`: a lista de instruções IRT emitidas até o momento (o acumulador de saída).

A camada `ExceptT String Identity` provê propagação de erros sem I/O; `Identity` é usada porque a geração de código é uma transformação pura. Dois auxiliares encapsulam as operações mais comuns:

```
freshLabel :: String -> CgM IR.Label
freshLabel prefix = do
  n <- gets nextLabel
  modify (\s -> s { nextLabel = n + 1 })
  return (prefix ++ "_" ++ show n)

emit :: IR.Stmt -> CgM ()
emit s = modify (\st -> st { stmtAcc = stmtAcc st ++ [s] })
```

`freshLabel` consome o contador e devolve um rótulo como `"while_head_0"`, `"if_true_1"`, etc. `emit` acrescenta uma instrução ao acumulador.

17.9.2 Ponto de Entrada

A função principal recebe a AST de um programa `TWhile` e produz um `IR.Program`, isto é, uma lista de `FuncDef`:

```
compileTWhile :: TWhile -> Either String IR.Program
compileTWhile (TWhile stmts) =
  case runCgM (mapM_ codegenStmt stmts) of
    Left err -> Left err
    Right ss -> Right [IR.FuncDef "main" [] (foldStmts ss)]
```

O programa `TWhile` inteiro é embutido em uma única função `main` sem parâmetros. A função auxiliar `foldStmts` converte a lista plana de instruções em uma árvore `SEQ` associada à direita:

```
foldStmts :: [IR.Stmt] -> IR.Stmt
foldStmts [] = IR.EXP (IR.CONST 0)
foldStmts [s] = s
foldStmts (s:ss) = IR.SEQ s (foldStmts ss)
```

17.9.3 Geração de Comandos

A função `codegenStmt` implementa as regras de tradução de comandos. Cada caso consome um construtor da AST de `TWhile` e emite zero ou mais instruções IRT.

Declaração e Atribuição

```
codegenStmt (SDecl v _ e) = do
  ir <- codegenExp e
  emit (IR.MOVE (IR.TEMP v) ir)

codegenStmt (SAssign v e) = do
  ir <- codegenExp e
  emit (IR.MOVE (IR.TEMP v) ir)
```

Ambos os casos traduzem diretamente para `MOVE(TEMP(v),  $\mathcal{E}[e])$` , conforme as regras da Seção 17.6.2. A anotação de tipo na declaração é descartada, pois a IRT é não-tipada.

Impressão e Leitura

```
codegenStmt (SPrint e) = do
  ir <- codegenExp e
  emit (IR.EXP (IR.CALL (IR.NAME "print") [ir]))

codegenStmt (SRead _ v) =
  emit (IR.MOVE (IR.TEMP v) (IR.CALL (IR.NAME "read_int") []))
```

A impressão compila para uma chamada à função embutida `print`, cujo efeito colateral é exibir o valor no terminal. A leitura descarta a expressão de *prompt* (que seria uma string, não representável na IRT inteira) e obtém o valor via a função embutida `read_int`.

Laço while

```
codegenStmt (SWhile e body) = do
  lh <- freshLabel "while_head"
  lb <- freshLabel "while_body"
  lf <- freshLabel "while_exit"
  emit (IR.LABEL lh)
  cond <- codegenExp e
  emit (IR.CJUMP cond lb lf)
  emit (IR.LABEL lb)
  mapM_ codegenStmt body
  emit (IR.JUMP (IR.NAME lh))
  emit (IR.LABEL lf)
```

A implementação segue fielmente o esquema da Seção 17.6.3:

Instrução emitida	Papel
LABEL lh	Cabeça do laço; destino do salto de retorno
CJUMP(cond, lb, lf)	Avalia a condição; entra no corpo ou sai
LABEL lb	Início do corpo
<i>corpo</i>	Instruções geradas recursivamente
JUMP(NAME lh)	Fecha o laço, volta ao teste
LABEL lf	Ponto de saída do laço

Condicional if-else

```
codegenStmt (SIf e b1 b2) = do
  lt <- freshLabel "if_true"
  lf <- freshLabel "if_false"
  le <- freshLabel "if_end"
  cond <- codegenExp e
  emit (IR.CJUMP cond lt lf)
  emit (IR.LABEL lt)
  mapM_ codegenStmt b1
  emit (IR.JUMP (IR.NAME le))
  emit (IR.LABEL lf)
  mapM_ codegenStmt b2
  emit (IR.LABEL le)
```

O JUMP(NAME le) ao final do ramo *then* é essencial: sem ele a execução “cairia” para o ramo *else* após concluir o ramo verdadeiro.

17.9.4 Geração de Expressões

A função `codegenExp` implementa $\mathcal{E}[\cdot]$ e devolve um `IR.Expr`:

```
codegenExp :: Exp -> CgM IR.Expr
codegenExp (EInt n)      = return (IR.CONST n)
codegenExp (EBool True)  = return (IR.CONST 1)
codegenExp (EBool False) = return (IR.CONST 0)
codegenExp (EString _)   = return (IR.CONST 0)
codegenExp (EVar v)      = return (IR.TEMP v)

codegenExp (e1 :+: e2) = IR.BINOP IR.BAdd <$> codegenExp e1 <*> codegenExp e2
codegenExp (e1 :*: e2) = IR.BINOP IR.BMul <$> codegenExp e1 <*> codegenExp e2
codegenExp (e1 :-: e2) = IR.BINOP IR.BSub <$> codegenExp e1 <*> codegenExp e2
codegenExp (e1 :/: e2) = IR.BINOP IR.BDiv <$> codegenExp e1 <*> codegenExp e2
codegenExp (e1 :<: e2) = IR.BINOP IR.BLt <$> codegenExp e1 <*> codegenExp e2
codegenExp (e1 :=: e2) = IR.BINOP IR.BEq <$> codegenExp e1 <*> codegenExp e2

codegenExp (e1 :&&: e2) = IR.BINOP IR.BAnd <$> codegenExp e1 <*> codegenExp e2
codegenExp (e1 :||: e2) = IR.BINOP IR.BOr <$> codegenExp e1 <*> codegenExp e2

codegenExp (Not e) = IR.BINOP IR.BEq (IR.CONST 0) <$> codegenExp e
```

Dois pontos merecem atenção:

- **Booleanos.** `true` compila para `CONST(1)` e `false` para `CONST(0)`. Como os únicos valores booleanos são 0 e 1, os operadores bit a bit `AND` e `OR` coincidem com os conectivos lógicos `&&` e `||`.
- **Negação lógica.** A expressão `not e` é traduzida para `BINOP(EQ, CONST(0),  $\mathcal{E}[e]$ )`: se `e` avalia para 0 (falso), o resultado é 1; se avalia para 1 (verdadeiro), o resultado é 0.
- **Strings.** A IRT é inteiramente inteira; strings não têm representação direta e são compiladas para `CONST(0)` como aproximação conservadora.

17.9.5 Exemplo: Fatorial Iterativo

O programa `TWhile` abaixo computa `5!` de forma iterativa:

```
let n : int := 5;
let acc : int := 1;
while n > 0 {
  acc := acc * n;
  n := n - 1;
}
print acc;
```

Após a invocação de `compileTWhile`, o código IRT gerado é (com rótulos renomeados para clareza):

```
func main() {
  MOVE(TEMP(n), CONST(5))
  MOVE(TEMP(acc), CONST(1))
  LABEL(while_head)
  CJUMP(BINOP(LT, CONST(0), TEMP(n)), while_body, while_exit)
  LABEL(while_body)
  MOVE(TEMP(acc), BINOP(MUL, TEMP(acc), TEMP(n)))
```



```

    MOVE(TEMP(n),  BINOP(SUB, TEMP(n),  CONST(1)))
    JUMP(NAME(while_head))
LABEL(while_exit)
EXP(CALL(NAME(print), TEMP(acc)))
}

```

A saída é uma função `main` sem parâmetros cuja sequência de instruções espelha diretamente a estrutura do programa fonte.

17.9.6 Integração no Pipeline

O executável `twhile` expõe a opção `-ir`:

```
twhile --ir -f programa.tw
```

O pipeline executa as seguintes etapas em ordem:

1. **Análise sintática:** o parser produz a AST de TWhile.
2. **Verificação de tipos:** o analisador semântico valida a AST e devolve o contexto de tipos final.
3. **Geração de código:** `compileTWhile` converte a AST verificada em um `IR.Program`.
4. **Impressão:** o módulo `IR.Frontend.Pretty.IRPretty` serializa o programa IRT para texto e o resultado é escrito num arquivo `.ir` com o mesmo nome base do arquivo fonte.

O arquivo `.ir` gerado pode então ser interpretado diretamente pelo executável `ir -interp`, fechando o ciclo *fonte* $\rightarrow$ *IRT* $\rightarrow$ *execução*.

17.10 Geração de Código para TImp: Funções e Registros

A linguagem TImp estende TWhile com **declarações de funções** e **tipos registro**. O *backend* correspondente encontra-se em `TImp.Backend.IR.TImpCodegen` e herda toda a estrutura de TWhile, acrescentando:

1. regras de tradução para acesso e atribuição de campos;
2. uma regra de alocação para a construção de registros;
3. a tradução de cada declaração de função para um fragmento `FuncDef` distinto;
4. propagação do comando `return` via `RETURN`.

17.10.1 Representação de Registros na Memória

Registros são alocados no **heap** como blocos contíguos de palavras de 64 bits ($w = 8$ bytes). Os campos são numerados pela ordem de declaração, a partir de 0. O campo de índice i de um registro cujo endereço base está em `TEMP( $x$ )` está no endereço:

$$addr(x, i) = \text{BINOP}(\text{ADD}, \text{TEMP}(x), \text{CONST}(i \cdot w))$$

Para um registro `Point { x : int; y : int }`, o campo `x` tem índice 0 (deslocamento 0) e o campo `y` tem índice 1 (deslocamento 8). Não há cabeçalho de tipo: a distinção de tipo é responsabilidade do sistema de tipos estático.

A alocação é realizada pela função embutida `alloc`, chamada com o número de campos como argumento. O interpretador IRT implementa `alloc` via um alocador *bump-pointer* iniciado num endereço fixo alto:

```
callFunc _ (NAME "alloc") [n] = do
  ptr <- gets heapPtr
  modify (\s -> s { heapPtr = ptr + n * 8 })
  return ptr
```

17.10.2 Tradução Dirigida por Sintaxe para Registros

Construção de Registro

Seja R declarado com campos $(f_0, \dots, f_{n-1})$ em ordem. A expressão **new** $R \{f_0=e_0, \dots, f_{n-1}=e_{n-1}\}$ é traduzida para:

$$\mathcal{E}[\text{new } R \{ \overline{f_i=e_i} \}] = \text{ESEQ}(s_{\text{alloc}}, \text{TEMP}(t))$$

onde t é um novo temporário e s_{alloc} é o bloco de alocação e inicialização dos campos:

$$\begin{aligned} s_{\text{alloc}} = & \text{SEQ}(\\ & \text{MOVE}(\text{TEMP}(t), \text{CALL}(\text{NAME}(\text{alloc}), \text{CONST}(n))), \\ & \text{SEQ}(\\ & \quad \text{MOVE}(\text{MEM}(\text{addr}(t, 0)), \mathcal{E}[e_0]), \\ & \quad \dots \\ & \quad \text{MOVE}(\text{MEM}(\text{addr}(t, n-1)), \mathcal{E}[e_{n-1}])) \end{aligned}$$

O bloco s_{alloc} aloca n palavras, armazena os valores dos campos em ordem e o **ESEQ** devolve **TEMP**(t) como endereço do registro recém-criado.

Acesso a Campo

Dado que e tem tipo R e f tem índice i em R :

$$\mathcal{E}[e.f] = \text{MEM}(\text{BINOP}(\text{ADD}, \mathcal{E}[e], \text{CONST}(i \cdot w)))$$

Tal como o acesso a componente de tupla (Seção 17.6.1), o índice é convertido em deslocamento de byte.

Atribuição a Campo

$$\mathcal{S}[x.f := e] = \text{MOVE}(\text{MEM}(\text{BINOP}(\text{ADD}, \text{TEMP}(x), \text{CONST}(i \cdot w))), \mathcal{E}[e])$$

A semântica de valor de TImp é preservada: esta instrução *não* cria uma cópia do registro; atualiza o campo f no bloco já alocado, pois a variável x detém o *endereço* do registro. (A cópia ocorre apenas quando uma variável do tipo registro é passada como argumento ou atribuída a outra variável, ponto em que o *front-end* pode inserir chamadas a **alloc**; na IRT, toda passagem de registro é por referência implícita via o temporário que guarda o endereço.)

17.10.3 Tradução de Funções

Cada declaração de função **fn** $f(\overline{x_i : \tau_i}) : \rho \{B\}$ torna-se um **FuncDef** independente:

$$\mathcal{F}[\text{fn } f(\overline{x_i : \tau_i}) : \rho \{B\}] = \text{FuncDef } f[x_1, \dots, x_n] (\text{foldStmts}(\mathcal{S}[B]))$$

Os nomes dos parâmetros formais x_i são usados diretamente como temporários no corpo, o que é correto porque o interpretador IRT inicializa **TEMP**(x_i) = v_i no estado local de cada chamada: não há prólogo de cópia de argumentos.

As regras de tradução para **return** são:

$$S[\llbracket \text{return } e; \rrbracket] = \text{RETURN}(\mathcal{E}[e]) \quad S[\llbracket \text{return}; \rrbracket] = \text{RETURN}()$$

Chamadas de função como expressão traduzem-se como antes:

$$\mathcal{E}[f(e_1, \dots, e_n)] = \text{CALL}(\text{NAME}(f), \mathcal{E}[e_1], \dots, \mathcal{E}[e_n])$$

17.10.4 Implementação em Haskell

Estado Estendido da Geração de Código

O estado de TWhile é estendido com dois campos adicionais:

```
data CgState = CgState
{ nextLabel    :: Int
, nextTemp     :: Int
, stmtAcc      :: [IR.Stmt]
, recFields    :: Map Name [Field]  -- ordem de campos por tipo R
, varTypes     :: Map Var Ty        -- tipo de cada variavel local
}
```

- **recFields** mapeia o nome de cada tipo registro à lista ordenada de seus campos. Permite calcular o índice i de um campo f em $O(\log n)$ com **elemIndex**.
- **varTypes** rastreia o tipo TImp de cada variável declarada. É necessário para traduzir **EField** e **SFieldAssign**: o nome do tipo registro deve ser recuperado para consultar **recFields**.
- **nextTemp** fornece novos temporários para a construção de registros, evitando colisões com os nomes de variáveis do programa.

O estado inicial é construído com **recFields** já preenchido e **varTypes** vazio (as variáveis são inseridas à medida que as declarações são processadas):

```
initState :: Map Name [Field] -> CgState
initState rf = CgState 0 0 [] rf Map.empty
```

Ponto de Entrada

```
compileTImp :: TImp -> Either String IR.Program
compileTImp (TImp decls stmts) = do
  let rf = Map.fromList
      [ (n, [f | FieldDecl f _ <- fields])
      | DRecord (RecordDecl n fields) <- decls ]
  funcDefs <- mapM (compileFuncDecl rf) [fd | DFunc fd <- decls]
  mainStmts <- runCgM rf (mapM_ codegenStmt stmts)
  let mainDef = IR.FuncDef "main" [] (foldStmts mainStmts)
  pure (funcDefs ++ [mainDef])
```

O mapa **rf** é construído uma vez a partir das declarações de tipo. Cada função é compilada independentemente por **compileFuncDecl**; os comandos de nível superior são colocados numa função **main** implícita.

Compilação de Declarações de Função

```
compileFuncDecl :: Map Name [Field] -> FuncDecl
                -> Either String IR.FuncDef
compileFuncDecl rf (FuncDecl name params _ body) = do
  let paramNames = [v | Param v _ <- params]
      paramTypes = [(v, t) | Param v t <- params]
  stmts <- runCgM rf $ do
    mapM_ (uncurry trackVar) paramTypes -- preenche varTypes
    mapM_ codegenStmt body
  pure (IR.FuncDef name paramNames (foldStmts stmts))
```

Os tipos dos parâmetros são pré-carregados em `varTypes` antes de iniciar a geração do corpo. Sem isso, um acesso `p.x` dentro da função não conseguiria determinar o tipo de `p`.

Geração de Comandos: Novos Casos

Declaração de variável tipada (rastrea o tipo para uso futuro):

```
codegenStmt (SDecl v t e) = do
  ir <- codegenExp e
  emit (IR.MOVE (IR.TEMP v) ir)
  trackVar v t -- insere v -> t em varTypes
```

Atribuição a campo (implementa a regra $S[x.f := e]$):

```
codegenStmt (SFieldAssign v f e) = do
  vts <- gets varTypes
  case Map.lookup v vts of
    Nothing -> throwError ("Unknown variable type for " ++ v)
    Just (TRecord rname) -> do
      i <- fieldIndex rname f -- índice i do campo f em R
      rhs <- codegenExp e
      let addr = IR.BINOP IR.BAdd (IR.TEMP v) (IR.CONST (i * wordSize))
      emit (IR.MOVE (IR.MEM addr) rhs)
      Just t -> throwError (v ++ " is not a record (type: " ++ show t ++ ")")
```

Retorno com e sem valor:

```
codegenStmt (SReturn Nothing) = emit (IR.RETURN [])
codegenStmt (SReturn (Just e)) = do
  ir <- codegenExp e
  emit (IR.RETURN [ir])
```

Chamada de função como comando:

```
codegenStmt (SCall fname args) = do
  argIRs <- mapM codegenExp args
  emit (IR.EXP (IR.CALL (IR.NAME fname) argIRs))
```

O valor de retorno é descartado via `EXP`, tal como em `TWhile`.

Geração de Expressões: Novos Casos

Acesso a campo (implementa a regra $\mathcal{E}[e.f]$, restrito a `EVar` para simplificar):

```

codegenExp (EField (EVar v) f) = do
  vts <- gets varTypes
  case Map.lookup v vts of
    Nothing -> throwError ("Unknown variable type for " ++ v)
    Just (TRecord rname) -> do
      i <- fieldIndex rname f
      let addr = IR.BINOP IR.BAdd (IR.TEMP v)
                    (IR.CONST (i * wordSize))
      pure (IR.MEM addr)
      Just t -> throwError (v ++ " has non-record type " ++ show t)

codegenExp (EField _ _) =
  throwError "Field access on non-variable not supported in IR codegen"

```

A restrição a EVar é conservadora mas correta para todos os programas produzidos pelo *front-end*: o verificador de tipos garante que o receptor de um acesso a campo é bem tipado, e a expressão mais comum é de facto uma variável. Expressões aninhadas como $f().x$ exigiriam um temporário intermediário.

Construção de registro (implementa a regra $\mathcal{E}[\text{new } R\{\dots\}]$):

```

codegenExp (ENew rname initFields) = do
  rf <- gets recFields
  case Map.lookup rname rf of
    Nothing -> throwError ("Unknown record type: " ++ rname)
    Just flds -> do
      let n = length flds
      t <- freshTemp
      -- avalia campos na ordem de declaracao (nao na ordem fornecida)
      fieldVals <- mapM (lookupFieldVal initFields) flds
      let allocStmt = IR.MOVE (IR.TEMP t)
                            (IR.CALL (IR.NAME "alloc") [IR.CONST n])
          fieldStmts = [ IR.MOVE
                        (IR.MEM (IR.BINOP IR.BAdd
                                         (IR.TEMP t) (IR.CONST (i * wordSize)))) v
                        | (i, v) <- zip [0..] fieldVals ]
      pure (IR.ESEQ (foldStmts (allocStmt : fieldStmts)) (IR.TEMP t))
  where
    lookupFieldVal pairs f =
      case lookup f pairs of
        Nothing -> throwError ("Missing field " ++ f)
        Just e -> codegenExp e

```

Três aspectos merecem atenção:

- Os campos são avaliados na **ordem de declaração** de R (lista `flds`), não na ordem em que o programador os escreveu. Isso garante que o deslocamento $i \cdot w$ está correto independentemente da ordem de inicialização.
- O temporário t é gerado por `freshTemp` (contador `nextTemp`), que produz nomes como `_t0`, `_t1`, distintos dos nomes de variáveis do programa.
- A chamada a `alloc` recebe `CONST(n)`, o número de campos, e não o número de bytes. O interpretador IRT multiplica por w internamente.

17.10.5 Exemplo Completo

Considere o programa TImp:

```
record Point { x : int; y : int; }
fn soma(p : Point) : int {
  return p.x + p.y;
}
var q : Point = new Point { x = 3, y = 4 };
print soma(q);
```

Após `compileTImp`, o código IRT gerado é:

```
func soma(p) {
  RETURN(BINOP(ADD,
    MEM(BINOP(ADD, TEMP(p), CONST(0))), -- p.x (indice 0)
    MEM(BINOP(ADD, TEMP(p), CONST(8)))) -- p.y (indice 1)
)
func main() {
  SEQ(
    MOVE(TEMP(q), -- var q = new Point{x=3,y=4}
      ESEQ(
        SEQ(MOVE(TEMP(_t0), CALL(NAME(alloc), CONST(2))),
          SEQ(MOVE(MEM(BINOP(ADD, TEMP(_t0), CONST(0))), CONST(3)),
            MOVE(MEM(BINOP(ADD, TEMP(_t0), CONST(8))), CONST(4)))),
        TEMP(_t0))),
    EXP(CALL(NAME(print), -- print soma(q)
      CALL(NAME(soma), TEMP(q))))
}
```

Pontos a observar:

- A função `soma` produz um `FuncDef` independente com parâmetro `p`. Os acessos `p.x` e `p.y` tornam-se `MEM` com deslocamentos 0 e 8.
- O registro é alocado com `alloc(2)` e o endereço base fica em `TEMP(_t0)`. Os dois campos são inicializados em seguida.
- A chamada `soma(q)` passa o endereço do registro (o valor inteiro em `TEMP(q)`) como argumento; dentro de `soma`, `TEMP(p)` contém esse endereço.
- O resultado de `soma(q)` é usado diretamente como argumento de `print`, sem temporário intermediário (a canonicalização extrairia essa chamada embutida se necessário).

17.11 Conclusão

A representação intermediária em árvore (IRT) introduzida neste capítulo satisfaz os critérios de simplicidade, independência de máquina e suporte a transformações enunciados na Seção 17.2. A semântica big-step formaliza o comportamento de cada construtor, permitindo raciocinar com rigor sobre a correção das traduções. As regras de tradução dirigida por sintaxe cobrem todas as construções de `TWhile` e `TImp` e são estruturalmente indutivas sobre a AST, o que facilita tanto a implementação quanto a prova de corretude.

Os próximos passos na cadeia de compilação são:

- **Canonicalização e formação de blocos básicos:** preparar a IRT linearizada para análise de fluxo.
- **Otimizações sobre a RI:** propagação de constantes, eliminação de código morto, movimentação de código invariante.

- **Seleção de instruções:** mapear construções da IRT para instruções da arquitetura-alvo (e.g., x86-64 ou WebAssembly).
- **Alocação de registradores:** decidir, para cada temporário, se ele vive em um registrador físico ou em uma posição da pilha.